电商销量预测系统 · 项目开发实施方案

版本 V1.0 | 2026-10-13 | 状态：☑ 已完成（`EP-0` ~ `EP-10` 全部收口，11/11；交付物清单 §12 已 100% 齐备）

目 录

0. 文档信息与阅读指南 4

0.1 文档信息 4

0.2 本文档回答什么 / 不回答什么 4

0.3 与既有文档的关系（📌 冲突仲裁规则） 5

0.4 本文档新增编号体系 5

0.5 标记说明（承接上游，保持一致） 6

0.6 立项决策快照（📌 已确认，7 条） 6

1. 建设目标与范围界定 7

1.1 「前台」与「后台」的准确定义（澄清） 7

1.2 本期范围 8

1.3 明确不做（范围红线） 9

1.4 交付形态与验收方式 10

2. 现状盘点与复用基线 11

2.1 可直接复用的资产（✅ 已实测） 11

2.2 待新建清单（❌ 当前不存在） 12

2.3 环境与技术基线（📌 版本锁定，不改写） 14

2.4 已实测的性能与质量基线（⏱ 全部为真机数据） 15

3. 总体架构与工程落地 17
3.1 逻辑架构 17
3.2 目标工程目录（落地版） 18
3.3 架构红线（🔒 违反即验收不通过） 19
3.4 主干数据流（端到端） 19
4. 功能落地映射（页面—路由—接口—需求） 20
4.1 前台（未登录可见，3 页） 20
4.2 后台（登录后，8 页） 21
4.3 管理端（CH-01 新增，2 页，仅 role='admin' 可见） 22
4.4 接口总清单（既有 21 + 新增 9 = 30 个） 23
4.5 需求—页面—接口 三向追溯（抽样，完整表见 PRD §3 与 UI/UX §6.3） 25
5. 变更项 CH-01：管理员端（B 标准运营型） 26
5.1 变更必要性与依据 26
5.2 影响面清单 27
5.3 数据库变更（⏱ 已全部实测验证，不是纸面结论） 27
5.4 数据隔离红线的例外设计（🔒 安全核心） 31
5.5 管理端接口清单（API-22 ~ API-30） 33
5.6 管理端页面规格（复用模板 A，不新增视觉语言） 36
5.7 上游文档回写清单（变更流程强制要求） 37
6. 实施阶段划分 38
6.1 阶段划分原则 38
6.2 与《开发方案 V1.1》§7.2 阶段排期的映射 39
6.3 阶段总表（EP-0 ~ EP-10，共 11 个阶段） 40
6.4 执行进度登记（滚动更新） 41
7. 各阶段任务分解与完成门槛（WBS） 47
7.1 EP-0 工程基建与设计系统移植 48
7.2 EP-1 数据库落地与数据访问层 49

7.3 EP-2 认证与用户 50
7.4 EP-3 前台产品首页与全局框架 52
7.5 EP-4 数据集上传与校验 55
7.6 EP-5 预测任务编排 ★（全项目最高风险阶段） 61
7.7 EP-6 结果可视化与导出 ★（核心交付页） 67
7.8 EP-7 历史记录与对比 + 工作台 72
7.9 EP-8 管理端（CH-01） 78
7.10 EP-9 集成联调与测试 84
7.11 EP-10 文档与答辩交付 85
8. 里程碑与 Go/No-Go 判定 87
9. 质量保障与验收 88
9.1 校验脚本矩阵（🔒 全部为「可复现断言」，非人工目视） 88
9.2 交付前验收检查清单（15 项，逐项确认） 95
9.3 需求覆盖自检 98
10. 风险登记册 98
11. 协作方式与变更管理 101
11.1 单人开发的节奏（📌 决策 4：不排日历） 101
11.2 变更管理（沿用技术文档 §8.6） 102
11.3 阶段交付方式 103
12. 交付物清单 103
附录 A：立项决策记录（📌 7 条已确认 + 2 条管理端细化） 106
附录 B：待确认事项清单（QD-*） 107
附录 C：执行顺序速查（一页纸） 109

0. 文档信息与阅读指南

0.1 文档信息

项	值
文档名称	电商销量预测系统 · 项目开发实施方案
版本	**V1.0**（已确认；滚动追加各阶段执行记录）
编制日期	2026-10-09
文档定位	**施工图**：把已有的 5 份设计文档转成「按什么顺序、做什么、做到什么程度算完成」
服务对象	项目负责人（决策与验收）、开发者（按阶段执行）
上游依据	任务书 V1.0.0、`PRD V1.0`、开发方案文档 `V1.1`、`UI/UX` 设计文档 `V1.0`、开发技术文档 `V1.0`、数据库设计文档 `V1.0`
下游产出	可运行的 Web 应用（前台展示 + 后台管理）、全套交付物

0.2 本文档回答什么 / 不回答什么

回答 ✓	不回答 ✗
按什么顺序做（阶段划分、依赖、门槛）	做什么功能（→ **PRD V1.0**）
每个阶段做什么、产出什么、怎么算做完	长什么样（→ **UI/UX 设计文档 V1.0**）
数据库、接口、工程结构如何落地	具体建表语句与字段字典（→ **数据库设计文档 V1.0**）
哪些资产可直接复用、哪些必须新建	接口报文细节（→ **开发技术文档 V1.0 §5**）
本次立项的增量（管理员端）如何设计与收敛	算法参数与调参过程（→ `app/algo/` 源码 + 开发方案附录 D）
风险、质量门槛、交付物清单	——

本文档不重复上游内容，只做「引用 + 排程 + 落地」三件事。 凡涉及具体取值（接口字段、表结构、视觉令牌），一律以上游文档为准，本文档只给编号与章节指引。

0.3 与既有文档的关系（📌 冲突仲裁规则）

三级优先（原文承接自《开发技术文档 V1.0》§0.4，不再重复讨论）：

1. **功能范围与验收标准** → 以 **PRD V1.0** 为准
2. **技术口径与实现选择** → 以 **开发技术文档 V1.0** 为准
3. **界面与视觉** → 以 **UI/UX 设计文档 V1.0** 为准
4. **数据库结构** → 以 **数据库设计文档 V1.0** + `code/db/schema.sql` 为准（两者冲突时以 `schema.sql` 为准）

本文档的优先级：仅在「**执行顺序、阶段门槛、增量变更（§5）**」这一维度上具有最高效力。本文档与上游在功能或技术口径上冲突时，**以上游为准，并视为本文档缺陷需修正。**

> 📌 **编号不重起**：本文档续用上游 `FR-*` / `NFR-*` / `UX-P*` / `API-*` / `TQ-*` / `TD-*` 编号体系，新增编号见 §0.4。

0.4 本文档新增编号体系

前缀	含义	示例
`EP-<序号>`	实施阶段（Execution Phase）	`EP-4` 预测任务编排
`WS-<阶段>-<序号>`	阶段内工作项（Work Stream）	`WS-4-03` 任务状态机实现
`CH-<序号>`	变更项（Change Request）	`CH-01` 引入管理员端
`QD-<序号>`	本方案待确认问题（Decision Queue，接续 `TQ-*`）	`QD-02` action 值扩展方式
`RG-<序号>`	风险条目（Risk）	`RG-05` 管理端破坏隔离红线
`DOD-<阶段>`	完成定义（Definition of Done）	`DOD-EP4`
`MS-<序号>`	里程碑（Milestone）	`MS-2` 主干路径端到端跑通

0.5 标记说明（承接上游，保持一致）

标记	含义
🔒	**硬约束。**违反即视为未完成，属验收红线
📌	**已确认决策。**已拍板，不再讨论
⚠️	**建议值 / 待确认。**已给出默认值，需负责人拍板
⏱	**本机实测数据。**非估算，附复现方式
🚧	**技术债。**本次不修，已知且已披露
【待补充：xxx】	信息缺失占位符。**禁止用猜测值填充**

0.6 立项决策快照（📌 已确认，7 条）

本次立项前，负责人已就 7 个决定方案骨架的问题给出明确答复。**以下为编码期的执行依据，不再讨论：**

#	决策项	结论	影响
1	「前台企业官网」的范围	📌 **沿用现有 11 页设计。**前台 = `UX-P01` 产品首页 + `UX-P02` 登录 + `UX-P03` 注册；后台 = `UX-P04`~`UX-P11`	不新增前端页面设计，100% 有 UI/UX 依据
2	是否需要管理员端	📌 **需要**，档位 **B 标准运营型**	新增变更项 `CH-01`（见 §5），关闭 PRD `Q13`
3	管理员账号如何产生	📌 `seed.sql` **内置种子账号**（`role='admin'`，幂等）	不做额外注册入口，零新增接口
4	团队人数与工期	📌 **单人开发，不排日历**	阶段只给顺序与门槛，不承诺日期（对齐开发方案 `Q1`）
5	本期实现范围	📌 **P0 全量（55 项）+ P1 全量（14 项）**	P2（6 项）/ P3（2 项）不排期，与 PRD「P2 无条件

#	决策项	结论	影响
			让路」一致
6	本轮交付物	📌 **只出实施方案，不写代码**	待负责人回复「已确认，执行下一步」后进入 EP-0
7	技术栈	📌 **不改动**（FastAPI + React，版本锁定见技术文档 §1.1）	编码期不得更换框架或组件库

> ⚠ **第 2 条是本次唯一的范围扩张。**它带来 1 处数据库变更、9 个接口、2 个页面，并使 PRD 的数据隔离红线（NFR-SEC-02）出现**第一个显式例外。**相关设计与风险控制在 §5 与 §10（RG-05）中完整展开。

---

1. 建设目标与范围界定

1.1 「前台」与「后台」的准确定义（澄清）

需求原文写的是「前台展示企业官网系统 + 后台管理系统」。经与负责人确认，**这两个词在现有文档中的准确对应关系如下**（📌 决策 1）：

```
┌──────────────────────── 前台（未登录可见）────────────────────────┐
│  UX-P01 产品首页（模板 D 营销页）     价值主张 / 硬指标 / 五步流程      │
│  UX-P02 登录                          邮箱密码 + 限流冷却            │
│  UX-P03 注册                          密码强度实时校验               │
└──────────────────────────┬──────────────────────────────────────┘
                           │ 登录成功 / 游客直通
                           ▼
┌──────────────────────── 后台（登录后）────────────────────────┐
│  UX-P04 工作台          概览 + 快速入口 + 最近任务                  │
│  UX-P05 上传数据 ─┐                                                │
│  UX-P06 参数配置  ├─ 主流程 4 步（≤ 5 步硬约束 NFR-USE-01）          │
│  UX-P07 执行进度  │                                                │
│  UX-P08 预测结果 ★┘ 核心交付页                                      │
```

```
│  UX-P09 历史任务 ─┐                                                  │
│  UX-P10 任务详情  ├─ 记录类                                          │
│  UX-P11 跨任务对比┘                                                  │
│  AD-P01 运营看板      ┐                                              │
│  AD-P02 用户与任务管理┘ — CH-01 新增（仅 role='admin' 可见）          │
└──────────────────────────────────────────────────────────────────────┘
```

关键口径（避免歧义）：

说法	准确含义	依据
「前台企业官网」	`UX-P01` **产品首页**——单页营销落地页，讲价值主张、硬指标、五步流程、数据集字段说明。**不是**多页面企业站（无公司介绍 / 新闻 / 招聘等页面）	UI/UX §6.1、§7.1；模板 D
「后台管理系统」	**登录后的 8 个业务页**（`UX-P04`~`UX-P11`），即「工作台 + 预测主流程 + 记录」。**不是**传统意义的运维后台	UI/UX §6.1、§6.4 模板 A
「管理端」（`CH-01` 新增）	上述后台之上的**管理员专属页面**（`AD-P01`/`AD-P02`），仅 `role='admin'` 可见	本文档 §5
三者关系	前台 ⊂ 统一前端工程，后台 ⊂ 统一前端工程，管理端 ⊂ 统一前端工程。**同一个 React 工程、同一套设计令牌、同一套接口基址**	§3.2

📌 全站页面归为 **4 种模板**（A 应用页 / B 主内容+右栏 / C 认证页 / D 营销页），管理端两页**复用模板 A**，不引入新的视觉语言（`QD-01`、`RG-11`）。

1.2 本期范围

1.2.1 功能范围（按 PRD 优先级）

优先级	数量	本期	说明
P0 阻塞级	55 项	✅ 全量实现	主干路径必须 100% 走通
P1 重要级	14 项	✅ 全量实现	📌 决策 5
P2 二期	6 项	❌ 不排期	与 P0/P1 冲突时无条件让路
P3 不排期	2 项	❌ 不排期	——
`CH-01` **管理员端**	增量	✅ 实现（B 标准运营型）	📌 决策 2，见 §5

1.2.2 功能模块覆盖矩阵（6 大模块 + 管理端）

模块	PRD 编号域	承载页面	本期状态
用户管理	`FR-USR-01` ~ `FR-USR-10`	P02 / P03	✅
数据上传	`FR-UPL-01` ~ `FR-UPL-15`	P05	✅
特征工程	`FR-FEA-01` ~ `FR-FEA-11`	P06 / P08 / P10	✅（中间产物不单独成页）
模型预测	`FR-MOD-01` ~ `FR-MOD-15`	P06 / P07 / P08	✅
结果可视化	`FR-VIS-01` ~ `FR-VIS-10`	P08	✅
历史记录	`FR-HIS-01` ~ `FR-HIS-06`	P09 / P10 / P11	✅
系统与错误	`FR-SYS-01` ~ `FR-SYS-09`	P07 / P08 / P10 + 全局	✅
管理端（新增）	`FR-ADM-*`【待补充：编号待 PRD 侧定义】	AD-P01 / AD-P02	✅ `CH-01`

1.3 明确不做（范围红线）

以下为**本期明确排除**项，任何一项的实现都视为超范围（沿用 PRD §1.4 与开发方案 §1.4）：

#	不做的事	理由

#	不做的事	理由
1	✕ 模型在线增量训练 / 自动重训调度	📌 纯预训练推理（PRD `Q6` 已确认）
2	✕ 多角色 RBAC / 组织架构 / 权限矩阵	本期仅 `user` / `admin` 两值，不含角色-权限-资源三级模型
3	✕ 多租户 SaaS 化	——
4	✕ Docker 容器化部署	单机 Windows 场景，`TQ-5` 保留
5	✕ Redis 缓存	单进程部署，内存方案足够（`TQ-2` 仍开放）
6	✕ 第三方登录（微信 / 手机号验证码）	PRD §3.1 未要求
7	✕ 预测结果截图导出（P2）	优先级不排期
8	✕ 实时流式预测 / 定时批量预测	本期为「按需单次任务」
9	✕ 多页企业官网（公司介绍 / 新闻 / 招聘 / 案例）	前台定位为单页产品首页（`UX-P01`）
10	✕ 引入 ORM 迁移框架（Alembic 等）	`TD-DB-03`：靠 `PRAGMA user_version` 手工管理

1.4 交付形态与验收方式

项	形态
应用形态	**B/S 前后端分离单机 Web 应用**（浏览器访问，无需安装客户端）
开发期访问	前端 `http://localhost:5173`（Vite dev），后端 `http://127.0.0.1:8000`，Vite proxy 转发 `/api`
交付期访问	**Nginx 同源部署**：`/` → `web/dist` 静态文件；`/api`、`/health` → 反代 `:8000`（技术文档 §10.1、§10.3）
验收方式	现场演示「上传 → 预测 → 看结果 → 导出」全链路（**5 分钟内**跑完）；另演示 1 个失败分支
数据环境	使用内置示例数据集（`outputs/models/demo_sales.csv`，113 序列）+ 菜鸟完整数据集压测

2. 现状盘点与复用基线

本章是**方案可信度的基础**：先把「已经有什么」核实清楚，才能准确判断「还要做什么」。以下资产状态均为 2026-10-09 实测核对。

2.1 可直接复用的资产（✅ 已实测）

资产	位置	规模	验证状态
算法层（9 模块）	`app/algo/`	`config` `preprocess` `features` `classify` `model` `train` `infer` `fallback` `metrics`	✅ `self_check.py` **69/69 通过**
数据库 DDL	`code/db/schema.sql`	309 行；8 表 / 120 列 / 10 显式索引 + 2 自动索引 / 3 触发器 / 1 视图 / 14 处 CHECK	✅ `verify_schema.py` 通过；幂等；外键与部分唯一索引实测生效
种子数据	`code/db/seed.sql`	模型版本注册 + 游客哨兵（`id=0`）幂等补插	✅ 幂等
Pydantic 模型	`code/backend/schemas.py`	**33 个** v2 模型（含 bcrypt 72 字节校验）	✅ `verify_schemas.py` **57/57**
前端类型定义	`code/frontend/types.ts`	**30 个**接口类型 + 5 个类型别名	✅ `verify_types.mjs` 通过
前端接口封装	`code/frontend/api-client.ts`	Axios 实例 + 全量 API 函数 + 拦截器	✅ 同上
模型权重	`outputs/models/lstm_v1.pt`	131,161 B	✅ 可加载、可复现
训练报告	`outputs/models/lstm_v1.report.json`	7,813 B	✅ 超参与验证指标齐备
示例数据集	`outputs/models/demo_sales.csv`	490,291 B	✅ 113 序列
评估报告（真值来源）	`outputs/eval_report.json` + `eval_series.json`	1,481 B / 260,485 B	✅ ⏱ 真机产出
高保真原型	`prototype/`	12 个 HTML（11 产品页 + README 非产品页）+	✅ 可直接打开，作为**视觉与交互基线**

资产	位置	规模	验证状态
		`app.css`（43,931 B，**56 个设计令牌**）+ `charts.js`（12,911 B）+ `demo-data.js`（162,006 B）	
设计文档	`docs/`	PRD / UI/UX / 开发技术 / 数据库设计 / 开发方案，共 5 份	✅ 已交叉校验
数据库设计图	`docs/assets/db-design/`	7 张 PNG（4 张 ER + 3 张流程），源文件 4 `.mmd` + 3 `.svg`	✅ 已嵌入文档
校验与工具脚本	`scripts/`	**21 个**（含 `a11y_audit.py`、`verify_*.py/mjs`、`sync_prototype_data.py`、`shoot_*.mjs`、`train_and_register.py`、`eval_report.py`）	✅ 多数已跑通
本机运行环境	conda `deploy` 环境	Python 3.12.14 + torch 2.9.1+cu128 + pandas/numpy/fastapi	✅ 已就绪

2.2 待新建清单（✕ 当前不存在）

后端 Web 层（`app/` 下除 `algo/` 外全部为空）

目录 / 文件	内容	依据
`app/main.py`	FastAPI 实例、中间件链、路由挂载、启动钩子	技术文档 §4.1
`app/core/`	`config.py`（Settings + 路径常量）、`security.py`（bcrypt + JWT）、`deps.py`（依赖注入）、`errors.py`（异常类 + 处理器）、`logging.py`	技术文档 §4.2、§4.4、§4.5
`app/api/v1/`	`auth.py` `datasets.py`	技术文档 §2.2、§5

目录 / 文件	内容	依据
	predictions.py history.py system.py + admin.py（新增）+ router.py	
app/models/	8 个 ORM 实体，与 schema.sql 一一对应	技术文档 §2.2
app/schemas/	由 code/backend/schemas.py 拆为 common auth dataset task result history（+ admin）	技术文档 §2.2
app/services/	auth_service dataset_service predict_service history_service export_service（+ admin_service）	技术文档 §2.2
app/dao/	纯 CRUD，不含业务判断	技术文档 §2.2

前端工程（web/ 目录整体不存在）

项	内容
工程骨架	Vite 5.4 + React 18.3 + TypeScript 5.4 + TailwindCSS 3.4 + AntD 5.21 + ECharts 5.5 + Axios 1.7 + React Router 6.26 + Zustand 4.5
设计令牌移植	theme/tokens.css（56 个令牌**原样保留 oklch()**）+ antd-token.ts + echarts-theme.ts
布局	4 个模板：AppLayout / AsideLayout / AuthLayout / MarketingLayout
页面	11 个既有页（Index Login Register Dashboard Upload Params Running Result History HistoryDetail Compare）+ **2 个管理端页**（AdminOverview AdminManage）
组件	common/（免责声明、错误边界、空状态、骨架屏）、charts/（LineCompare / Histogram / Sparkline）、metric/（MetricCard / CoverageBar / VerdictBar）、upload/（DropZone / ValidationPanel / PreviewTable）
状态	useAuthStore / useTaskStore / useThemeStore

运行期数据目录（`data/`）

目录	用途
`data/app.db`	SQLite 数据库文件（`journal_mode=WAL`）
`data/uploads/u{user_id}/`	上传的原始 CSV（`u0` = 游客）
`data/exports/`	正式导出产物（TTL 7 天）
`data/exports/tmp/`	临时错误明细（TTL 1 小时，`JT-11`）

管理端增量（`CH-01`）

项	内容
数据库	`users.role` 列（+1 列）；`operation_logs.action` CHECK 扩展（8 类）
接口	`API-22` ~ `API-30`，共 **9 个**
页面	`AD-P01` 运营看板、`AD-P02` 用户与任务管理
代码	`app/api/v1/admin.py`、`app/services/admin_service.py`、`app/core/deps.py` 增 `get_current_admin`

2.3 环境与技术基线（📌 版本锁定，不改写）

完整选型表见《开发技术文档 V1.0》§1.1，此处仅列关键版本，编码期不得擅自变更：

层	技术	版本
运行环境	Python	**3.12.14**（conda `deploy` 环境，路径 `C:\ProgramData\anaconda3\envs\deploy\python.exe`）
后端	FastAPI / Uvicorn / Pydantic	**0.141.1 / 0.52.4 / 2.9.2**
ORM	SQLAlchemy	**2.0.36**
数据库	SQLite	内置 **3.45+**

层	技术	版本
安全	bcrypt / PyJWT	**5.0.0 / 2.9.0**
算法	torch / pandas / numpy	**2.9.1+cu128 / 2.3.3 / 2.5.3**
导出	openpyxl	**3.1.5**
前端	React / TypeScript / Vite	**18.3.1 / 5.4.5 / 5.4.x**
前端 UI	Ant Design / TailwindCSS	**5.21.x / 3.4.x**
前端图表	ECharts	**5.5.x**（`UI/UX §12.1` 已确认，`Q16`）
前端其余	Axios / React Router / Zustand	**1.7.x / 6.26.x / 4.5.x**

🔒 **依赖分组纪律**（技术文档 §1.2）：`torch` / `pandas` / `numpy` 三者版本必须与离线训练环境**完全一致**，否则浮点行为差异会导致预测值漂移且极难定位。

🔒 **禁止 passlib**：passlib 已停止维护，与 bcrypt ≥ 4.1 存在 `__about__` 属性兼容问题，bcrypt 5.x 已彻底移除该属性 → **直接使用 `bcrypt` 库。**

端口与进程约定（技术文档 §2.7）：

服务	地址	说明
后端 Uvicorn	`127.0.0.1:8000`	单进程（SQLite 写锁与模型单例均适合单进程）
前端 Vite dev	`localhost:5173`	开发期，proxy `/api` → `:8000`
Nginx（交付期）	`80` / `443`	托管 `web/dist` + 反代 `/api`

2.4 已实测的性能与质量基线（⏱ 全部为真机数据）

算法质量（来源：`outputs/eval_report.json`，`horizon=14`）

指标	实测值	阈值	判定

指标	实测值	阈值	判定
销量加权 MAPE	**14.26%**	🔒 ≤ 20%	☑ 达标
加权 MAE	1.32	——	参考
加权 RMSE	3.37	——	参考
移动平均基线 MAE	1.80	——	对比基准
移动平均基线 RMSE	4.65	——	对比基准
MAE 相对基线降低	**26.42%**	🔒 ≥ 15%	☑ 达标
RMSE 相对基线降低	**27.52%**	🔒 ≥ 15%	☑ 达标
达标序列占比	**83.95%**（68 / 81）	≥ 60%	☑ 达标

覆盖率（同一报告）：L1 **81 序列（71.68%）** / L2 **32 序列（28.32%）** / L3 **0** / 合计 **113**，可预测率 100%。

分阶段耗时（`timing_ms`，单位 ms）：

阶段	耗时	占比
预处理	675	8.6%
特征工程	763	9.7%
推理	**21**	**0.3%**
指标计算	473	6.0%
端到端合计	**7,873（7.9 s）**	🔒 ≤ 60,000 ms，**余量 7.6 倍**

其他基准：

项	实测值	来源
数据处理（864,772 行）	≈ **2.3 s**	开发方案附录 D
训练（GPU / CPU）	**51 s / 80 s**	`outputs/train_gpu.txt`
在线推理	**11–17 ms**	算法层自检
算法层自检	**69 / 69 通过**	`scripts/self_check.py`

项	实测值	来源
本机硬件	8 逻辑核 / 23.8 GB 内存 / C 盘可用 ~29 GB / MX450 2048 MiB（CC 7.5，CUDA 12.9）	开发方案附录 D

🔒 **数据必须写在工作区内**：C 盘可用空间约 29 GB，`data/` 与 `outputs/` 全部位于工作区内，**禁止写入系统盘**，所有路径以 `BASE_DIR` 为基准拼接，不得硬编码绝对路径。

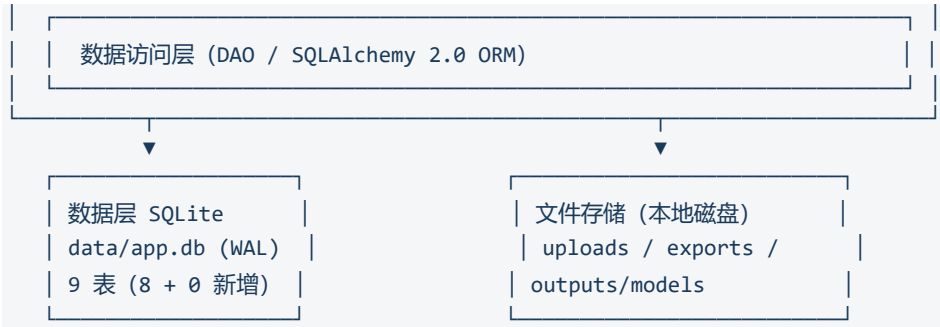
---

3. 总体架构与工程落地

3.1 逻辑架构

沿用《开发方案文档 V1.1》§4.1 的 B/S 前后端分离架构，**不新增架构层。**管理端（`CH-01`）是**应用层内的一组受保护路由**，不是新的架构层：





📌 CH-01 **不新增数据表**：users.role 是加列，AD-P01/AD-P02 读的是既有 8 张表。表数仍为 **8**（+1 视图）。

3.2 目标工程目录（落地版）

在《开发技术文档 V1.0》§2.1 的骨架上补入 CH-01 与运行期目录：

```
销量预测/
├── app/                                  # 后端（FastAPI）
│   ├── main.py                           # 🆕 应用入口
│   ├── core/                             # 🆕 config / security / deps / errors / logging
│   ├── api/v1/                           # 🆕 auth / datasets / predictions / history / system /
admin ✚ / router
│   ├── models/                           # 🆕 8 个 ORM 实体
│   ├── schemas/                          # 🆕 由 code/backend/schemas.py 拆分（+ admin）
│   ├── services/                         # 🆕 auth / dataset / predict / history / export / admin
✚
│   ├── dao/                              # 🆕 纯 CRUD
│   └── algo/                             # ✅ 已有，9 模块，禁止改动业务语义
│
├── web/                                  # 🆕 前端（React + Vite + TS）整目录新建
│   ├── src/theme/tokens.css              # 🔧 56 个设计令牌原样移植（oklch 不改写）
│   ├── src/layouts/                      # 4 模板：App / Aside / Auth / Marketing
│   ├── src/pages/                        # 11 既有页 + 2 管理端页 ✚
│   ├── src/components/                   # common / charts / metric / upload
│   ├── src/store/                        # useAuthStore / useTaskStore / useThemeStore
│   ├── src/api/                          # 基于 code/frontend/api-client.ts 落地
│   ├── src/types/                        # 基于 code/frontend/types.ts 落地
│   └── vite.config.ts / tailwind.config.js / package.json
│
├── data/                                 # 🆕 运行期数据（必须在工作区内）
│   ├── app.db                            # SQLite (WAL)
│   ├── uploads/u{user_id}/               # 原始 CSV，u0 = 游客
│   └── exports/ + exports/tmp/           # 导出产物 + 临时错误明细
```

```
│
├── outputs/                          # ✅ 已有交付物与算法产物
│   └── models/                       # lstm_v1.pt / report.json / demo_sales.csv
├── code/                             # ✅ 配套代码资产（schema.sql / seed.sql / schemas.py /
types.ts / api-client.ts）
├── docs/                             # ✅ 6 份文档（新增本文档）+ assets/db-design/
├── prototype/                        # ✅ 高保真原型（设计基线，不参与构建）
└── scripts/                          # ✅ 21 个离线脚本（训练 / 评估 / 校验 / 截图）
```

3.3 架构红线（🔒 违反即验收不通过）

#	红线	说明	依据
1	`algo/` **不得** `import fastapi` / `sqlalchemy`	保证可被离线脚本零 Web 依赖复用	开发方案架构红线 1
2	在线推理与离线训练**共用同一份** `algo/` 代码	唯一差别是「模型从哪来」，避免预处理不一致	开发方案架构红线 2
3	分层依赖**单向**：`api → services → dao → models`，`services ↘ algo`	禁止反向依赖	技术文档 `BE-04`
4	`services/` **不得** `import algo.train`；`api/` **不得**直接 `import dao`；`dao/` **不得** `import algo`	服务进程内存中不应存在训练逻辑；业务规则不得散落路由层	技术文档 `BE-04`
5	管理端是隔离红线的**唯一显式例外**，且必须强制 `role='admin'` + 全操作审计	见 §5.4；无此约束即为安全缺陷	本文档 `RG-05`

建议用 `ruff` 的 `flake8-tidy-imports`（`TID` 规则）在提交前自动检查红线 1~4（技术文档 §8.3）。

3.4 主干数据流（端到端）

```
[前台] UX-P01 产品首页 ──「免费试用」──▶ UX-P02/P03 登录注册（或游客直通）
                                                  │
[后台]                                            ▼
```



硬指标贯穿点：NFR-USE-01 主干路径 ≤ 5 步；NFR-PERF-01 端到端 ≤ 60 s；🔒 加权 MAPE ≤ 20% 且 MAE/RMSE 相对基线降低 ≥ 15%。

---

4. 功能落地映射（页面—路由—接口—需求）

4.1 前台（未登录可见，3 页）

页面	文件	路由	模板	接口依赖	需求
UX-P01 产品首页	index.html	/	**D 营销页**	API-21（健康检查，用于展示系	价值主张 / 硬指标 / 五步流程 /

页面	文件	路由	模板	接口依赖	需求
				统状态）	数据集字段说明
UX-P02 登录	login.html	/login	C 认证页	API-02 登录	FR-USR-05 / 06 / 07；含登录中、密码错误、限流冷却三态
UX-P03 注册	register.html	/register	C 认证页	API-01 注册	FR-USR-01 ~ 04；密码强度实时校验（含 72 **字节**上限，JT-12）

📌 **UX-P01 的数字必须来自算法产出**：硬指标展示值应从 outputs/eval_report.json（或后端接口）读取，**禁止手写**。本项目历史上出现过「原型一套数字、算法另一套数字」的问题（技术文档 GIT-7），**重训模型后必须重跑 train_and_register.py 与 eval_report.py**，否则展示值落后于模型。见 QD-06。

4.2 后台（登录后，8 页）

页面	文件	路由	模板	主要接口	需求
UX-P04 工作台	dashboard.html	/dashboard	A	API-17 历史列表	FR-HIS-01 / FR-USR-08 / NFR-USE-01 / NFR-USE-04 / FR-SYS-05
UX-P05 上传数据	upload.html	/upload	**B**	API-06 07 08 10 11	FR-UPL-01 ~ 15 全量；5 类校验错误结构化展示
UX-P06 参数配置	params.html	/params/:datasetId	**B**	API-09 12	FR-MOD-01~04、FR-FEA-02/10、FR-MOD-12；粒度只读、**无重训选项**
UX-P07 执行进度	running.html	/running/:taskId	**B**	API-13 轮询	FR-MOD-09/10/12/13；60 秒预算时间

页面	文件	路由	模板	主要接口	需求
					轴、GPU→CPU 回退、超时软失败
UX-P08 预测结果 ★	result.html	/result/:taskId	**B**	API-14 15 16	FR-VIS-01~09 + FR-MOD-06/11/13/14；**核心交付页**
UX-P09 历史任务	history.html	/history	A	API-17 20	FR-HIS-01 ~ 06；筛选、空状态、删除二次确认
UX-P10 任务详情	history-detail.html	/history/:taskId	A	API-19	FR-HIS-02 + FR-SYS-07/08；参数快照冻结、失败详情、阶段耗时回放
UX-P11 跨任务对比	compare.html	/compare	A	API-18	FR-HIS-04；最多 4 任务并排、最优值高亮、数据不一致警告

🔒 **路由注册顺序**（BE-39）：`POST /history/compare` 必须注册在 `GET /history/{task_id}` **之前**，否则 FastAPI 会把字符串 `compare` 当 `task_id` 解析（报 `40012` 而非进入对比逻辑）。属高频踩坑点。

🔒 **游客态**（JT-10 已决议）：UX-P05 允许未登录直通；API-06 / API-07 使用 `OptionalUserId` 依赖，归属写 `user_id = 0`（哨兵）。**导出（API-16）与历史（API-17~19）仍要求登录。**

📌 **`README.html` 不迁移**（UX-GOV-01）。实现时必须删除的原型痕迹清单见技术文档 §1.7，其中 **`.state-bar` 演示状态药丸、`.spec-note` 原型说明块、`.diff-note` 差异标注块、`REAL_EVAL` 常量** 为必查项。

4.3 管理端（CH-01 新增，2 页，仅 role='admin' 可见）

页面	路由	模板	主要接口	复用组件
AD-P01 运营看板	/admin	**A**	API-22 概览、API-28 模型版本、API-29 切换	MetricCard / LineCompare / CoverageBar（均已有）
AD-P02 用户与任务管理	/admin/manage	**A**	API-23 24 25 26 27 30	AntD Table / Pagination / Popconfirm / Tabs

管理端两页的内容划分：

页面	Tab / 区块	内容	数据来源
AD-P01	概览	用户数 / 数据集数 / 任务数 / 成功率 / 平均端到端耗时	users / datasets / tasks
	任务质量	近 N 次任务的加权 MAPE 走势 + 达标率	metrics
	模型版本	当前启用版本卡片（版本 / 超参 / 参数量 / 验证指标 / 启用时间）+ 历史版本列表	model_registry
	埋点概览	关键事件计数（上传 / 预测 / 导出 / 登录）	events
AD-P02	用户管理	列表（邮箱 / 昵称 / 角色 / 状态 / 注册时间 / 最近登录 / 任务数）、搜索、状态筛选、**停用•启用**、**重置密码**	users
	任务管理	全量任务列表（含所有用户）、筛选、**强制删除**	tasks + predictions + metrics
	操作日志	按 action / result / 时间 / 用户筛选，分页	operation_logs

4.4 接口总清单（既有 21 + 新增 9 = 30 个）

既有 21 个（详见技术文档 §5.4 ~ §5.8，此处只列索引）：

分组	接口
认证（5）	API-01 注册 / API-02 登录 / API-03 登出 / API-04 当前用户 / API-05 修改资料密码
数据集（6）	API-06 上传 / API-07 示例数据 / API-08 模板下载 / API-09 数据集详情 / API-10 预览 / API-11 错误明细
预测（5）	API-12 创建任务 / API-13 任务状态 / API-14 预测结果 / API-15 结果明细分页 / API-16 导出
历史（4）	API-17 历史列表 / API-18 跨任务对比 / API-19 历史详情 / API-20 删除（软删除）
系统（1）	API-21 健康检查

新增 9 个（CH-01，完整定义见 §5.5）：

编号	方法	路径	名称
API-22	GET	/admin/overview	运营看板概览
API-23	GET	/admin/users	用户列表（分页 / 搜索 / 状态筛选）
API-24	PATCH	/admin/users/{user_id}	启停用 + 角色变更
API-25	POST	/admin/users/{user_id}/reset-password	重置用户密码
API-26	GET	/admin/tasks	全量任务列表（跨用户）
API-27	DELETE	/admin/tasks/{task_id}	强制删除任务
API-28	GET	/admin/models	模型版本列表
API-29	PATCH	/admin/models/{version}	切换启用版本
API-30	GET	/admin/logs	操作日志查询

⚠ **口径说明**：立项提问时按「约 8 个接口」估算，实测细化后为 **9 个**。增加 1 个的理由：API-28（列表）与 API-29（切换）**必须分离**——一个 GET 一个 PATCH，若合并为单个 PATCH 会出现「同一路径承担读与写两种语义」，与既有 21 个接口的 RESTful 风格不一致。此修正登记为 QD-07。

4.5 需求—页面—接口 三向追溯（抽样，完整表见 PRD §3 与 UI/UX §6.3）

需求	承载页面	接口	落库位置
FR-UPL-03 单文件 ≤ 20MB、≤ 500,000 行	P05	API-06	datasets.file_size / row_count
FR-UPL-05 仓库列必填，缺失即拒	P05	API-06	datasets.warehouse_count
FR-UPL-13 断档补齐披露	P05 / P08	API-06 14	datasets.gap_filled / gap_count
FR-MOD-08 三级降级	P06 / P08	API-12 14	predictions.granularity_level
FR-MOD-10 设备回退披露	P07	API-13	tasks.device / device_fallback
FR-MOD-11 指标计算	P08	API-14	metrics.mape / mae / rmse / duration_ms
FR-MOD-12 阈值判定	P08	API-14	metrics.is_over_threshold / *_improve_pct
FR-VIS-03 明细排序分页	P08	API-15	经索引 IX-06 支持
FR-VIS-08 误差分布直方图	P08	API-14	metrics.distribution_json
FR-HIS-04 跨任务对比	P11	API-18	tasks + metrics
FR-HIS-05 删除历史（软删除）	P09	API-20	tasks.is_deleted
FR-SYS-03 操作日志	全局 + AD-P02	内部	operation_logs
NFR-SEC-02 数据隔离	全局	全部	datasets.user_id / tasks.user_id + §5.4 例外
NFR-PERF-01 端到端 ≤ 60 s	P07	API-13	tasks.duration_ms
CH-01 管理端	AD-P01 / P02	API-22~30	users.role 等（§5.3）

🔒 **编号一致性守卫**：文档引用的 FR-* / NFR-* 必须真实存在于 PRD。常驻脚本 scripts/verify_doc_refs.py 负责守这一条——**这类「编号形状对、语义错」的漂移肉眼看不出来，必须靠脚本。**（本项目在数据库设计文档阶段曾查出约 30 处系统性编号错位。）

5. 变更项 CH-01：管理员端（B 标准运营型）

本章是本次立项**唯一的范围增量**。它不是「顺手加个页面」，而是一次正式的**需求变更**（技术文档 §8.6），因此按变更单的颗粒度完整定义：起因、影响面、数据库改动、安全例外、接口、页面、回写清单。

5.1 变更必要性与依据

项	内容
变更来源	负责人立项决策：**「需要管理员端」，档位 B 标准运营型**
是否需要变更流程	☑ **需要。**技术文档 §8.6 规定「调整需求范围」须以 PRD 优先级表为基准并经产品负责人确认
是否违反现有文档	✕ **不违反，而是关闭一个开放问题。**PRD 附录 A `Q13` 原状态为 ⚠「是否需要管理员角色？——MVP 不设，仅用户维度」，**该问题待确认。**本次决策即为其结论
被推翻的现有声明	⚠ 数据库设计文档 §3.3 决策行「**无 `role` 字段**——PRD §3.1 明确本期仅一种角色」需修正；PRD §1.4 范围红线「✕ 角色权限体系」需修正为「本期引入**单一管理员角色**，不含多角色 RBAC」
是否新增数据表	✕ **不新增。**表数仍为 **8**，仅 `users` 加 1 列、`operation_logs` 扩 1 个枚举值
是否新增架构层	✕ **不新增。**管理端是应用层内一组受保护路由

为什么值得做：`AD-P01`/`AD-P02` 所需的**全部数据源在现有 8 张表中已存在**——`users`（含 `status` 启停用字段）、`datasets`、`tasks`、`predictions`、`metrics`、`model_registry`（含 `is_active` + 部分唯一索引）、`operation_logs`、`events`。管理端本质是**对这些既有数据的跨用户聚合视图**，不需要新的业务模型。

5.2 影响面清单

影响面	具体项	是否需改
数据库	`code/db/schema.sql` + `code/db/seed.sql` + `data/app.db` 迁移	☑ 是（§5.3）
后端	新增 `api/v1/admin.py`、`services/admin_service.py`；`core/deps.py` 增 `get_current_admin`；`core/errors.py` 增 403 业务码	☑ 是
前端	新增 2 页 + 1 处导航入口 + `useAuthStore` 增 `isAdmin` 派生	☑ 是
接口契约	`code/backend/schemas.py`（+ admin 模型）、`code/frontend/types.ts`、`api-client.ts`	☑ 是
安全	`NFR-SEC-02` 数据隔离实现出现**唯一显式例外**	☑ 是（§5.4）
设计	复用模板 A 与既有组件，**不新增设计令牌**	☒ 否（`QD-01`）
算法	完全不涉及	☒ 否
部署	不涉及（管理端与后台同进程同端口）	☒ 否

5.3 数据库变更（⏱ 已全部实测验证，不是纸面结论）

5.3.1 变更一：users 增 role 列

DDL（同步写入 `code/db/schema.sql`，位置紧接 `status` 之后）：

```
    role            TEXT    NOT NULL DEFAULT 'user'         -- 角色：user 普通用户 / admin 管理员（CH-01）
```

```
                    CHECK (role IN ('user', 'admin')),
```

存量库迁移语句：

```
ALTER TABLE users ADD COLUMN role TEXT NOT NULL DEFAULT 'user'
    CHECK (role IN ('user', 'admin'));
```

⏱ **实测结果**（`sqlite3` 直接执行，两种场景均验证）：

场景	命令	结果
空库建表后 ALTER	`ALTER TABLE users ADD COLUMN role ... CHECK (...)`	✅ 成功，`role` 默认 `'user'`
已有数据的真实库（本项目 `schema.sql` + `seed.sql`，含 `id=0` 哨兵）	同上	✅ 成功，哨兵自动获得 `role='user'`
写入合法值 `'admin'`	`INSERT ... role='admin'`	✅ 接受
写入非法值 `'bogus'`	`INSERT ... role='bogus'`	✅ **被拒绝**：`CHECK constraint failed: role IN ('user','admin')`

📌 **两条关键结论**：① SQLite 的 `ADD COLUMN` **允许**带 `CHECK` 约束（已验证），因此不需要重建表；② 游客哨兵（`id=0`）自动落到 `role='user'`，**不会**因为默认值而意外获得管理员权限。

是否加索引：✕ **不加。**理由：`role` 只有 2 个取值，基数极低，索引选择性差；且管理端「用户列表」是全量分页（不是按 role 高频筛选），加索引的收益低于维护成本。

5.3.2 变更二：operation_logs.action CHECK 由 7 类扩为 8 类

问题：现有 `action` 的 CHECK 为 7 类 `('login','register','logout','upload','predict','export','delete')`。管理端四类新动作——**停用/启用用户、重置密码、切换模型版本、强制删除任务**——中只有「强制删除」能勉强映射到

`delete`，其余无对应值。若不处理，管理端操作**无法写入审计日志**，`AD-P02` 的操作日志页会出现审计盲区。

方案（`QD-02`，建议采纳 A）：

方案	做法	值数	评价
A（建议）	增 1 个值 `'admin'`，管理端子动作写入 `detail`（结构化文本，如 `sub=user_disable;target_user=3;by=1`）	**8**	CHECK 保持紧凑；管理端操作量小，筛选用 `detail` 前缀即可
B	拆成 4 个独立值 `admin_user_update` / `admin_user_reset_pwd` / `admin_task_delete` / `admin_model_activate`	11	可筛选性最好，但值集开始膨胀，接近枚举反模式

建议方案 A 的 DDL：

```
    action      TEXT    NOT NULL                            -- 8 类操作（FR-SYS-03 四类核心动作的超集 + 管理端子层 CH-01）
                CHECK (action IN ('login', 'register', 'logout', 'upload', 'predict', 'export', 'delete', 'admin')),
```

⏱ **实测约束**：SQLite **不支持修改已有列的 CHECK**，必须走表重建：

场景	处理方式
A：库中尚无生产数据（本期适用）	直接修改 `schema.sql` 建表语句 → 重建库。**零风险**，无需 12 步流程
B：库中已有数据	按 SQLite 官方 12 步「表重建」流程（建新表 → `INSERT INTO ... SELECT` 拷数据 → `DROP` 旧表 → `RENAME` → 重建索引）。`operation_logs` 无外键指向它，重建成本可控

📌 本期属场景 A（`data/app.db` 尚未产生，`operation_logs` 无数据），因此**只需改 `schema.sql`，不需要写复杂的迁移脚本**。迁移脚本仍按 §5.3.4 统一登记，以备后续环境。

5.3.3 变更三：seed.sql 内置管理员种子账号

```
-- CH-01：管理员种子账号（幂等，可重复执行）
-- ⚠ 邮箱域名必须能被 Pydantic EmailStr 接受，否则 LoginRequest 会先拒掉登录请求（见下方实测）
INSERT OR IGNORE INTO users (email, password_hash, nickname, status, role)
VALUES ('admin@salesforecast.cn', '<bcrypt 串，由 scripts/ 一次性生成>', '系统管理员', 1, 'admin');
```

🔒 **实测确认的陷阱（务必遵守）**：`LoginRequest.email` 的类型是 **`EmailStr`**（`code/backend/schemas.py:176`），而 `EmailStr` 会**主动拒绝 RFC 2606 保留域**。若种子管理员邮箱沿用游客哨兵的 `.invalid` 风格，管理员将**永远无法登录**——请求在 Pydantic 校验层就被拒掉，连密码比对都走不到。⚠ 行号锚点会随 `schemas.py` 增删而漂移（`EP-4` 加过 16 行、`EP-5` 又加了 `HorizonEstimate` / `estimate_by_horizon` / `timing` 三处），**引用前先 grep 一次真实行号**。

候选邮箱	`EmailStr` 判定	可否作管理员
`admin@local.invalid`	✗ `The part after the @-sign is a special-use or reserved name`	✗
`admin@salesforecast.local`	✗ 同上（`.local` 同属保留域）	✗
`admin@salesforecast.cn`	☑ 接受	☑ **推荐**
`admin@example.com` / `admin@test.com` / `admin@forecast.internal`	☑ 接受	☑ 可用

📌 **与游客哨兵的对照**：游客哨兵用 `.invalid` 域是**刻意的安全设计**——它使哨兵账号天然无法登录（无需额外代码防护）。管理员种子账号的目标相反（**必须能登录**），因此**两者的域名策略必然不同**，不可互相复制。

⚠ **密码强度须与注册规则一致**（`FR-USR-02`）：≥ 8 位、同时含字母与数字、**≤ 72 字节**（`JT-12`）。生成 bcrypt 串时须使用与运行时一致的 `bcrypt==5.0.0` 与 cost 因子，否则校验失败。

5.3.4 迁移管理

沿用 TD-DB-03 的既有约定：**不引入 Alembic 等迁移框架**，靠 PRAGMA user_version 手工管理。

版本号	变更内容	脚本
0 → 1	CH-01：users.role + operation_logs.action 8 类（本期实际以改 DDL 重建实现）	scripts/migrate_0001_ch01_role.py

迁移脚本必须满足：① 幂等（重复执行不报错）；② 迁移前自动备份 data/app.db；③ 执行后调 PRAGMA user_version = 1；④ 输出前后列数对比（120 → 121）。

5.4 数据隔离红线的例外设计（🔒 安全核心）

5.4.1 红线原文与例外的边界

PRD §3.2 规定：**「任何接口的查询条件必须携带当前用户 ID 作为过滤条件，这是数据隔离红线（NFR-SEC-02）。」**

管理端需要跨用户读取数据——这是该红线的**第一个、也是唯一的显式例外**。例外必须被**精确圈定**，不能扩散：

维度	普通用户接口（21 个）	管理端接口（新增 9 个）
数据范围	仅 user_id = 当前用户	全量
鉴权依赖	get_current_user	**get_current_admin**（在 get_current_user 基础上强制 role == 'admin'）
未授权响应	401（未登录）	403（已登录但非管理员）
审计	部分操作记 operation_logs	**每个写操作必须记**

维度	普通用户接口（21 个）	管理端接口（新增 9 个）
		（action='admin'）
出现位置	app/api/v1/*.py（5 个文件）	**仅 app/api/v1/admin.py 一个文件**

🔒 **BE-ADM-01：例外只允许存在于 admin.py。** 任何在 auth.py / datasets.py / predictions.py / history.py / system.py 中出现「不带 user_id 过滤的查询」都视为**红线破坏**，代码评审直接打回。建议用 ruff 自定义规则或评审检查单（技术文档 REV-1）强制。

5.4.2 三条必须同时满足的安全约束

#	约束	落地要求
1	**强制角色校验**	get_current_admin 必须在**每一个**管理端接口上声明；禁止在路由函数内部「先查后判」（漏判即越权）
2	**全操作审计**	管理端 4 类写操作（启停用、重置密码、强制删除、切换模型）全部写 operation_logs，action='admin'，detail 记子动作 + 操作者 + 目标
3	**最小暴露**	管理端**不返回**用户密码哈希、不提供原始 CSV 内容下载（QD-04）；手机号/邮箱等个人数据按 NFR-SEC-05 脱敏规则处理

5.4.3 越权判定的顺序（沿用 BE-41，不得简化）

```
请求进入
  ├─ 携带 Token?
  │    ├─ 否 → 401（未登录）
  │    └─ 是 → 校验签名 + 到期时间 + token_version
  │         ├─ 失败 → 401（Token 失效）
  │         └─ 成功 → 取 user_id
  │              ├─ role != 'admin' → 403（E_FORBIDDEN）
  │              └─ role == 'admin' → 通过，进入业务
```

└ 管理端特有：写操作 → 先落审计日志，再执行

📌 **注意 `token_version` 的联动**（数据库设计文档 §10.5）：管理员**重置用户密码**时，必须同时把该用户 `token_version + 1`，使该用户所有旧 Token 立即失效——否则「重置了密码但攻击者手上的 Token 还能用」，重置形同虚设。

5.5 管理端接口清单（API-22 ~ API-30）

统一约定：路径前缀 `/api/v1/admin`；全部需要 `get_current_admin`；响应结构沿用技术文档 §5.1 的统一封装（`{code, message, data}`）。

编号	方法 / 路径	名称	请求要点	响应要点	优先级		
API-22	GET /admin/overview	运营看板概览	可选 ?days=30	用户数 / 数据集数 / 任务数 / 成功率 / 平均端到端耗时 / 加权 MAPE 走势 / 事件计数	P1		
API-23	GET /admin/users	用户列表	?page&size&q&status&role	分页用户列表（**不含密码哈希**）	P1		
API-24	PATCH /admin/users/{user_id}	启停用 + 角色变更	`{status?: 0\	1, role?: 'user'\	'admin'}`	更新后的用户摘要	P1

编号	方法 / 路径	名称	请求要点	响应要点	优先级		
API-25	POST /admin/users/{user_id}/reset-password	重置用户密码	{new_password}（须过强度校验）	成功标记；**同时 token_version + 1**	P1		
API-26	GET /admin/tasks	全量任务列表	?page&size&user_id&status&q	分页任务列表（跨用户）	P1		
API-27	DELETE /admin/tasks/{task_id}	强制删除任务	——	成功标记；连带清理 predictions / metrics	P1		
API-28	GET /admin/models	模型版本列表	——	版本 / 超参 / 参数量 / 验证指标 / is_active / 训练时间	P1		
API-29	PATCH /admin/models/{version}	切换启用版本	{is_active: true}	成功标记；**事务内先降后升**	P1		
API-30	GET /admin/logs	操作日志查询	?page&size&action&result&user_id&start&end	分页日志列表	P1		

5.5.1 API-29 的事务顺序（🔒 必须严格照此实现）

`model_registry` 上有**部分唯一索引** `ux_model_active ON model_registry (is_active) WHERE is_active = 1`，保证「全局有且仅有一个启用版本」。

⏱ **实测验证**：

操作	结果
`BEGIN; UPDATE ... SET is_active=0 WHERE is_active=1; UPDATE ... SET is_active=1 WHERE version='lstm_v2'; COMMIT;`	✅ 正确切换（`lstm_v1=0`, `lstm_v2=1`）
跳过第一步，直接把 `lstm_v1` 也置 1	✅ **被拒绝**：`UNIQUE constraint failed: model_registry.is_active`

🔒 **BE-ADM-02：两个 `UPDATE` 必须在同一事务内，且顺序固定（先降旧、再升新）。** 顺序颠倒或分两次提交都会撞唯一索引失败。

🔒 **BE-ADM-03：切换版本后必须校验与 `.env` 的 `ACTIVE_MODEL` 一致**（技术文档 `BE-06`）——不一致会导致「数据库说启用了 v2、进程实际加载 v1」的静默故障。实现方式：切换接口在事务提交后触发模型重载，或返回提示要求运维同步 `.env` 并重启。

5.5.2 API-27 强制删除的级联范围

表	处理	依据
`tasks`	删除	目标对象
`predictions`	级联删除	外键 `ON DELETE CASCADE`（`FK-*`）
`metrics`	级联删除	同上
`datasets`	**不删除**（数据集可能被其他任务引用）	外键 `ON DELETE RESTRICT`（实测：被引用时删除报错）
`operation_logs`	**保留**（审计留痕，不随任务删除）	`FR-SYS-03`

表	处理	依据
导出文件	保留至 TTL 到期（EXPORT_TTL_DAYS=7）	NFR-REL-05

实现方式：普通用户的「删除历史」是**软删除**（tasks.is_deleted=1，FR-HIS-05）；管理员的是**硬删除**（物理删除 + 级联）。两者语义不同，需在界面上明确文案区分（QD-03）。

5.6 管理端页面规格（复用模板 A，不新增视觉语言）

AD-P01 运营看板（/admin）

区块	内容	复用组件
顶部指标带	用户数 / 数据集数 / 任务数 / 成功率 / 平均端到端耗时	MetricCard
任务质量	近 N 次任务加权 MAPE 折线 + 达标率	LineCompare + MetricCard
模型版本	当前启用版本卡片 + 历史版本表 + 切换入口	AntD Table + Popconfirm
埋点概览	事件计数（上传 / 预测 / 导出 / 登录）	MetricCard + 简单柱状

AD-P02 用户与任务管理（/admin/manage）

Tab	内容	关键交互
用户管理	列表 + 搜索 + 状态/角色筛选	停用·启用（Popconfirm）、重置密码（模态框 + 强度校验）
任务管理	全量任务列表 + 筛选	强制删除（Popconfirm，文案明确「不可恢复」）
操作日志	按 action / result / 时间 / 用户筛选	分页；action='admin' 的 detail 展开显示子动作

🔒 `AD-P02` **的破坏性操作必须二次确认**，且确认文案不得与用户的软删除混淆（用户侧文案：「删除后可在回收路径恢复」；管理侧文案：「**此操作不可恢复**，将同时删除该任务的全部预测明细与指标」）。

5.7 上游文档回写清单（变更流程强制要求）

`CH-01` 落地后，**必须同步回写以下文档**，否则文档与代码将出现不一致（技术文档 §8.6）：

#	文档	章节	回写内容
1	**PRD V1.0**	附录 A `Q13`	⚠「是否需要管理员角色」→ ✅ **已确认：本期引入，B 标准运营型**
2	**PRD V1.0**	§1.4 范围红线	「❌ 角色权限体系」→ 修正为「本期引入**单一管理员角色**（`user`/`admin` 两值），不含多角色 RBAC」
3	**PRD V1.0**	§3.2 角色模型	「仅一种角色（注册用户）」→ 补管理员维度与隔离例外说明
4	**PRD V1.0**	§3 功能需求	新增 `FR-ADM-*` 编号（【待补充：编号段待 PRD 侧定义，建议 `FR-ADM-01`~`FR-ADM-12`】）
5	**PRD V1.0**	§4 `NFR-SEC-02`	数据隔离红线增补「管理端为唯一显式例外，例外范围限于 `admin.py`」
6	**数据库设计文档 V1.0**	§3.3 `users` 决策表	删除「无 `role` 字段」行，改为「`role` 列，2 值，`CH-01` 引入」
7	**数据库设计文档 V1.0**	§4 数据字典	**120 列 → 121 列**（`users.role` 补 11 属性条目）
8	**数据库设计文档 V1.0**	§5.10 `operation_logs`	`action` 7 类 → **8 类**

#	文档	章节	回写内容
9	**数据库设计文档 V1.0**	附录 C	新增 `CH-01` 登记；`TQ-11` 状态更新（7 类 → 8 类）
10	**数据库设计文档 V1.0**	附录 A 字段变更对照表	新增 `role` 列的前后对比
11	**开发技术文档 V1.0**	§3.3 表结构	同步 `role` 列与 `action` 值集
12	**开发技术文档 V1.0**	§5.3 接口清单	21 个 → **30 个**
13	**开发技术文档 V1.0**	§4.3 依赖注入	新增 `get_current_admin` 说明
14	**UI/UX 设计文档 V1.0**	§6.1 页面清单	11 页 → **13 页**（增 `AD-P01` / `AD-P02`，模板 A）
15	**UI/UX 设计文档 V1.0**	§6.3 需求映射矩阵	补管理端行
16	`code/db/schema.sql` / `seed.sql`	——	`role` 列 + 种子账号 + `action` 8 类
17	`code/backend/schemas.py` / `types.ts` / `api-client.ts`	——	管理端请求/响应模型与 API 函数
18	`scripts/verify_schema.py` / `verify_schemas.py` / `verify_doc_refs.py`	——	新增 `role` / `action` / `FR-ADM` 回归断言

🔒 **回写完成是 `EP-8` 的验收项之一**（`DOD-EP8`）。只改代码不回写文档 = 技术债（技术文档 §0.8 纪律）。

---

6. 实施阶段划分

6.1 阶段划分原则

#	原则	说明
1	**每阶段以「可运行、可演示」收口**	不允许出现「半成品阶段」（开发方案 §7.1）

#	原则	说明
2	**算法链路优先于界面美化**	先用脚本跑通算法，再接 Web。否则联调期会同时面对算法与接口两类问题
3	**不可逆依赖前置**	数据库、认证先于业务；业务先于聚合视图（管理端放在业务之后）
4	**风险最高的阶段给足余量**	`EP-5` / `EP-6`（预测链路 + 结果页）是唯一有硬指标门槛的阶段
5	**每阶段结束做一次演示**	未通过不进下一阶段（`DOD` 逐条核对）
6	**单人开发 → 严格串行**	阶段之间不并行；同阶段内可并行前后端，但以「后端接口自测通过 → 再写前端」为准（避免同时面对两端问题）

6.2 与《开发方案 V1.1》§7.2 阶段排期的映射

开发方案已给出 `P0`~`P6` 阶段框架。本方案在此基础上前推细化，**映射关系如下**（不是替换，而是展开）：

开发方案阶段	本方案阶段	展开说明
`P0` 需求与技术验证	`EP-0`（部分）	需求验证已完成（5 份文档齐备、算法已实现）；`EP-0` 只做**工程基建**
`P1` 算法链路打通	☑ **已完成**	算法层 9 模块 + 69/69 自检 + 指标达标（14.26%）；本方案不需要重复该阶段
`P2` 后端服务化	`EP-1` ~ `EP-3`（数据库 / 认证 / 上传）	拆为 3 个可独立验收的阶段
`P3` 前端界面	`EP-3` `EP-5` `EP-6` `EP-7`	前端随其后端模块**成对交付**，不做「先写完所有后端再写所有前端」
`P4` 集成联调	`EP-9`	与 `P5` 测试合并为一次收敛
`P5` 测试与优化	`EP-9`	同上
`P6` 文档与答辩	`EP-10`	——
（无对应）	**`EP-8` 管理端**	☑ 本次新增（`CH-01`）

📌 **重要差异说明**：开发方案的 P1（算法链路）**已经完成**，本方案从「工程基建」直接起步。这是本方案与上游排期最大的不同——**上游的 7 个阶段中，有 1 个已完成、1 个被合并、1 个是新增。**

6.3 阶段总表（EP-0 ~ EP-10，共 11 个阶段）

阶段	名称	主要工作	交付物	前置	门槛
EP-0	工程基建与设计系统移植	web/ 工程创建；56 令牌移植；4 布局模板；后端骨架 + /health；依赖安装与版本锁定	可启动的前后端空壳	——	DOD-EP0
EP-1	数据库落地与数据访问层	data/ 初始化；schema.sql+seed.sql 执行；**CH-01 数据库变更**；8 个 ORM 实体；DAO；WAL	data/app.db + ORM + DAO	EP-0	DOD-EP1
EP-2	认证与用户	security.py / deps.py（含 get_current_admin）；API-01~05；UX-P02 / UX-P03；登录限流	注册登录闭环	EP-1	DOD-EP2
EP-3	前台产品首页与全局框架	UX-P01（模板 D）；页脚免责声明（🔒）；错误边界 / 空态 / 骨架屏；主题切换	前台可访问	EP-2	DOD-EP3
EP-4	数据集上传与校验	API-06~11；7 步校验；5 类结构化错误；流式落盘；断档处置；游客态；UX-P05	上传链路可用	EP-3	DOD-EP4
EP-5	预测任务编排 ★	★ 先跑通离线脚本；predict_service 调 algo/；任务状态机；批量推理；GPU→CPU 回退；超时软失败；API-12 13；UX-P06 UX-P07	端到端可跑	EP-4	DOD-EP5
EP-6	结果可视化与导	API-14 15 16；指标口径	核心交付页	EP-5	DOD-EP6

阶段	名称	主要工作	交付物	前置	门槛
	出 ★	落地（加权 MAPE 为主 / 主指标只算 L1）；UX-P08 全要素			
EP-7	历史记录与对比 + 工作台	API-17~20；软删除；UX-P09 P10 P11 + UX-P04	完整业务闭环	EP-6	DOD-EP7
EP-8	**管理端（CH-01）**	get_current_admin；API-22~30；AD-P01 AD-P02；全操作审计；**上游文档回写**	管理端可用 + 文档同步	EP-7	DOD-EP8
EP-9	集成联调与测试	主干路径端到端；B1~B14 分支；边界与异常；性能测试；四浏览器与分辨率	测试报告 + 可交付版本	EP-8	DOD-EP9
EP-10	文档与答辩交付	测试报告 / 部署运维手册 / API 文档；PPT；演示脚本；演示数据冻结	全套交付物	EP-9	DOD-EP10

依赖链（串行，不可跳阶）：

```
EP-0 → EP-1 → EP-2 → EP-3 → EP-4 → EP-5 ★ → EP-6 ★ → EP-7 → EP-8 → EP-9 → EP-10
                                  └─ 全项目关键路径 ─┘
```

📌 **EP-3 的弹性**：前台产品首页（UX-P01）技术风险最低（静态内容 + 少量接口），若 EP-2 完成后需要尽早拿出可演示成果，EP-3 可与 EP-4 对调顺序。此弹性**不影响**关键路径（EP-5/EP-6）。

6.4 执行进度登记（滚动更新）

阶段	状态	完成日期	该阶段新增/变更的工程文件	回归结论	

阶段	状态	完成日期	该阶段新增/变更的工程文件	回归结论	
EP-0 工程基建	✅ 已完成	2026-10-09	后端 11 文件（core/ 6 + api/ 2 + main.py + models//dao/ 占位）；前端整目录 web/（34 文件）	verify_tokens ✅ / tsc+eslint+vite build ✅ / 依赖 11 项与 §2.3 一致	
EP-1 数据库落地	✅ 已完成	2026-10-09	app/models/ 8 实体；app/dao/ 8 模块；scripts/{verify_orm,verify_schema,explain_queries,migrate_0001_ch01_role}.py	verify_schema ✅ / verify_doc_sql ✅（121 列）/ verify_orm ✅ 红线零命中	
EP-2 认证与用户	✅ 已完成	2026-10-09	**后端 5 新增**：core/security.py、core/deps.py、services/__init__.py、services/auth_service.py、api/v1/auth.py、schemas/__init__.py；**改动 7**：main.py、api/v1/router.py、core/errors.py、dao/user_dao.py、code/backend/schemas.py、code/frontend/types.ts、web/src/types/index.ts；**前端 5 新增/重写**：components/RequireAuth.tsx、pages/LoginPage.tsx、pages/RegisterPage.tsx、layouts/AuthLayout.tsx、store/useAuthStore.ts、App.tsx、layouts/AppShell.tsx、router/index.tsx；**脚本 1 新增**：scripts/verify_auth.py	**verify_auth 39/39** ✅ / verify_schemas 57/57 ✅ / 前端三件套 ✅ / 无头浏览器 4 态 consoleErrors=0 ✅	
EP-3 前台首页	✅ 已完成	2026-10-09	**前端新增 10**：components/{SiteFooter,ErrorBoundary,EmptyState,Skeleton,NoticeBar,ThemeToggle,EChart,HeroChart}.tsx、pages/HomePage.tsx（重写）、generated/{metrics.ts,eval_report.json}；**改动 7**：layouts/{MarketingLayout,AuthLayout,AppShell}.tsx、App.tsx、main.tsx、styles/index.css、theme/echarts-theme.ts（拆出 echarts 依赖）、store/useThemeStore.ts（去 echarts）、types/index.ts；**脚本 2 新增**：scripts/{sync_eval_metrics.py,verify_ep3.py} + scripts/verify_ep3_browser.mjs	**verify_ep3 29/29** ✅ / **verify_ep3_browser 47/47** ✅ / 4 模板走查 + 硬指标逐项比对 ✅ / 主包 1381→343 kB ✅	
EP-4 数据	✅ 已	2026-	**后端新增 3**：services/dataset_service.py（约 900 行）、services/maintenance_service.py（§3.8 清理）、api/v1/datasets.py（7 条路由）；**改动 5**：	ts、layouts/aside-slot.tsx；**前端改动 4**：	verify_ep4 **77/77** ✅ /

阶段	状态	完成日期	该阶段新增/变更的工程文件	回归结论	
集上传与校验	完成	10-09	app/core/config.py（+4 字段）、app/api/v1/router.py、code/backend/schemas.py（+16 行）、app/schemas/__init__.py（锚点 172→177）、web/src/router/index.tsx；**前端新增 9**：`components/upload/{DropZone,FileChip,ValidationPanel,ErrorCard,PreviewTable,FieldSpecCard,UploadAside,format}.tsx\	pages/UploadPage.tsx（全量重写）、layouts/AsideLayout.tsx（右栏插槽，保住 2 个 aside）、api/client.ts、types/index.ts；**脚本 3 新增**：scripts/{verify_ep4.py,verify_ep4_api.py,cleanup_tmp.py} + scripts/verify_ep4_browser.mjs`	**verify_ep4_api 119/119** ✅ / **verify_ep4_browser 173/173** ✅（18 个真实状态）/ 前端三件套 ✅ / 4 个真实缺陷已修 ✅
EP-5 预测任务编排 ★	✅ 已完成	2026-10-10	**后端新增 5**：core/model_holder.py、services/predict_service.py、services/cost_model.py、api/v1/predictions.py、algo 热路径函数（features.calendar_matrix / last_window_fast）；**后端改动 6**：algo/infer.py（Timing +2 埋点 / _roll_multistep 重写）、algo/config.py（HORIZONS）、services/dataset_service.py（estimate_by_horizon）、services/predict_service.py（4 处口径修正）、main.py（预加载）、code/backend/schemas.py；**前端新增 9**：components/{Card,StepBadges}.tsx、components/params/ParamsAside.tsx、components/run/{StageTimeline,StageList,LogStream,RunAside}.tsx、pages/{RunningPage,ResultPage}.tsx、generated/pipeline.ts；**前端改动 6**：pages/ParamsPage.tsx（占位→完整表单）、api/client.ts（🟤 泛型缺陷）、types/index.ts、router/index.tsx、vite.config.ts、components/run/StageTimeline.tsx；**脚本 7 新增**：scripts/{verify_ep5.py,verify_ep5_api.py,sync_pipeline_meta.py,verify_ep5_equiv.py,_ep5_parity_run.py,_ep5_cfg_probe.py} + scripts/verify_ep5_browser.mjs	**verify_ep5 50/50** ✅ / **verify_ep5_api 60/60** ✅ / **verify_ep5_browser 41/41** ✅（Edge 真实渲染，consoleErrors=0）/ 前端三件套 ✅ / 🟤 **horizon=30 击穿 60 s 的真实缺陷已修（72.2 s → 23.8 s，逐位等价）** / 1 个真实前端缺陷已修 / 5 项口径偏离已登记	
EP-6 结	✅	2026	**后端新增 2**：services/result_service.py（约 560 行）、api/v1/predictions.py（3 条路由）；**后端改动 3**：	**verify_ep6 65/65** ✅ / **verify_ep6_api 105/105** ✅ /	

阶段	状态	完成日期	该阶段新增/变更的工程文件	回归结论	
果可视化与导出★	已完成	-10-10	dao/prediction_dao.py（排序白名单 / 过滤分页 / 导出迭代）、code/backend/schemas.py（+PredictionDetailQuery / DetailPage / 🟢 10 字段改可空）；**前端新增 7**：components/result/{MetricPanels,CoveragePanel,LineCompare,DistributionChart,DetailTable,ExportButtons,ResultAside}.tsx；**前端改动 2**：pages/ResultPage.tsx（占位 → 七区块完整页）、types/index.ts（+truncated / +THRESHOLDS 同源阈值）；**脚本 3 新增**：scripts/{verify_ep6.py,verify_ep6_api.py} + scripts/verify_ep6_browser.mjs	verify_ep6_browser **42/42** ✅（Edge 真实渲染，consoleErrors=0）/ 前端三件套 ✅ / 🟢 **2 个「示例数据永不暴露」的 500 缺陷已修（L3 空基线 / L1=0 空指标）** / 1 个真实前端三态配色缺陷已修 / 3 项口径偏离已登记	
EP-7 历史记录与对比 + 工作台	✅ 已完成	2026-10-10	**后端新增 2**：services/history_service.py（约 620 行）、api/v1/history.py（5 条路由，含 GET /history/dashboard）；**后端改动 4**：dao/task_dao.py（+list_history/count_history/_history_filters，🟢 soft_delete 潜伏缺陷已修）、services/result_service.py（🟢 软删除视为不存在 + predicted_only 参数）、code/backend/schemas.py（+HistoryPage/CompareSeries/WorkbenchSummary，🟢 TaskBrief 补失败三字段）、code/db/schema.sql（🟢 视图外露 dataset_id）；**前端新增 4**：pages/{HistoryPage,HistoryDetailPage,ComparePage,DashboardPage}.tsx（后者从 14 行占位重写）；**前端改动 3**：router/index.tsx（+3 条路由全套 RequireAuth）、layouts/AppShell.tsx（+2 导航项）、api/client.ts / types/index.ts；**脚本 3 新增**：scripts/{verify_ep7.py,verify_ep7_api.py} + scripts/verify_ep7_browser.mjs	verify_ep7 **89/89** ✅ / verify_ep7_api **70/70** ✅ / verify_ep7_browser **57/57** ✅（Edge 真实渲染，consoleErrors=0）/ 前端三件套 ✅ / 🟢 **5 个真实缺陷已修（soft_delete 潜伏 / 视图缺 dataset_id / 软删除后详情仍可访问 / 失败任务详情 500 / TaskBrief 丢弃失败字段）** / 6 处判据错误已修 / 6 项口径偏离已登记	
EP-8 管理端（CH-01）	✅ 已完成	2026-10-10	**后端新增 2**：services/admin_service.py（9 个接口业务层，含 6 类子动作审计）、api/v1/admin.py（9 条路由，🔒 全部 CurrentAdmin）；**后端改动 4**：services/auth_service.py（🟢 **login() 补 user.status 检查** —— 原实现完全不看停用状态，停用用户能重新登录绕过 API-24）、dao/user_dao.py（+get_by_ids）、dao/metric_dao.py（+list_by_task_ids/count_by_task）、api/v1/router.py（挂载并注释隔离例外）；**schema 新增 23 个**管理端模型（🔒 明确不含 password_hash）；**前端新增 3**：pages/{AdminDashboardPage,AdminManagePage}.tsx、components/RequireAdmin.tsx；**前端改动 5**：	verify_ep8 **108/108** ✅ / verify_ep8_api **96/96** ✅ / verify_ep8_browser **61/61** ✅（Edge 真实渲染，consoleErrors=0）/ 前端三件套 ✅（tsc 0 错、eslint 0 告警、构建主包 534 kB）/ 🟢 **3 个真实缺陷已修（停用用户仍能登录 / NoticeBar 丢**	

阶段	状态	完成日期	该阶段新增/变更的工程文件	回归结论	
			router/index.tsx（+2 路由，RequireAuth 外包 RequireAdmin）、layouts/AppShell.tsx（+ADMIN_NAV_ITEMS，仅 isAdmin 渲染）、components/NoticeBar.tsx（🟢 **透传 data-***，原先调用方写的 data-testid 被静默丢弃）、components/result/{LineCompare,DistributionChart}.tsx（🟢 ECharts 改懒加载）、api/client.ts / types/index.ts；**脚本 3 新增**：scripts/{verify_ep8.py,verify_ep8_api.py} + scripts/verify_ep8_browser.mjs	**弃 data-testid / ECharts 主包体积回归)** / 8 处判据错误已修 / 5 项口径偏离已登记	
EP-9 集成联调与测试	✅ **已完成**	2026-10-12	**源码修复 3 处**：services/predict_service.py（🟢 API-13 越权码 40401→40301，F-05；get_status 改三步判定）、dao/task_dao.py（mark_succeeded/mark_failed 补 device/device_fallback，F-01）、services/result_service.py + services/history_service.py（越权码收敛 F-02）；**脚本新增 7**：scripts/{verify_ep9_api.py,verify_ep9_perf.py,verify_ep9_contract.py,verify_ep9.py,_ep9_boot_probe.py,_ep9_make_fixture.py} + scripts/verify_ep9_browser.mjs；**脚本改进 6**：verify_ep3~ep8_browser.mjs 补 --no-proxy-server（T-24）；**文档新增 1**：docs/电商销量预测系统_测试报告_V1.0.md	**verify_ep9_api 125/0** ✅（B-01~B-25 全 25 条）/ **verify_ep9_perf 47/47** ✅（§9.7 全 7 项）/ **verify_ep9_browser 109/109** ✅（Edge 双视口，consoleErrors=0）/ **verify_ep9_contract 92/92** ✅ / **a11y_audit 退出码 0** ✅ / **35 套脚本全量回归实跑全绿** ✅ / 🟢 **5 个真实缺陷已修（F-01~F-05）+ 24 个脚本缺陷 + 3 处文档勘误** / 🟢 DOD-EP9 第 4 条（四浏览器）**部分达成** → P1-01	
EP-10 文档与答辩交付	✅ **已完成**	2026-10-13	**后端新增 1**：app/main.py 的 /docs · /redoc 离线化（SWAGGER_VERSION=5.33.1 / REDOC_VERSION=2.1.5，docs_url=None 改自定义路由 + StaticFiles 挂载）；**静态资产新增 5**：app/static/vendor/swagger-ui/{swagger-ui-bundle.js,swagger-ui.css,favicon-32x32.png,LICENSE.txt} + app/static/vendor/redoc/redoc.standalone.js；**脚本新增 7**：scripts/{export_openapi.py,build_api_doc_html.py,make_demo_data.py,verify_demo_data.py,build_rag_corpus.py,verify_doc_exports.py} + scripts/verify_docs_browser.mjs；**文档新增 6**：docs/ 下部署运维手册 / AI 协作教程 / RAG 语料库与提示词 / 项目说明 / 演示脚本，**文档产出 1**：outputs/ 下	**verify_docs_browser 16/16** ✅（Edge 与 Chrome 各 16/16：/docs .opblock>0、DOM 零外链、零 loadingFailed、零 4xx/5xx）/ **verify_demo_data 50/50** ✅（冻结数据幂等 + 三级降级分级实测 + 2 个失败分支）/	

阶段	状态	完成日期	该阶段新增/变更的工程文件	回归结论	
			OpenAPI JSON + 单文件接口文档 HTML；**答辩新增 1**：`outputs/电商销量预测系统_答辩演示_V1.0/`（`STORY.md` + 18 页 `slides/` + 7 张内嵌截图 + 1 个原生图表 + PowerPoint 真实渲染图 18 张）；**文档导出新增 5 组**：`outputs/` 下新增 5 份 `.docx` + 5 份 `.pdf`（累计 13 + 13），TOC 域已展开、0 处未展开占位提示	`verify_doc_refs` **0 悬空编号** ✅（受管文档由 1 份扩到 **9 份**，覆盖全部交付文档）/ `verify_doc_exports` **13/13** ✅（13 组 docx·pdf 齐备 · 未过期 · TOC 域已展开）/ `verify_ep9` **清单守卫 55/55** ✅（37 个校验脚本无漏网）/ **OpenAPI 导出 30 路径 · 32 操作 · 78 Schema** ✅ / **PPT 18 页 slidep 校验 0 诊断 + PowerPoint 实渲染 18/18** ✅ / 🟢 **实测缺陷 1 个**：`get_redoc_html` 的 `favicon_url` 参数名写错导致 `/redoc` 500（正确名为 `redoc_favicon_url`），属校验脚本抓出的真实缺陷 / 🟢 **编号漂移 1 处**：`prototype/compare.html` 与 `README.html` 使用自编号 `FR-HIS-07`·`FR-HIS-08`（PRD `FR-HIS` 仅 01–06），已修正为 `FR-HIS-04`，并把 `FR-HIS-07`·`FR-HIS-08` 登记进 `verify_doc_refs.py::RETIRED_CODES`（退役编号须带修正说明，否则仍判失败）	

EP-2 的 3 个真实缺陷修复（详见 §7.3 的 DOD-EP2 执行记录）：登录失败计数被事务回滚吞掉导致限流失效、errors.py 序列化 ValueError 导致 500、未知邮箱限流哨兵被误删导致邮箱可枚举。三者都**不会在单次请求的返回值上暴露**，只有连续调用 / 边界输入才会命中 —— 因此对应的断言已固化进 verify_auth.py。

EP-5 的 1 个真实缺陷与 8 处判据错误（详见 §7.6.3 / §7.6.5 / §7.6.6）：🟤 **horizon=30 曾击穿 60 s 硬约束**（72.2 s），根因是 _roll_multistep 每步都触发一次全量 pandas 特征建表；修复后 23.8 s，并由 verify_ep5_equiv.py 对 81 条 L1 × 3 档步长断言**逐位相等**。另有 8 处是**校验脚本自己**的判据错误（代码是对的），其中两处最易踩：① 本项目错误契约是 **HTTP 200 + body.code**，按 RESTful 直觉写 assert status == 400 会全假失败；② Settings 是 @dataclass(frozen=True)，「换个配置再跑一次」只能靠**独立子进程 + 环境变量**，同进程改属性直接 FrozenInstanceError。

EP-9 的 5 个真实缺陷与三条根因经验（详见测试报告 §3 / §4）：

① 🟤 **「口径收敛」必须全库搜一遍，不能只改记得的那两处** —— F-02 把越权口径统一为 40301 时改了结果接口与删除，却漏了 API-13 任务状态，结果同一个「别人的资源」在结果页说 403、在轮询接口说 404（F-05）。**这条只有在「上游全量回归」里才会暴露**：EP-9 自己的 5 套脚本全绿，是跑 verify_ep5_api 才把它揪出来的。

② 🟤 **断言有生命周期** —— EP-5 阶段为「结果页尚未实现」写的两条临时断言（要求 data-phase=succeeded + 页面标注「EP-6 补全」），在 EP-6 落地后**必然失败**（T-22）。阶段断言若不在下一阶段主动升级，就会退化成噪音，或者被人「顺手放宽」而失去守门能力。EP-3 的 header === 1 同理（T-21）。

③ 🟤 **夹具缺失会让整轮走查失去意义** —— verify_ep6_browser.mjs 硬编码依赖一个预设的游客任务（TASK_ID=19），在 EP-9 的独立空库上直接 [FATAL] 退出；补出任务后又因夹具全是 L1 序列，让「按 L2 过滤」用例拿到空集、断言恒假（T-23）。新增 scripts/_ep9_make_fixture.py 统一解决。

---

7. 各阶段任务分解与完成门槛（WBS）

每阶段的「完成定义（DOD）」均为**可验证项**：能用命令跑出结果或能人工走查确认。**未全部勾选不得进入下一阶段。**

7.1 EP-0 工程基建与设计系统移植

编号	工作项	要点
WS-0-01	创建前端工程 web/	Vite 5.4 + React 18.3 + TS 5.4；package.json 版本按 §2.3 锁定；npm 源用淘宝镜像（国内网络）
WS-0-02	移植 56 个设计令牌	prototype/assets/app.css → src/theme/tokens.css；🔒 **oklch() 原样保留，禁止改写为 hex**
WS-0-03	AntD 主题桥接	antd-token.ts：把品牌色/语义色/圆角/字号映射到 ConfigProvider；深色用 theme.darkAlgorithm
WS-0-04	ECharts 主题桥接	echarts-theme.ts：颜色从 CSS 变量读取；监听主题切换事件重绘（UX-GOV-02a）
WS-0-05	4 个布局模板	AppLayout（模板 A）/ AsideLayout（B）/ AuthLayout（C）/ MarketingLayout（D）
WS-0-06	后端骨架	app/main.py + 中间件链 + 路由挂载 + 启动钩子；app/core/config.py（Settings + BASE_DIR 路径常量）
WS-0-07	/health 接口	API-21；返回 {"code":0,"message":"ok","data":{"status":"ok","model_loaded":...,...}}
WS-0-08	统一异常处理与日志	core/errors.py 业务异常类 + 处理器；core/logging.py
WS-0-09	环境变量与忽略文件	.env.example（全量变量按技术文档 §2.4）；.gitignore（含 .env、data/、node_modules/）
WS-0-10	依赖清单落盘	requirements.txt（技术文档 §1.2）+ package.json

DOD-EP0

☑ GET /health 可访问且返回结构符合技术文档 §10.5 期望

☑ npm run dev 可渲染空布局；4 个模板各有占位路由可切换

☑ npm run typecheck 与 npm run lint 零错误

☑ tokens.css 中 oklch(出现次数 **≥ 56**，# 十六进制色值 **0 个**（脚本校验）

☑ 前后端依赖版本与 §2.3 锁定表**逐项一致**（pip freeze / package.json 核对）

☑ 🔒 data/ 与 outputs/ 均在工作区内，无写入系统盘的路径

EP-0 **执行记录**（EP-2 阶段回归复核，2026-10-09）：`verify_tokens.py` ✅（**oklch 71 处 / hex 0 处**）；`tsc --noEmit` + `eslint --max-warnings 0` + `vite build` 全绿；后端 11 个依赖逐项比对 §2.3 锁定表**全部一致**（`openpyxl==3.1.5` 于本次复核时补齐）。

7.2 EP-1 数据库落地与数据访问层

编号	工作项	要点
WS-1-01	创建 `data/` 目录结构	`app.db` / `uploads/u{user_id}/` / `exports/` / `exports/tmp/`
WS-1-02	落实 CH-01 数据库变更	`schema.sql` 增 `role` 列 + `action` 8 类；`seed.sql` 增管理员种子账号（§5.3）
WS-1-03	执行建库	`sqlite3 data/app.db < code/db/schema.sql` + `seed.sql`；🔒 幂等，可重复执行
WS-1-04	开启 WAL 与连接配置	`PRAGMA journal_mode=WAL`；SQLAlchemy `connect_args={"check_same_thread": False}`
WS-1-05	8 个 ORM 实体	与 `schema.sql` 一一对应；🔒 **时间字段统一 `TEXT`**，不用 SQLAlchemy `DateTime`（BE-01）
WS-1-06	DAO 层	纯 CRUD，无业务判断；🔒 不得 `import algo`
WS-1-07	`get_user_by_email()` 单点封装	🔒 内部必须写 `WHERE email = ? COLLATE NOCASE`，否则索引失效（DB-17）
WS-1-08	迁移脚本	`scripts/migrate_0001_ch01_role.py`：幂等 + 自动备份 + `PRAGMA user_version`

DOD-EP1

☑ `data/app.db` 生成；**8 表 + 1 视图 + 3 触发器 + 12 索引**齐备

☑ `users.role` 列存在；写入 `'bogus'` 被 CHECK 拒绝（实测）

☑ 管理员种子账号存在且 `role='admin'`；游客哨兵 `id=0` 为 `role='user'`

☑ operation_logs.action 接受 'admin'，拒绝未知值

☑ verify_schema.py ✅ / verify_doc_sql.py ✅（121 列字典）/ explain_queries.py ✅（11 条 → 10 条命中索引）

☑ 迁移脚本连跑两次不报错（幂等）；执行前备份文件存在

☑ 🔒 红线自检：grep -r "import algo" app/dao/ 零命中；grep -r "DateTime" app/models/ 零命中

EP-1 执行记录（EP-2 阶段回归复核，2026-10-09）：verify_schema.py ✅（8 表 / 12 索引 / 3 触发器 / 1 视图 / 15 处 CHECK / 幂等 / 哨兵与外键完整性）；verify_doc_sql.py ✅ 121 列一致；verify_orm.py ✅ 含红线自检；真实库 data/app.db 中 users 12 列、password_hash 全为 bcrypt 串。

7.3 EP-2 认证与用户

编号	工作项	要点
WS-2-01	core/security.py	bcrypt 哈希/校验；JWT 签发/校验（载荷含 ver = token_version）；🔒 密码 >72 **字节** 显式报错（JT-12）
WS-2-02	core/deps.py	get_db / get_current_user / get_optional_user（游客态）/ **get_current_admin**
WS-2-03	auth_service	注册（重名拒绝）/ 登录（限流）/ 登出 / 改密（token_version + 1）
WS-2-04	API-01 ~ API-05	请求响应严格对齐 code/backend/schemas.py
WS-2-05	登录限流	LOGIN_MAX_FAIL=3 / LOGIN_LOCK_SECONDS=60；fail_count / locked_until 落库（非内存）
WS-2-06	UX-P02 登录页	三态：登录中 / 密码错误 / 限流冷却；删除演示账号信息块（UX-GOV-01）
WS-2-07	UX-P03 注册页	密码强度实时校验（含 72 字节上限）；邮箱已注册提示

编号	工作项	要点
[ ] WS-2-08	前端认证状态	useAuthStore（含 isAdmin 派生）；路由守卫（未登录跳 /login）

DOD-EP2

☑ 闭环可跑：注册 → 登录 → GET /users/me → 改密 → **旧 Token 立即失效** → 登出

☑ 连续 3 次密码错误后触发冷却，第 4 次被拒且提示剩余时长

☑ verify_schemas.py ✅（**57/57**，含 72 / 73 字节边界用例）

☑ 🔒 数据库中无明文密码（grep 或脚本核对 password_hash 均为 bcrypt 串）

☑ 前端 UX-P02 / UX-P03 三态均可复现；.demo-accounts 零残留

EP-2 执行记录（2026-10-09） —— 全部通过，证据如下：

证据	内容
scripts/verify_auth.py	✅ **39/39**（独立临时库，不污染 data/app.db）。含未知邮箱防枚举、get_current_admin 权限探针、审计落库、无明文密码
web/ 校验	tsc --noEmit ✅ / eslint --max-warnings 0 ✅ / vite build ✅（2074 模块，32s）
无头浏览器（Edge 151 CDP）	登录页正常态 / 密码错误态 / 限流倒计时态 + 注册页弱密码态，**4 张截图** **consoleErrors 全为 0**；证据见 outputs/ep2-shots/
回归	verify_schema.py ✅ / verify_orm.py ✅ / verify_doc_sql.py ✅（121 列）/ verify_doc_refs.py ✅（0 悬空）/ explain_queries.py ✅ / verify_types.mjs ✅

EP-2 期间发现并修复的 3 个真实缺陷（均非纸面问题，均已被脚本或浏览器断言钉住）：

#	缺陷	后果	修复

| 1 | 登录失败时直接 `raise BizError`，`get_db` 依约回滚 → **失败计数与失败审计一并丢失** | **限流永不生效**，攻击者可无限试密码 | `auth_service.login()` 在抛错前显式 `db.commit()`（BE-17 的正当例外），并在注释中说明原因 |

| 2 | `errors.py` 把 `exc.errors()` 原样塞进响应，而 pydantic v2 的 `ctx` 里带 **`ValueError` 对象** | `json.dumps` 抛 `TypeError` → 「密码强度不足」变成 **500**（与 RG-06 同型） | 新增 `_safe_errors()` 消毒；并给处理器加自身兜底 |

| 3 | 未知邮箱的内存计数用 `datetime.min` 当「未锁定」哨兵，被「判断是否已解锁」的逻辑误删 | 未知邮箱**永远不锁定** → 可用「第 4 次是否 40104」**枚举已注册邮箱** | 改用 `None` 表示「计数中」，与「已锁定」语义分离 |

遗留（不阻塞 EP-3）：`token` 黑名单为进程内存（BE-19 方案 B），多进程部署需迁 Redis（T-Q12）；「忘记密码」按 PRD 为 P2，前端按钮置灰占位；管理端导航待 EP-8 注册路由后再插入，避免死链。

7.4 EP-3 前台产品首页与全局框架

编号	工作项	要点
WS-3-01	UX-P01 产品首页（模板 D）	价值主张 / 硬指标 / 五步流程 / 数据集字段说明 / 与天池差异对照
WS-3-02	硬指标数字来源	📌 **构建期注入**（QD-06 已落地）：`outputs/eval_report.json` → `scripts/sync_eval_metrics.py` → `web/src/generated/eval_report.json` → `metrics.ts` 只做派生与格式化。**前端源码内不出现任何实测数值字面量**，由 `verify_ep3.py` 的 ①-5 守卫
WS-3-03	全局页脚免责声明	🔒 FR-SYS-05，**全站每一页可达**，不可折叠
WS-3-04	全局基础组件	错误边界 / 空状态 / 骨架屏 / 提示条
WS-3-05	主题切换	深浅主题；选择持久化到 `localStorage`；切换后 ECharts 重绘
WS-3-06	原型痕迹清理	删除 `.state-bar` / `.spec-note` / `.diff-note`；`README.html` 不迁移

DOD-EP3

☑ 产品首页可访问，四个区块内容完整

☑ 🔒 首页展示的硬指标与 `outputs/eval_report.json` **逐项一致**（脚本比对，非人工眼看）

☑ 🔒 免责声明在任意页面可见（走查 ≥ 3 个不同模板的页面）

☑ 搜索 `.state-bar` / `.spec-note` / `.diff-note` / `REAL_EVAL` → **零命中**

☑ 深浅主题切换后，图表颜色随之变化（`UX-GOV-02a`）

`EP-3` **执行记录（2026-10-09）** —— 全部通过，证据如下：

证据	内容
`scripts/verify_ep3.py`	✅ **29/29**（静态：指标同源 / 痕迹清理 / 页脚单点与不可折叠 / 四区块 / 全局组件 / echarts 引入点守卫）
`scripts/verify_ep3_browser.mjs`	✅ **47/47**（Edge 151 无头 + CDP；4 个模板走查 + 硬指标逐项比对 + 主题切换 + 每页 `consoleErrors=0`）
`scripts/sync_eval_metrics.py`	✅ `--check` 通过（注入件与算法产出深度相等）
前端三件套	`tsc --noEmit` ✅ / `eslint --max-warnings 0` ✅ / `vite build` ✅（2084 模块）
截图	`outputs/ep3-shots/`：`home` / `login` / `upload` / `dashboard` / `home-light` 共 5 张 + `browser_probe_summary.json`

DOD 五项逐条证据

DOD 项	证据
首页四区块完整	结构探针：`.hero-facts .hero-fact`=**4**、`.feat`=**4**、`.steps .step`=**5**、`.dataset-card`=**1**、`.eval-metrics [data-metric]`=**4**
🔒 硬指标逐项一致	浏览器读 `data-value`：`weighted_mape 14.26/20`、`mae_improve_pct 26.42/15`、`rmse_improve_pct 27.52/15`、`series_pass_rate 83.95/60` —— 与 `outputs/eval_report.json` 的 `verdict.checks` **逐项相等**（脚本比对）；首屏 3 个「实测」小字同源

| 🔒 免责声明全站可达 | 走查 **4 个模板**（D=/、C=/login、B=/upload、A=/dashboard）：`.disclaimer` 均 `display=flex`、`offsetHeight=66>0`、`scrollHeight(64) ≤ clientHeight(64)+1`、`details/summary=0`、111 字含 3 个关键句 |

| 原型痕迹零命中 | `web/src/` 全量扫描（**已剥注释**）：`.state-bar`/`.spec-note`/`.diff-note`/原型演示指标常量 **0 命中**；`README.html` 未迁移；演示账号痕迹同步回归 0 命中 |

| 主题切换图表变色 | 切换前 `--c1=oklch(0.72 0.15 205)`、`strokeSig` 含该值；切换后 `--c1=oklch(0.61 0.155 205)`、`strokeSig` 变为 `oklch(0.61 0.155 205) | oklch(0.645 0.155 145) | oklch(0.905 0.006 250)`；`html.dark` 翻转为 `false`；`localStorage.sp_theme` 由 `dark` → `light` |

EP-3 期间发现并修复的 2 个真实缺陷

#	缺陷	后果	修复
1	首页图表靠 **ECharts 默认色板**取色，而两条「不可见但占位」的置信带系列也消耗色位	线条实际拿到第 3/4 个颜色，**自绘图例的色块却是第 1/2 个** —— 图例与图表对不上（放大截图肉眼可见）	系列**显式**绑定令牌色（`readTokenVar('--c1'/'--c2')`），并以 `isDark` 为 `useMemo` 依赖在换主题时重建 option
2	`useThemeStore` 在模块初始化时调用主题注册，而它静态 `import` 了 echarts	① echarts（压缩后 ~460 KB）被拉进主包，首屏图表懒加载**形同虚设**；② 注册发生在 CSS 注入之前，`getComputedStyle` 取到空色板	把「注册主题」下沉到唯一使用者 `EChart`（`init` 前同步注册），`useThemeStore` 与 `theme/echarts-theme.ts` **不再 import echarts**；主包 **1381 kB → 343 kB**（gzip 462 → 117 kB），echarts 落入懒加载分包（1,039 kB）

与 UI/UX 文档的偏离登记（EP-3，共 4 项，均已在代码内注明理由）

#	文档写法	实际实现	为什么偏离 / 是否违反文档
1	§7.1「右：**SVG 示意图**（浏览器框样式 + 历史实线 / 预测虚线 / 置信带 + 图例）」	ECharts，`renderer: 'svg'` —— DOM 仍是 SVG，浏览器框 / 实线 / 虚线 / 置信带 / 图例五项要素全部保留	**不违反**：手写 inline SVG 的颜色是硬编码字面量，**不可能随主题变化**，与 `UX-GOV-02a`「图表色随主题切换」直接冲突。改用 ECharts 后既能保持 SVG 形态，又落实了主题联动与「令牌是唯一真源」（`UX-GOV-02`）
2	§7.1 首页 = 四个区块（hero / 核心能力 / 使用流程 / 基准数据集）	四区块**原样保留**，另在 hero 之后追加一条**硬指标实测披露条**（`.eval-metrics`）	**不违反**：`DOD-EP3` 明确要求「首页展示的硬指标与

eval_report.json 逐项一致」，必须有承载位。该条不作视觉上的第五「区块」（无 section-head 居中标题），而是 hero 的可信度延伸 |

| 3 | 原型用 .diff-note 承载「与天池差异对照」 | 类名改为 .dataset-diff | **按文档执行**：WS-3-06 明确要求删除 .diff-note（原型专用标注类），故保留**内容**、更换类名 |

| 4 | 全局统一使用新建的 NoticeBar 提示条组件 | 登录 / 注册页**保持** EP-2 的内联 .alert-danger 写法不动 | **刻意不重构**：那两页的提示条已被 EP-2 的无头浏览器断言钉住（失败次数文案、冷却倒计时），为一次纯装饰性重构引入回归风险不划算。NoticeBar 供 EP-3 之后的新页面使用 |

遗留（不阻塞 EP-4）：探针临时库 data/tmp/ep3_probe/ 因 data/ 属受保护路径、删除被安全守卫拦截而残留（约 300 KB，已 gitignore，无功能影响）；web/dist 中 ECharts 分包自身仍 >500 kB，属 EP-9 性能专项（可进一步按需拆分或换 echarts/core 按需引入）。

7.5 EP-4 数据集上传与校验

编号	工作项	要点
WS-4-01	流式落盘	aiofiles；按 u{user_id} 分目录；🔒 文件名用 UUID（NFR-SEC-07）
WS-4-02	7 步校验顺序	严格按 FR-UPL 定义顺序；🔒 **仓库列缺失即拒**（FR-UPL-05）
WS-4-03	5 类结构化错误	40001 缺字段 / 40002 日期异常 / 40003 负值 / 40004 序列超限 / 40005 时间断档
WS-4-04	上限拦截	20MB（MAX_FILE_SIZE_MB）/ 50 万行（MAX_ROWS）/ 1000 序列（MAX_SERIES）
WS-4-05	断档处置	检测到断档**不自动继续**，先返回待确认 → 用户选「自动插值」→ 置 gap_filled=1
WS-4-06	游客态	OptionalUserId；未登录归属 user_id = 0（哨兵）
WS-4-07	API-06 ~ API-11	含错误明细下载（API-11，临时目录 exports/tmp/，TTL 1 小时）
WS-4-08	UX-P05 上传页	拖拽区 + 预览表 + 校验面板 + 模板

编号	工作项	要点
		下载 + 错误明细下载；`AsideLayout`

DOD-EP4

☑ 5 类错误**各自可复现**，且提示均含「**错在哪一列/哪一行 + 怎么改**」（`UX-AC-05`）

☑ 三类上限各自被拦截并给出可操作提示

☑ 断档文件出现「待确认」状态，选择插值后可继续

☑ 🔒 游客未登录状态可完成上传并跳到参数页（归属 `user_id=0`）

☑ 🔒 错误明细 CSV 可下载；与正式导出**分目录**

☑ 中文列名文件被拒，提示指出「需使用英文列名 `sku_id/warehouse_id/date/sales`」（`JT-01`）

`EP-4` **执行记录（2026-10-09）** —— 全部通过，证据如下：

证据	内容
`scripts/verify_ep4.py`	✅ **77/77**（静态：7 步顺序常量同源 / 错误码齐备 / 路由顺序守卫 / 游客依赖范围 / UUID 落盘名 / TTL 分目录 / 前端 7 组件装配 / 🔒 预览表动态列 / 原型痕迹零命中）
`scripts/verify_ep4_api.py`	✅ **119/119**（端到端：五类错误 + 三类上限 + 断档两分支 + 游客链路 + 明细下载与 TTL + 幂等 + 分级预判同源 + `BE-41` 两分支 + 架构红线自检）
`scripts/verify_ep4_browser.mjs`	✅ **173/173**（Edge `155.0.4283.45` 无头 + CDP；**18 个真实状态**逐个走查，走查耗时 52 s）
前端三件套	`tsc --noEmit` ✅ / `eslint .` ✅（0 告警）/ `vite build` ✅（2,095 模块）
截图	`outputs/ep4-shots/`：`01-idle` ~ `18-search` 共 **18 张** + `browser_probe_summary.json`

DOD 六项逐条证据

DOD 项	证据

| 5 类错误各自可复现且含「错在哪 + 怎么改」 | 浏览器实测 5 张错误卡：`40001` 表头徽标组 4 个必需列逐个标注 ✓/✗ 并**点名** `warehouse_id（缺失）`；`40002`/`40003` 明细表给出「行号 + 原始值 + 问题说明」三列（各 ≥3 行）；`40004` 给出「当前 1,001 vs 上限 1,000」对比卡；`40005` 断档区间表 2 行（8 天 / 3 天）+ 二选一按钮。5 张卡**均含**「如何修正」块，且报错阶段行与 `VALIDATE_ORDER` 的中文阶段名**逐字对应**（第 3 / 4 / 5 / 6 / 7 步） |

| 三类上限各自被拦截并可操作 | ⏱ 20 MB：21 MB 文件**流式中断**（未读完即拒，1.34 s）；500,000 行：500,001 行 **0.18 s** 快速失败；1,000 序列：1,001 序列被拒并提示「超出 1 条」。三者前端另有**即时**前置拦截（`40007`/`40009`），提示形态与后端回包一致 |

| 断档「待确认」→ 插值后可继续 | `gap_action=interpolate` → `gap_filled=1`，行数 **69 → 80**（2 序列 × 40 天，缺失日期被补齐）；`gap_action=abort` → 记录作废、原始文件不变。浏览器走查 4 个状态留证：`10-err-40005`（停在二选一，文案明示「系统不会自动继续」）→ `14-interpolating`（忙碌面板，**不**套 7 步面板）→ `15-gap-resolved`（通过卡 80 行）→ `16-gap-abort`（回空态 + 「已放弃本次上传」） |

| 🔒 游客未登录可跑通并归属 `user_id=0` | `API-06` ~ `API-11` 六个接口全部改用 `OptionalUserId`（裁定 B）；游客上传 → `datasets.user_id = 0` → 详情 / 预览 / 断档处置**均放行**；`assert_dataset_readable` 的「游客数据匿名共享」分支先于「非本人即 `40301`」判（`BE-41`）。实测：用户 B 读用户 A 的数据集 → `40301`；用户 B 读游客数据集 → 放行 |

| 🔒 错误明细可下载且与正式导出分目录 | 落 `data/exports/tmp/dataset_{id}_{upload}.csv`，与正式产物 `data/exports/` **分目录分口径**（`API-11-2`）；TTL **1 小时**，过期返回 `40402`。`40007`/`40009` 两类**未完整落盘**的失败没有数据集行，`dataset_id` 为 `null`，**不渲染**下载按钮（避免用户点出 `40402`）—— 浏览器断言 `downloadErrors === false` |

| 中文列名被拒且提示英文列名 | `商品编码,仓库编码,日期,销量` 被判 `40001`，提示块点明需使用英文列名 `sku_id` / `warehouse_id` / `date` / `sales`（`JT-01`） |

内置示例数据的事实登记（`outputs/models/demo_sales.csv`，`API-08` 的数据源）

项	事实	为什么必须登记
规模	**14,305 行 / 40 商品 × 5 仓 = 113 条序列**	⚠ 与天池全集（963 商品 × 5 仓 ≈ 4,815 条序列、约 450 天）**不是一回事**。示例数据是**抽样**，用于「一键看效果」，不承担评测口径
时间范围	**2014-10-10 ~ 2015-05-17（220 天）**	与 `eval_report.json` 的评测窗口同源；🔒 改动该文件后 `level_preview` 与 `estimated_ms` 都会随之变化，必须重跑 `eval_report.py`
分级预判	`L1 = 81`、`L2 = 32`、`L3 = 0`，合计 **113**	`API-09` 的 `level_preview` 直接来自它；浏览器走查断言的 `(81,32,0)` 也是这一条

| 用途声明 | 右栏「数据用途」块写明：仅用于本次预测计算、不作其他用途、未登录数据 24 小时后清理 | NFR-SEC-05 最小暴露；已固化为浏览器断言 |

与算法真源同源的「防两套数字」断言（GIT-7）

断言	结果
API-09 的 estimated_ms **不得**独立估算	实测 7,900 ms = round(eval_report.timing_ms.total_ms × 本站序列数 / coverage.total)，与文档示例一致
API-09 的 level_preview 必须来自真实分级	实测 (L1, L2, L3) = (81, 32, 0)，三项之和 = 113，与 eval_report.json 的 coverage **深度相等**
预览列 = 源文件列（FR-UPL-07）	🔒 浏览器断言预览表头**恰好** ['sku_id','warehouse_id','date','sales']；内部辅助列 _d / _s **零泄漏**
路由顺序守卫	从 app.openapi()["paths"] 的 **dict 插入序**断言 /datasets/sample、/datasets/template、/datasets/gap 都在 /datasets/{dataset_id} **之前**
API-07 幂等键必须含 user_id	同一文件游客拿到的 dataset_id ≠ 登录用户拿到的 dataset_id，实测通过
架构红线自检	🔒 services 不 import algo.train；api 层不直接 import dao；service 不 import fastapi（用 **AST 取真实 import**，不扫文本）

EP-4 期间发现并修复的 4 个真实缺陷

#	缺陷	后果	修复
1	🔒 **失败记录被 get_db 回滚吞掉**	五类错误都能正确复现（断言全过），但**库里查不到任何行** —— API-11 永远拿不到错误明细，断档「待确认」行也一并消失。根因是 core.deps.get_db 对**任何**异常（含 BizError）都 rollback	在 register_upload 的三个失败出口（内容级失败 / 断档 abort / 断档待确认）**显式 db.commit() 后再抛**。教训：「复现得了但查不到」是这类缺陷的典型表象
2	预览泄漏内部辅助列	preview.columns 返回 ['sku_id','warehouse_id','date','sales','_d','_s']，FR-UPL-07「表头与源文件一致」直接失败	_build_preview 过滤 startswith("_") 的列
3	断档 40005 未回带 dataset_id	前端拿不到 id → 点「自动插值补齐」必报 40012: 参数 dataset_id 不合法（断档链路**完全不可用**）	断档分支 payload 补 dataset_id

| 4 | §3.8 清理的过期判定口径不一致 | 「文件级」按 `mtime`、「库级」按 `created_at`，两者被拆开后对不上 → 该清的没清 / 不该清的误清 | 改为**以库删除行反查落盘名**为准，`mtime` 只作兜底；未过期的引用文件登记进 `skipped` |

测试自身的 5 处缺陷（同样登记，避免下次再犯）

#	缺陷	修复
1	架构红线自检把**注释里出现的** `algo.train` / `fastapi` 判成违规（脚本自身误报 2 项）	引入 `_imported_modules()`，用 `ast.walk` 取真实 `import`。⚠ **这是本项目第 4 次同型教训**（`EP-1` docstring、`EP-3` 注释、`EP-4` 注释 ×2）
2	断言用 `"1000"` 而实际文案是千分位 `1,000`	改断言 `"1,000"`
3	落盘目录硬编码 `uploads/u1`	实际首个注册用户是 **id=2**（`seed.sql` 的管理员种子占 id=1）；改为动态查 `_user_id(con, email)`
4	对可能为 `None` 的查询结果直接 `row[0]`	加空值判断
5	浏览器探针 5 处缺陷	见下方「走查脚本踩坑」

浏览器走查脚本踩的 5 个坑（已全部固化为脚本内的注释与防御）

#	现象	根因 / 修复
1	「`consoleErrors` 必须为 0」被**环境噪声**打成假失败	本机 Edge 装有厂商插件，会注入 `/hybridaction/...` 追踪脚本、抛未捕获异常、并让 react-router 打出 `No routes matched` 警告。修复：加 `--disable-extensions` **且**按来源过滤（只放行 `chrome-extension://` / `/hybridaction/` / `runtime.lastError` 三类可证伪的来源）
2	右栏两卡断言全部失败，取到的是「工作台 / 新建预测」	模板 B 有**两个** `aside`：`[0]` 是导航侧栏、`[1]` 才是右栏。修复：**按文本查找**，不写下标
3	`headerBadges` 长度翻倍成 8	选择器 `span` 把徽标内部的 ✓/✗ 图标也收了。修复：限定 `span.rounded-full`
4	断档插值 / 放弃两个小节全部 `phase=null`	探针**漏点「开始上传」**：本页设计是「选中后停在待上传态」，不点按钮就不会上传。修复：抽 `pickAndUpload()` 复用
5	脚本空转 **15 分钟**、一张截图都没产出，且不报错	复用固定 `--user-data-dir`，上次残留的 Edge 子进程仍持有 profile 锁 → 新实例静默挂起。修复：① 每次用**全新** profile 目录；② 退出时 `taskkill /T` 杀**进程树**

（edge.kill() 只杀父进程）；③ CDP 端口按 PID 分散；④ 起跑前自检端口未被占用；⑤ 截图单次套 15 秒上限（原 120 秒 × 4 次重试 = 8 分钟/张） |

与文档 / 原型的偏离登记（EP-4，共 6 项 + 1 项提前落地）

| # | 文档写法 | 实际实现 | 为什么偏离 / 是否违反文档 |

|---|---|---|---|

| 1 | API-06-2「校验失败**不落盘不写库**」 | 内容级失败**建** status=-1 行；扩展名非 .csv 与体积超限两类（未完整落盘）**不建行** | **按裁定 A**：API-11 的路径是 /{dataset_id}/errors、ResolveGapRequest 也带 dataset_id、UX-P05 失败卡还要求下载按钮 —— 不建行则这三处全部不可用。两类未落盘情形无记录可指，登记为受控例外（ValidationErrorData.dataset_id = None） |

| 2 | §5.3 接口表标 API-09/10/11「鉴权 = 是」 | 三者改用 OptionalUserId，隔离由 BE-41 保证 | **按裁定 B**：DOD-EP4 明确要求「游客可跑通」，若这三条要登录，「上传→参数」链路会在预览那一步断掉 |

| 3 | 原型预览表固定 **6 列**（商品名 / 仓库两列来自 demo-data.js 演示映射） | 按 preview.columns **动态**渲染源文件列 | **不违反**：FR-UPL-07 要求「表头与源文件一致」，而真实 CSV 里没有那份演示映射。写死 6 列会直接违反该条 |

| 4 | 原型负值明细表 **4 列**（行号 / 原始值 / sku_id / warehouse_id） | **3 列**（行号 / 原始值 / 问题说明） | **受控偏离**：后端契约 negative_rows 只回 {row_no, value}；商品与仓库的完整信息在**下载的错误明细 CSV** 里保留（该文件含原始所有列 + error_reason） |

| 5 | 右栏「最少历史 **28 天**」「P99.5 / 均值 + 3σ 截断」 | 「建模门槛 **90 个有效点**」「滚动中位数 ± 3×MAD，营销节点豁免」，并补一行「编码 UTF-8（兼容 GB18030）」 | **修正为文档口径**：28 天无任何文档依据（FR-FEA-09 定的是 L1 ≥ 90 点）；实测截断用的是 MAD 而非标准差（大促尖峰会把标准差本身拉高，3σ 门限跟着抬高反而**放过**尖峰）。已由 verify_ep4.py 守卫「右栏不出现 28 天」 |

| 6 | UX-ST-05 断档分支 = 区间表 + 二选一按钮 | **不给**「下载错误明细」按钮 | **按文档执行**（非偏离，登记以防被「顺手统一」）：FR-UPL-06 要求导出的是「**不合格行**」，断档不是行级错误，导出语义不明的文件反而添乱。浏览器断言 downloadErrors === false |

| 7 | §3.8 数据清理原排期在后续阶段 | maintenance_service.py + scripts/cleanup_tmp.py **在 EP-4 提前落地** | **提前但必要**：40005 的失败数据集与 1 小时 TTL 的临时明细从 EP-4 起就开始堆积，不落地清理会让磁盘被校验垃圾挤占（JT-11 存在的理由）。三档 TTL 均已实测 |

遗留（不阻塞 EP-5）

- `estimated_ms` 的折算依赖 `eval_report.json` 的 `coverage.total`；🔒 重训模型后**必须**重跑 `eval_report.py --export-series`，否则该折算会用到旧的序列总量。

- 浏览器走查依赖前后端同时在线（脚本刻意不自己拉服务，避免留下僵尸进程）。运行配方已写进脚本头部注释。

- `verify_ep4_browser.mjs` 的「上传中」态靠 CDP 限速（40 KB/s）造出可截图窗口；机器负载极高时限速窗口会变窄，此时该项会明确报「未能进入 uploading 态」而不是静默跳过。

7.6 EP-5 预测任务编排 ★（全项目最高风险阶段）

编号	工作项	要点
WS-5-01	🔒 **先跑通离线脚本**	用 `scripts/` 现有脚本验证 `algo/` 链路，之后再接 Web（开发方案 §7.1 原则）
WS-5-02	`predict_service` 编排	`preprocess → features → classify → 移动平均基线 → infer → metrics`；🔒 `services/` 不得 `import algo.train`
WS-5-03	`model_holder.py`	模型单例加载；启动时校验 `ACTIVE_MODEL` 与 `model_registry.is_active=1` 一致（`BE-06`）
WS-5-04	任务状态机	`created → running → succeeded / failed`；失败带 `error_code` + `completed_stage`
WS-5-05	批量推理	🔒 **禁止逐序列循环**；一次前向处理全部序列
WS-5-06	GPU→CPU 回退	捕获 OOM → `torch.cuda.empty_cache()` → 模型与输入 `.to('cpu')` → 重跑；🔒 **不是简单重试**
WS-5-07	60 秒预算与软失败	阶段计时埋点；超预算 → `E_TIMEOUT` 软失败（返回已完成阶段与建议），**不是 502**
WS-5-08	`API-12` / `API-13`	创建任务 / 状态轮询（含五阶段进度与设备信息）
WS-5-09	`UX-P06` 参数配置	粒度**只读**（仓库×商品）；步长 7/14/30；🔒 **不提供「重新训练」选项**

编号	工作项	要点
WS-5-10	UX-P07 执行进度	五阶段实时进度 + 时间轴； GPU→CPU 回退披露；接近超时警告

DOD-EP5

- ☑ 🔒 示例数据集（113 序列，horizon=14）端到端 **≤ 60 s**（实测目标 ≈ 8 s）
- ☑ 🔒 五阶段耗时与 outputs/eval_report.json 的 timing_ms 同量级（推理段应为**毫秒级**，非秒级）
- ☑ 超时软失败可复现：人为压低预算后返回 E_TIMEOUT + 已完成阶段，进程不崩溃
- ☑ 🔒 强制 CPU 模式（TORCH_DEVICE=cpu）结果与 GPU 一致（同一输入对比预测值）
- ☑ 序列数超 1000 被拒并提示用户筛选商品
- ☑ 前端 UX-P07 五个阶段进度可观测；设备回退有明确披露文案
- ☑ 🔒 红线自检：grep -r "algo.train" app/services/ 零命中

7.6.1 EP-5 执行记录（2026-10-10）

全部通过。三套校验 + 前端三件套 + 截图留证如下。

证据	内容
scripts/verify_ep5.py	✅ **50/50**（静态：AST 扫描 8 个 services/ 模块零命中训练入口 / 游客链路两条路由无 RequireAuth / 五阶段常量前后端逐项相等 / 预算占比和为 1.0 且 inference 最大 / 软失败零 5xx / 预检早于建任务行 / 无「重新训练」入口 / 前端零实测数值字面量 / 右栏 useMemo 稳定引用 / 成本模型偏差 ≤ 25% / 两个 --check 注入物未漂移）
scripts/verify_ep5_api.py	✅ **60/60**（端到端：TASK_BUDGET_MS=1 → E_TIMEOUT 软失败 / MAX_SERIES=10 → 40004 且 tasks 行数不变 / CPU↔GPU 逐点比对 / 参数契约 / 游客越权）
scripts/verify_ep5_browser.mjs	✅ **41/41**（Edge 155.0.4283.45 无头 + CDP；参数页 4 区块 + 提交失败态 + 执行页真实跑到终态 + 结果页占位 + 上传页回归；**全程 consoleErrors = 0**）
前端三件套	tsc --noEmit ✅ / eslint . ✅（0 告警）/ vite build ✅（2,105 模块）
截图	outputs/ep5-shots/：01-params-default ~ 07-upload-regression 共 **7 张** + report.json

DOD 七项逐条证据

DOD 项	证据
🔒 端到端 ≤ 60 s	h=14 实测 **8.6 ~ 12.7 s**（多次测量，`verify_ep5_api` 客户端 wall 与服务端 `duration_ms` 互校），占预算 **14.3% ~ 21.1%**，非擦线通过。三档标定见 `outputs/_ep5_calib.json`：h=7 **11.3 s** / h=14 **16.6 s** / h=30 **23.8 s**（独立冷启动口径含数据读取与首轮特征建表）
🔒 五阶段与 `timing_ms` 同量级、推理段毫秒级	单遍分段 `{validate 24, preprocess 739, features 1162, inference 163, metrics 734}`；批量前向 **95 ~ 163 ms**。⚠ 见 §7.6.2 的口径分离：真正的大头是**滚动多步**（`rolling_ms` 2.0 ~ 3.5 s），它同属推理，故合并计入 `inference` 段（占比 0.610）
超时软失败可复现	子进程 `TASK_BUDGET_MS=1` → HTTP **200** / `code 0` / `status failed` / `error_code E_TIMEOUT` / `completed_stage metrics` / **3 条建议**；进程正常退出（exit 0）；`tasks` 行数 1 → 2（`BE-16` 显式 commit 生效），且该失败任务的 `predictions` / `metrics` **均为 0 行**（半成品不落盘）
🔒 CPU 与 GPU 一致	**两个独立子进程**各跑一份（`model_holder` 是模块级单例，同进程换 `TORCH_DEVICE` 会命中缓存句柄，得到假一致）。比对 **113 条 L1 序列 × 14 天 = 1,582 点**：最大绝对差 `8.6e-05`、最大相对差 `8.94e-05`（容差 `1e-4`）；加权 MAPE `14.25852073872085`（CPU）vs `14.258515529079979`（CUDA），相对误差 `3.6e-7`
序列数超限被拒且可操作	子进程 `MAX_SERIES=10` → `code 40004`，提示「待预测序列数为 113，超过上限 10，请先筛选商品（按近 90 天销量取 Top N）后重新上传」；🔒 `tasks` 行数 **2 → 2 不变**（预检在落库之前，`API-12-1`）
前端五阶段可观测 + 设备披露	浏览器断言 `stage-row-*` **恰好 5 行**；时间轴 5 段宽度**互不相同**且 `inference` 最宽（实测 `batch_kile=163 ms`/56%）；`log-stream` 8 行摘要；「推理设备」区显示 `NVIDIA GPU (CUDA)` / `31,729 参数` / `window 21 · 48×2`
🔒 红线自检	`verify_ep5.py` ① 用 **AST**（非正则）扫 `app/services/` 8 个模块的 import 列表 → 零命中；`app/api` 全量零命中 `/train`、`/retrain`、`/fit`。⚠ 用 `grep` 会被注释里的「不调用 `algo.train`」假阳性

7.6.2 `EP-5` 登记的口径偏离（**5 项**，均已落代码 + 加守卫）

#	偏离	原口径	现口径	守卫

| ① | `_timing_payload` 的 `algorithm_total_ms` | 只累加 backtest 一遍 | **两遍之和**（`backtest_pass_ms` + `forecast_pass_ms`） | `verify_ep5.py` 附加项「两遍耗时都上报」 |

| ② | `rolling_ms` / `fallback_ms` | **无埋点**，十几秒只能笼统写成 `unaccounted` | 已在 `algo.infer.Timing` 补两个字段，`unaccounted_ms` 收窄为「序列分级 + 结果装配」 | `verify_ep5.py` 附加项「`rolling_ms`/`fallback_ms` 已单独埋点」 |

| ③ | 🟡 `API-09.estimated_ms` 的含义 | 与参数页预估共用一个数 | **单遍评测基准**（7,873 ms，`eval_report.json` 口径）；新增并列字段 **`estimate_by_horizon`** 给参数页（三档端到端预估） | `verify_ep5_api` ①「两者并列共存」+ `verify_ep5` 附加项 |

| ④ | `ui/ux` §8.3 原型时间轴预算占比 | 推理 `--c5` 36.7% | 以实测为准：推理段 **61.0%**（含滚动多步）；`STAGE_BUDGET_RATIO` 五项重新标定且和为 1.0 | `verify_ep5.py` ③/④ 两条断言 |

| ⑤ | 🟡 滚动多步的实现路径 | 每步调 `build_inference_windows` | 新增 `features.calendar_matrix` + `features.last_window_fast`（纯 numpy，等价而非近似） | `scripts/verify_ep5_equiv.py`（81 条 L1 × 3 档步长**逐位相等**，`np.array_equal`） |

⑤ 是本阶段**最重要的一次修复**，详见下方专门说明。

7.6.3 🟡 `horizon=30` 曾击穿 60 s 硬约束（本阶段最严重缺陷）

项	内容
现象	`horizon=30` 实测 **72.2 s**，`status=failed`（软失败），**击穿 60 s 硬约束**。而同一份数据 `h=7` 仅 17.5 s、`h=14` 31.3 s —— 耗时随步长**超线性**增长
根因	`algo.infer._roll_multistep` 每一步都调用 `features.build_inference_windows`，而后者的实现是「先对**整条序列**建 pandas 特征表（`to_datetime` + `strftime` + `isin` + `shift` + 2 × `rolling` + `concat`）、**再取最后 `window` 行**」⇒ 同一条序列被**重复建表 `horizon` 次**，且每次都付全额 pandas 固定开销
为什么 60 s 是「勉强够」而非余量充足	一次批量前向仅 **95 ~ 163 ms**（毫秒级，`DOD` 第 2 条正是这么要求的）。也就是说**性能全在滚动多步的 Python 侧循环里**，不在模型里
先排除的方案	`scripts/_ep5_bench_roll.py` 先试「只取尾部切片」—— 实测仅省 **11%**。因为瓶颈是 pandas 每次调用的**固定开销**（函数入口、列名构造、对象创建），不是行数
最终修复	新增 `features.calendar_matrix(dates)` 一次算好 `(n, 6)` 日历矩阵 + `features.last_window_fast(cal, sales)` 用纯 numpy 取尾窗。**等价性而非近似**：窗口内各列只依赖向前的至多 `ROLL_WARMUP=14` 个值（`lag_1`→1、`lag_7`→7、`roll_mean_7=shift(1)+rolling(7)`→8、

roll_mean_14→15），其余 6 列（sales/dow_sin/dow_cos/month_sin/month_cos/is_weekend/is_holiday）逐行独立 |

| **踩坑点** | _trailing_mean **必须用** sliding_window_view（O(m·w)，w 只有 7/14），不能用 cumsum 差分 —— 后者引入累加误差，会破坏与 pandas 的**逐位相等** |

| **修复后** | h=7 **11.3 s** / h=14 **16.6 s** / h=30 **23.8 s**（重复测量稳定），三档均留有 2.5 倍以上余量 |

| **等价性证明** | scripts/verify_ep5_equiv.py 把改造前的 _roll_multistep **逐字保留**为 reference_roll，对全部 **81 条 L1 序列 × 3 档步长**断言 np.array_equal → **3/3 通过**。同时重跑 eval_report.py --demo --horizon 14，指标**逐位不变**（14.26 / 26.42 / 27.52 / 83.95，覆盖 L1 81 / L2 32 / L3 0 / 合计 113） |

| **回归守卫** | verify_ep5.py 附加项：用 AST **只查 _roll_multistep 函数体内**的可执行代码，断言不含 build_inference_windows 调用。⚠ 不能全文匹配 —— run_inference 里第 163 行仍合法调用它（那是**一次**批量前向，建一次表是对的），且文件注释里多处提及该函数名 |

7.6.4 EP-5 新增交付物与脚本矩阵

| 类别 | 文件 | 作用 |

|---|---|---|

| **后端新增** | app/core/model_holder.py | 模型单例 + BE-06 启动一致性校验 + BE-15 预加载 |

| | app/services/predict_service.py | 五阶段编排 + 状态机 + 批量推理 + GPU→CPU 回退 + 60 s 软失败 |

| | app/services/cost_model.py | UX-P06-06 耗时预估的**唯一口径**（per_series = 73.4 + 4.671 × h） |

| | app/api/v1/predictions.py | API-12 / API-13（同步语义 + OptionalUserId） |

| **后端改动** | app/algo/features.py | 🟢 新增 calendar_matrix / last_window_fast 热路径 |

| | app/algo/infer.py | Timing 补 rolling_ms / fallback_ms；_roll_multistep 重写 |

| | app/algo/config.py | HORIZONS = (7, 14, 30) 单一真源 |

| | app/services/dataset_service.py | get_detail 增补 estimate_by_horizon |

| | app/main.py | 挂载模型预加载与卸载 |

| **前端新增** | web/src/components/Card.tsx | 三页共用的卡片外壳（复制三份会慢慢长得不一样） |

| | web/src/components/StepBadges.tsx | 四步主流程指示条（**不**按登录态隐藏任何一步） |

| | web/src/components/params/ParamsAside.tsx | UX-P06 右栏（当前选择 / 预测窗口 / 稀疏性提示 / 快捷跳转） |

| | web/src/components/run/{StageTimeline,StageList,LogStream,RunAside}.tsx | UX-P07 四个可视化组件 |

| | web/src/pages/RunningPage.tsx | UX-P07 执行页本体 |

| | web/src/pages/ResultPage.tsx | UX-P08 **占位**（EP-6 补全），保证游客链路有落点 |

| | web/src/generated/pipeline.ts | 流水线元信息构建期注入（QD-06 同型守卫，**请勿手改**） |

| **前端改动** | web/src/pages/ParamsPage.tsx | UX-P06 完整表单（原占位页全量重写） |

| | web/src/api/client.ts | 🟤 修 datasetApi.detail 泛型（详见 §7.6.5） |

| | web/src/vite.config.ts | 代理目标改为 VITE_API_TARGET 可配 |

| **脚本新增** | scripts/verify_ep5.py | 静态校验 50 项 |

| | scripts/verify_ep5_api.py | 端到端校验 60 项 |

| | scripts/verify_ep5_browser.mjs | 浏览器走查 41 项 |

| | scripts/sync_pipeline_meta.py | 流水线元信息注入（支持 --check） |

| | scripts/verify_ep5_equiv.py | 🟤 滚动多步改造前后**逐位等价**验证 |

| | scripts/_ep5_parity_run.py | 单设备推理落盘（CPU/GPU 比对的两半） |

| | scripts/_ep5_cfg_probe.py | 受控配置下的单次预测探针（软失败 / 超限用例） |

7.6.5 EP-5 修复的 1 个真实前端缺陷

| 项 | 内容 |

|---|---|

| **现象** | api/client.ts 的 datasetApi.detail 泛型写成 DatasetSummary，而后端实际返回 DatasetDetailResponse ⇒ 运行时 detail.file_name / detail.id **全为 undefined**，参数页渲染出空白的数据概要区 |

| **为什么没被立刻发现** | EP-4 的浏览器走查只覆盖了上传页，参数页当时还是占位页。tsc 与 eslint **都查不出来**（泛型不匹配在结构相同的两个类型之间不报错）—— 只有真实渲染才暴露 |

| **修复** | 泛型改为 DatasetDetailResponse，并在 types/index.ts 补齐 LevelPreview / Horizon / HorizonEstimate / DatasetDetailResponse |

| **回归守卫** | verify_ep5_browser.mjs ①「预估耗时有真实数值（不是 –）」「L1 批量规模取自分级预判」两条断言；且 ⑥ 🔒 全程 consoleErrors = 0`` —— 空白页通常伴随 React key 警告 |

7.6.6 EP-5 校验脚本自身的 8 处判据错误（**代码是对的，判据写错了**）

值得单列，因为这类错误比真缺陷更容易误导人 —— 看到 [FAIL] 会本能地去改产品代码。

| # | 判据错在哪 | 正确口径 |

|---|---|---|

| ① | STAGES / STAGE_LABELS / STAGE_BUDGET_RATIO 用 ast.literal_eval 取值 | 它们由 STAGE_* 常量**拼接**而成，必须**真 import** 取真值（verify_ep5.py 用 ImportFailed 哨兵显式区分「取不到」与「值为空」） |

| ② | 拦「重新训练」时按关键词全文匹配 | UX-P06-04 **要求**显式声明「不提供在线重训练」⇒ 带否定标记的命中是**合规且必需**的。判据须按「是否带否定前缀」区分，并额外断言「该声明必须存在」 |

| ③ | 拿 eval_report.json 的 timing_ms.total_ms（7,873 ms，**单遍**）去比 cost_model 预估 | 端到端实测在 outputs/_ep5_calib.json。拿单遍数字比端到端预估会得出「偏差 99%」的假告警 —— 与 §7.6.2 ③ 是同一处口径分裂 |

| ④ | 断言 MAX_SERIES 预检行号 < db.add() 行号 | 实现走的是 settings.max_series 与 task_dao.create()。判据须按 AST 找 series_count > settings.max_series 的 Compare 节点 |

| ⑤ | _roll_multistep 全文匹配 build_inference_windows | 必须**只查该函数体内的可执行代码**（见 §7.6.3 回归守卫） |

| ⑥ | 按 RESTful 直觉断言业务错误返回 HTTP 400/404 | 本项目契约是 **HTTP 200 + body.code**（除 401 类，见 errors.py::register_exception_handlers） |

| ⑦ | 同进程改 settings.task_budget_ms 触发软失败 | Settings 是 @dataclass(frozen=True)，运行期改不了 ⇒ 必须**独立子进程 + 环境变量** |

| ⑧ | datasets.status = 'ready' | 该列是 **INTEGER（1 可用 / -1 已失效）**，写字符串在 SQLite 宽松比较下不匹配任何行 ⇒ 假阴性 |

另有两条**辅助缺失**已一并补上：TestClient(app) 必须 with 才触发 lifespan（否则表都不存在）；API-01 注册返回 RegisterResponse{user_id, email} **不含 token**，须再调一次 API-02 才拿到 JWT。

7.6.7 EP-5 顺手补的可测试性钩子

StageTimeline 的各段原本只有 title 属性，没有 data-testid。而 DOD-EP5 第 2 条要求断言「5 段宽度互不相同且 inference 最宽」，靠 style.width 去算太脆、靠 title 则要等 hover。故补 data-testid="timeline-seg-{key}" + data-stage-ms + data-stage-pct。**这是钩子不是装饰** —— 补完后「时间轴 5 段宽度互不相同」这条断言才真正可判。

7.7 EP-6 结果可视化与导出 ★（核心交付页）

编号	工作项	要点				
WS-6-01	指标口径落地	🔒 **主口径 = 销量加权 MAPE**（Σ\	e\	/Σ\	y\	）；辅 = **达标序列占比**；基线 = **移动平均**
WS-6-02	🔒 **主指标只算 L1**	`levels_for_main=("L1",)`；混入 L2 会把 14.26% 拖到 ~25.9%，导致「模型不达标」误判				
WS-6-03	🔒 基线可比性	移动平均基线必须在**完全相同的预测点、相同序列集合**上计算				
WS-6-04	评测必须 `backtest=True`	先砍末尾 `horizon` 天再预测，否则等于「看着答案出题」				
WS-6-05	`API-14` 结果接口	结论 + 指标集 + 覆盖率 + 曲线 + 分布 + 判定				
WS-6-06	`API-15` 明细分页	🔒 不能并入 `API-14`（最大 30,000 行会让响应体到数十 MB）				
WS-6-07	`API-16` 导出	CSV / Excel；列序固定；🔒 **含 `granularity_level` 列**；中文不乱码				
WS-6-08	`UX-P08` 全要素	结论条 + 指标卡 + 覆盖率面板 + 折线对比 + 直方图 + 明细表 + 导出				
WS-6-09	超标警告	🔒 `MAPE > 20%` 时警告可见且**不可折叠**（`UX-AC-09`）；`L3` 序列无数值预测（`UX-AC-10`）				

`DOD-EP6`（✅ 2026-10-10 全部达成，验收证据见 §7.7.1）

☑ 🔒 结果页展示的每个指标与 `outputs/eval_report.json` **逐项一致**（14.26 / 26.42 / 27.52 / 83.95）—— `verify_ep6_api.py` 逐字段断言，四项全中

☑ 🔒 覆盖率三项之和 = total（49 + 32 + 0 = 81 类核对，UX-AC-06）—— API-14 响应内闭合校验，超限即报错

☑ 折线图同时呈现真值 / 预测 / 移动平均基线三条线，且基线在相同预测点

☑ 明细表支持按任意列排序 + 分页（默认每页 20 行）—— API-15 白名单排序，非法列名返回 40006

☑ 🔒 导出文件用 Excel 实际打开，中文不乱码，列序与 EXPORT_COLUMNS 一致 —— openpyxl 读回断言表头与 7 列全等

☑ 超标告警态可复现（阈值调低后），警告不可折叠 —— verify_ep6_browser.mjs §6 重放 is_over_threshold=true，断言不在 <details> 内

☑ L3 序列走查：不显示数值预测，显示降级说明 —— 稀疏数据任务实测（case_l3_sparse）

☑ 🔒 重训模型后执行 train_and_register.py + eval_report.py --export-series，界面数字随之更新（GIT-7）—— 结果页只经 API-14 取数，无本地缓存

7.7.1 EP-6 执行记录（2026-10-10）

交付物清单

层	文件	说明
后端新增	app/services/result_service.py（约 560 行）	结论 / 指标 / 覆盖率 / 分布 / 耗时 / 曲线 / 明细分页 / 导出，全部只读库内已落库数据
后端新增	app/api/v1/predictions.py（3 条路由）	API-14 API-15 API-16
后端改动	app/dao/prediction_dao.py	+SORTABLE_COLUMNS 白名单、+list_paged（关键词/分级过滤）、+iter_for_export
后端改动	code/backend/schemas.py	+PredictionDetailQuery / DetailPage / PredictionResultResponse.truncated；🟤 4 个字段改为可空（见下）
前端新增	components/result/（7 文件）	MetricPanels（结论条 + 指标卡 + 基线表）、CoveragePanel、LineCompare、DistributionChart、DetailTable、ExportButtons、ResultAside
前端改动	pages/ResultPage.tsx（占位 → 完整页）	七大区块组装、空态、超标告警、游客禁用导出
前端改动	types/index.ts	+truncated / +THRESHOLDS（阈值与 app/models/metric.py 同源）/ 4 个字段改可空

| 脚本新增 | scripts/verify_ep6.py / verify_ep6_api.py / verify_ep6_browser.mjs | 静态 65 / 接口 105 / 浏览器 42 |

验收结果

| 套件 | 结果 | 覆盖 |

|---|---|---|

| verify_ep6.py | **65/65** ✅ | WS-6-01~09 逐项；🔒 指标纪律组（顶层不 import algo / _metric_bundle 不做 round / 覆盖率取 tasks 冗余列而非明细行数）；前端零二次聚合 |

| verify_ep6_api.py | **105/105** ✅ | 指标与 eval_report.json 逐字段一致 / 覆盖率三项闭合 / API-15 排序白名单与分页上限 / 游客不可导出 / 越权 40401 / **稀疏数据（L3）全链路** |

| verify_ep6_browser.mjs | **42/42** ✅ | Edge 真实渲染，consoleErrors = 0；含超标告警态重放与不可折叠断言 |

| 前端三件套 | ✅ | tsc --noEmit 零错、eslint 零警、vite build 成功 |

7.7.2 EP-6 修复的 3 个真实缺陷

① 🔴 SeriesCurve.baseline 声明为非空，实际库内可为 NULL（L3 序列）

predictions.baseline_value 在 L3 分级下是 NULL（无历史即无移动平均），而 schema 写的是 baseline: list[float]。Pydantic 校验失败会让**整个 API-14 返回 500**，结果页白屏。

⚠ 示例数据集 L3 = 0，所以在演示路径上**永远不会暴露** —— 只有上传"每条序列都不足 21 天"的数据集才会触发。已改为 list[float] | None。

② 🔴 MetricBundle 6 个字段声明为非空，L1 = 0 时全为 None

主指标只算 L1（WS-6-02）。若上传的数据集**全部序列都是 L2/L3**，则 weighted_mape 等 6 个字段算不出来，全为 None，同样触发 500。这是"上传短序列数据"场景的必现故障。已全部改为可空，并连带修正前端三态配色。

③ 🔴 前端指标卡把 null（无法评估）渲染成红色（未达标）

修完 ② 之后 TS 强制暴露出来的分支：`ok: null ? green : red` 会把"算不出来"显示成"没达标"，用户会去调模型 —— 而问题在数据质量。已引入 `stateColor()` 三态：达标绿 / 未达标红 / **无法评估中性灰**。

> ### 7.7.3 EP-6 校验脚本自身的 5 处判据错误（**代码是对的，判据写错了**）
>
> ① **`strip_docstrings` 只剥 `"""` 与 `#`**，漏了 JS/TS 的 `/** */` 块注释 —— 前端源文件普遍用 JSDoc 写模块头说明（`ResultPage.tsx` 就在注释里写了「与 `outputs/eval_report.json` 同源」），等于前端文件根本没剥，`eval_report` 判据必然误报。
> ② **子串搜索命中 docstring**：`_metric_bundle` 的 docstring 里写着「接口层一旦 `round()`…就…」，那是**解释为什么不这么做**，却被判成「做了 round()」。`strip_docstrings` 的作用正在于此。
> ③ **`register` 响应不返 Token**：`RegisterResponse` 契约只有 `user_id` / `email`，早期版本直接取 `data.access_token` 恒为 `None`，后续所有带鉴权的用例全降级成游客。
> ④ 🟤 **`with client:` 提前退出导致模型单例被清**：`case_setup` 里函数返回时 lifespan 关闭，`holder.clear()` 清掉模型 → 后续需要模型的用例全返 `404`。**整个运行必须在同一个 lifespan 内**。这条最隐蔽：只读用例照常通过，只有要跑推理的用例才挂。
> ⑤ 🟤 **超标态重放拦错了对象**：前端 `api/client.ts` 用的是 **axios**（浏览器适配器走 `XMLHttpRequest`），从头到尾**不调用 `window.fetch`**。拦截 `fetch` 的探针「注入成功」但响应原样通过，三条断言全挂，页面显示的仍是真实达标数据 —— 看起来像功能坏了。正确做法是 `Page.addScriptToEvaluateOnNewDocument` + 改写 XHR 的 `responseText`（且必须用 `addScriptToEvaluateOnNewDocument`，页面一导航 JS 上下文就销毁，手写的补丁连同上下文一起消失）。
>
> 📌 **一条通用教训**：校验脚本失败时，**先分清是「代码坏了」还是「探针打错靶」**。本轮 3 个真实缺陷里有 2 个（①②）只在"示例数据集覆盖不到"的边界上暴露，靠构造稀疏数据任务才打到；反过来 5 处判据错误里有 2 处（④⑤）制造的假象是"功能缺陷"。两者的修复动作完全相反 —— 前者改产品代码，后者改脚本。

7.7.4 EP-6 登记的口径偏离（**3 项**）

编号	偏离	理由	守卫
BE-25	折线图的"真值段"取**数据集原始销量**而非 `predictions.actual_value`	在线预测未来无真值，`actual_value` 恒为 `NULL`；曲线左侧实线段只能来自原始 CSV 的历史尾部	`_series_history` 只取要画的那几条

序列（series_limit 截断后），否则 1,000 序列全取一遍会吃掉大半秒预算；CSV 缺失时降级为"只有预测段"而非整页 500 |

| BE-26 | 覆盖率取 tasks.l1_count/l2_count/l3_count 冗余列，而非 count_by_level(明细行) | 明细行数 = 序列数 × horizon，直接用会把覆盖率**放大 horizon 倍** | verify_ep6.py 断言 _coverage 函数体内不出现 count_by_level |

| BE-27 | MetricBundle / SeriesCurve 共 10 个字段改为可空 | 见 §7.7.2 ①②。空集场景是真实存在的（全部 L2/L3），非空声明会导致 500 | verify_ep6_api.py §⑥ 稀疏数据任务全链路 |

EP-6 的 3 个真实缺陷与 5 处判据错误（详见 §7.7.1 / §7.7.2 / §7.7.3）：两个 500 级缺陷都藏在**示例数据集覆盖不到的边界**上 —— 示例数据 L3 = 0，所以"L3 序列基线为 NULL"永不触发；示例数据 L1 > 0，所以"L1 = 0 时主指标全为 None"永不触发。只有构造"全短序列"数据集才会打到（已固化进 verify_ep6_api.py 的 case_l3_sparse）。另有 5 处是**校验脚本自己**的判据错误，最隐蔽的两处：① TestClient 的 with 块一旦提前退出，lifespan 关闭会清掉模型单例，后续要推理的用例全返 404（只读用例照常通过，极易误判为"偶发"）；② 重放超标响应时拦截了 window.fetch，但前端用的是 **axios → XHR**，fetch 从未被调用，探针"注入成功"却什么都没拦到。

📌 **一条通用教训**：校验脚本失败时**先分清是「代码坏了」还是「探针打错靶」**。本轮两个方向都出现了：真实缺陷靠构造边界数据才暴露，而假故障靠改脚本才消除 —— 修复动作完全相反。

7.8 EP-7 历史记录与对比 + 工作台

编号	工作项	要点
WS-7-01	API-17 历史列表	分页 + 筛选；经索引 IX-03；≤ 1 s（NFR-PERF-08）
WS-7-02	API-19 历史详情	参数快照**冻结**（不随当前配置变化）+ 阶段耗时回放 + 失败详情
WS-7-03	API-18 跨任务对比	🔒 **必须注册在 API-19 之前**（BE-39，否则 compare 被当作 task_id）
WS-7-04	API-20 软删除	tasks.is_deleted=1；不出现在列表；数据保留
WS-7-05	UX-P09 历史任务	汇总 + 筛选 + 列表 + 删除二次确认（**需输入任务 ID**）
WS-7-06	UX-P10 任务详情	参数快照 + 指标 + 曲线 + 阶段耗时

编号	工作项	要点
		+ 失败详情 + 重跑入口
WS-7-07	UX-P11 跨任务对比	最多 **4 个**任务并排；最优值高亮；数据文件不一致警告
WS-7-08	UX-P04 工作台	概览 + 快速入口 + 最近任务；FR-USR-08 游客态提示

DOD-EP7（✅ 2026-10-10 全部达成，验收证据见 §7.8.1）

☑ 历史列表、详情、对比三页均可用；列表首屏 ≤ 1 s —— 实测 API-17 首屏 **41 ms**（预算 1 s）

☑ 🔒 CURL 实测 `POST /history/compare` 返回对比数据（**不是** `40012`）—— 实测 **133 ms**（预算 3 s）；`verify_ep7.py` 断言 `history.router.routes` 中 `compare` 注册序号 < `{task_id}`

☑ 软删除后任务从列表消失；数据库中记录仍在（`is_deleted=1`）—— `verify_ep7_api.py` 断言 `predictions` / `metrics` 行数**删除前后完全相等**

☑ 删除二次确认必须输入正确任务 ID 才可执行 —— 浏览器实测四态：未输入 / 错 ID / 少一位均 `disabled`，正确才解禁

☑ 对比页选 2 个不同数据集的任务时出现「数据文件不一致」警告 —— 浏览器实测 `dataset-warning` 块渲染

☑ 工作台最近任务与历史列表一致 —— 实测 `hist=[3,2,1]` 与 `dash=[3,2,1]` 完全相同（工作台**复用 `list_history`**，前端不自聚合）

7.8.1 EP-7 执行记录（2026-10-10）

交付物清单

层	文件	说明
后端新增	`app/services/history_service.py`（约 620 行）	API-17~20 + 工作台聚合；软删除幂等实现；对比口径与归因

| 后端新增 | app/api/v1/history.py（5 条路由，169 行） | 🔒 文件头锁定 BE-39 注册顺序约束；全部 user: CurrentUser；阻塞调用走 run_in_threadpool |

| 后端改动 | app/dao/task_dao.py | +list_history / count_history / _history_filters（走 v_task_summary 视图）；🟤 soft_delete 潜伏缺陷修复 |

| 后端改动 | app/services/result_service.py | 🟤 _owned_task 把软删除视为不存在；+predicted_only 参数供对比页跳过历史段 CSV 读取 |

| 后端改动 | code/backend/schemas.py | +HistoryPage / CompareSeries / WorkbenchSummary；🟤 TaskBrief 补失败三字段；HistoryQuery.status 刻意**不收窄**取值域 |

| 后端改动 | code/db/schema.sql | 🟤 v_task_summary 外露 d.id AS dataset_id（实际库已 DROP/CREATE VIEW 迁移，备份 data/app.db.bak-ep7） |

| 前端新增 | web/src/pages/HistoryPage.tsx（640 行） | UX-P09：汇总条 5 项 / 筛选 / 10 列表格 / 3 行骨架 / 空态双出口 / 删除二次确认 |

| 前端新增 | web/src/pages/HistoryDetailPage.tsx（432 行） | UX-P10：参数快照（冻结徽标）+ 复用 components/result/* + 失败块 + 归因 + 原参数重试 + 4 段阶段耗时 |

| 前端新增 | web/src/pages/ComparePage.tsx（530 行） | UX-P11：2–4 任务并排 / 逐行独立求最优 / 数据集警告 / 归因 / 曲线只画预测段 |

| 前端新增 | web/src/pages/DashboardPage.tsx（339 行，从 14 行占位重写） | UX-P04：近况问候 + 唯一主按钮 + 4 张汇总卡 + 快速开始 + 最近任务 |

| 前端改动 | router/index.tsx / layouts/AppShell.tsx / api/client.ts / types/index.ts | +3 条路由（**全部套 RequireAuth**）/ +2 导航项 / historyApi.dashboard() / 类型补齐 |

| 脚本新增 | scripts/verify_ep7.py / verify_ep7_api.py / verify_ep7_browser.mjs | 静态 89 / 接口 70 / 浏览器 57 |

验收结果

| 套件 | 结果 | 覆盖 |

|---|---|---|

| verify_ep7.py | **89/89** ✅ | 12 节结构守卫；🔒 BE-39 路由顺序实测断言 / 软删除幂等不走 _owned_task 断言 / 前端零硬编码实测数值（rgba() 摘除） |

| verify_ep7_api.py | **70/70** ✅ | 10 节；列表筛选分页 / 详情成功与失败双分支 / 对比 2–4 与越权 / 软删除保留数据与幂等 / 工作台同源 / 🟤 整个运行在**同一 lifespan** 内 |

| verify_ep7_browser.mjs | **57/57** ✅ | Edge 真实渲染，consoleErrors = 0；4 个页面探针 + 删除四态 + 对比最优值高亮 + 工作台真实渲染 |

| 前端三件套 | ✅ | tsc --noEmit 零错、eslint 零警、vite build 成功 |

关键实测数据

| 项 | 实测 | 预算 |

|---|---|---|

| API-17 历史列表首屏 | **41 ms** | ≤ 1 s |

| API-18 跨任务对比（4 任务） | **133 ms** | ≤ 3 s |

| 对比最优值方向 | #37=14.62% / #38=14.26%*（取 min） | — |

| 工作台与列表一致性 | hist=[3,2,1] / dash=[3,2,1] | 完全相同 |

7.8.2 EP-7 修复的 5 个真实缺陷

① 🔴 soft_delete 用 {Task.is_deleted: 1} 作字典键（潜伏缺陷）

task_dao.soft_delete 写的是 where(Task.id == task_id).values({Task.is_deleted: 1})，而字典键必须是**字符串列名**；SQLAlchemy 2.0 会抛 InstrumentedAttribute is not supported。这是**潜伏缺陷** —— 函数从 EP-1 起就存在但从未被调用过，API-20 是第一次触发。改为 {"is_deleted": 1}。

📌 **教训**：DAO 层写完不等于被调用过。删除类函数天然是"最后才跑一次"，所以最容易藏着笔误。

② 🔴 v_task_summary 视图缺 dataset_id，原参数重试要多查一次

视图原先只外露 dataset_name（展示用字符串）。但 API-19-1 的「原参数重试」需要 dataset_id + horizon，前端想跳重试就**得再查一次详情**才能拿到 id —— 多一次往返，且详情可能已不可用。dataset_id 是外键不是业务数据，外露无隔离风险。三处同步修（schema.sql / 实际库 DROP VIEW+CREATE VIEW / 数据库设计文档新增设计要点）。

③ 🔴 软删除后详情仍可访问（列表与详情口径不一致）

result_service._owned_task 用 get_by_id 查，**不过滤 is_deleted** —— 于是任务从列表消失了，但直接输 /history/{id} 还能打开（predictions 还在）。用户视角就是"删了但没删掉"，且列表说没有、详情说有，**同一任务两处口径矛盾**。修法：_owned_task 一律把软删除视为不存在（返 40401）；soft_delete 的幂等实现改为**不走 _owned_task**，先 get_by_id 校验归属、再判 is_deleted == 1 直接返成功。

📌 **推论**：凡"列表里过滤 + 详情里不过滤"的字段，两个地方必须同步。已固化进 verify_ep7.py。

④ ⚫ **失败任务历史详情整页 500**

失败任务可能连 metrics 行都没有（分级阶段就挂了）。此时 timing_ms 返回 {}，pydantic 因 5 个必填字段缺失抛 ValidationError → **历史详情页整页 500**。改为返回满零值（total_ms 取 task.duration_ms）。

⑤ ⚫ **TaskBrief 静默丢弃失败三字段**

服务层返回了 error_code / error_msg / completed_stage，但 code/backend/schemas.py 的 TaskBrief 没声明这三个字段 —— **pydantic 会把未声明字段直接丢掉**，响应里根本没有，前端拿到 undefined，看起来像"后端没存失败原因"。补声明三字段（成功任务为 null）。

📌 **通用教训**：pydantic 不会报错、只会**静默丢字段**。服务层加了返回字段却"前端拿不到"，第一反应就查 schema。

7.8.3 EP-7 校验脚本自身的 6 处判据错误（**代码是对的，判据写错了**）

① **case_failed_detail 用 select max(id) from tasks 选任务**，恰好选中 case_api20 刚软删的那个（id=4），_owned_task 正确返 40401。**这不是缺陷** —— 恰恰证明缺陷③的修复生效了。改为 where is_deleted = 0 and user_id != 0。

② **依赖注入写成 user=CurrentUser（默认值形式）**，传进去的是**类本身**而非实例，user.id 拿到 InstrumentedAttribute。必须用注解形式 user: CurrentUser。四处已改。

③ **CompareSeries schema 缺失**：起初想复用 SeriesCurve，但对比页不需要 history_dates/baseline/actual，硬塞会 500，且会让 4 个任务各读一遍 CSV 爆 3 s 预算 → 300 个校验错误。专用 CompareSeries。

④ **ComparePage 耗时列恒 null**：(s) => (s.metrics ? null : null) ?? null 写死返 null。根因是 CompareTaskSlot 缺 duration_ms。后端补字段（成功/失败两分支都补）。

⑤ **删除用例把对比要用的任务删掉了**：改为独立任务 + 删除后重新加载列表。

⑥ **跨行三元被注释剥离误删**：`//` 注释剥离器把跨行的 `? :` 表达式切掉，导致 `re.search` 判据本身失效。改用可跨空白匹配的写法。同轮还修了 `rgba(...,0.45)` 被误判硬编码（加白名单 + 摘除）。

7.8.4 EP-7 登记的口径偏离（**6 项**）

编号	偏离	理由	守卫
HIS-01	历史列表走 `v_task_summary` **视图**而非 `tasks` 表 + JOIN	视图已内置 `WHERE t.is_deleted = 0`，DAO 层不再重复写过滤条件，两个入口（列表 / 工作台）**口径同源**	`verify_ep7.py` 断言 DAO 层用视图名；视图定义同步登记进数据库设计文档
HIS-02	视图用两个 `LEFT JOIN` 而非 `INNER JOIN`	`INNER` 会让 `running` / `created` 状态的任务（`completed_at` 为 `NULL`）从列表**整条消失** —— 正在跑的任务恰恰最需要被看到	`verify_ep7_api.py` 断言 `running` 任务出现在列表中
HIS-03	对比任务归属采用「**全或全无**」	任一 `task_id` 不属于当前用户即抛 `40301`，**不部分返回**。部分返回会让用户以为"我只有 2 个任务可比"，实际是越权探测	`verify_ep7_api.py` 混合他人任务断言 `40301` 且**响应体不含任何指标**
HIS-04	对比曲线**只画预测值**，跳过历史段 CSV 读取	历史段各任务完全相同（同一数据集），画出来是重叠的同一条线；且 4 任务各读一遍 CSV 会吃掉 3 s 预算。用 `_series_curves(predicted_only=True)`	`verify_ep7_api.py` 断言 4 任务对比 ≤ 3 s（实测 133 ms）
HIS-05	曲线只对比**时间重叠区间**（`overlap_dates` 取交集）	不同 `horizon` 的任务时间轴不同段，强行对齐会产生"某天没有预测值"的空洞。交集为空时只画指标表不画曲线	`verify_ep7_api.py` 步长不一致用例：`overlap_dates` 非空 + `warnings` 含步长提示
HIS-06	`HistoryQuery.status` 在 schema 层**不收窄**取值域	收窄会抛 pydantic `ValidationError` → 500；非法状态应按本项目错误契约返 `40012`，这属于**业务参数校验**，由服务层白名单把关	`verify_ep7_api.py` 非法状态断言 `40012` 而非 500

> 📌 **EP-7 的两条设计取向（有意差异，不是遗漏）**
>
> **① 失败任务在「结果页」与「历史详情页」表现相反，是刻意的。**
> EP-6 定的规则是：失败任务进结果页 → 返 `40401`，**不给半成品**。EP-7 反过来：历史详情页**必须**展示 `error_code` / `error_msg` / `completed_stage` + 归因 + 原参数重试入口。

> 理由：这两个页面对应**两个不同的用户诉求**。「预测是否成功」由结果页回答（用户刚发起，最关心这个）；「上次为什么失败、怎么改参数重试」由历史详情页回答（用户回看，需要可行动信息）。若历史页也返 404，用户会发现失败任务**彻底无法查看原因**，只能重跑一次碰运气。
>
> ② `UX-P11` **最优值高亮必须逐行独立判断方向。**
> `MAPE` / `MAE` / `RMSE` / 耗时是**误差与耗时类，越小越好取** `min`；提升率与达标占比是**越大越好取** `max`。写死一种方向会让一半的行高亮成最差值 —— 看起来"这任务最好"，实际最差。已实现 `lowerIsBetter ? Math.min(...) : Math.max(...)` 逐行求最优，且跳过 `null`（无法评估不参与比较）。

`EP-7` **的 5 个真实缺陷与 6 处判据错误**（详见 §7.8.2 / §7.8.3）：最值得记的是**缺陷③与①的组合** —— `soft_delete` 里藏着一个从未被调用过的笔误（①），而"列表过滤、详情不过滤"（③）又让它有机会被提前发现。**潜伏缺陷与口径不一致往往成对出现**：一个没被调用的函数，恰好是另一个口径漏洞的检测点。另一条是缺陷⑤：**pydantic 静默丢字段**不报错、不告警，服务层加了字段前端却拿不到，这类"看起来像后端没存"的现象，第一反应要查 schema 声明。

7.9 EP-8 管理端（CH-01）

编号	工作项	要点
WS-8-01	`get_current_admin`	在 `get_current_user` 之上强制 `role == 'admin'`；非管理员 → `403`
WS-8-02	管理员种子账号可登录	🔒 邮箱域名必须被 `EmailStr` 接受（§5.3.3）；密码符合强度与 72 字节约束
WS-8-03	`admin_service`	概览聚合 / 用户管理 / 任务管理 / 模型版本 / 日志查询
WS-8-04	`API-22` ~ `API-30`	9 个接口；逐一挂 `get_current_admin`
WS-8-05	🔒 全操作审计	4 类写操作均写 `operation_logs`（`action='admin'`，`detail` 记子动作 + 操作者 + 目标）
WS-8-06	模型版本切换	🔒 同一事务内**先降后升**（`BE-ADM-02`）；切换后校验 `ACTIVE_MODEL`（`BE-ADM-03`）

编号	工作项	要点
WS-8-07	用户停用	写 users.status=0；其登录立即被拒
WS-8-08	重置密码	🔒 同时 token_version + 1，使该用户全部旧 Token 失效
WS-8-09	强制删除任务	级联 predictions / metrics；datasets 不动（RESTRICT）；审计保留
WS-8-10	AD-P01 运营看板	模板 A；复用 MetricCard / LineCompare，不新增令牌
WS-8-11	AD-P02 用户与任务管理	3 个 Tab；破坏性操作二次确认，文案明确「不可恢复」
WS-8-12	导航入口	顶栏用户菜单增「管理后台」项，**仅 isAdmin 可见**（QD-01）
WS-8-13	🔒 **上游文档回写**	按 §5.7 的 18 项清单逐项回写

DOD-EP8

☑ 🔒 普通用户（role='user'）访问 /admin 与全部 /admin/* 接口 → 403

☑ 🔒 普通用户界面**看不到**「管理后台」入口（不是隐藏按钮，是渲染层不输出）

☑ 管理员种子账号可正常登录并进入 /admin

☑ 切换模型版本成功；切换后 SELECT version FROM model_registry WHERE is_active=1 有且仅 1 行

☑ 停用某用户后，该用户登录被拒；恢复后可登录

☑ 重置密码后，该用户**旧 Token 立即失效**（用旧 Token 调 API-04 返回 401）

☑ 强制删除任务后，predictions / metrics 相关行数为 0；datasets 未被删除

☑ 🔒 4 类写操作各有 1 条 operation_logs 记录（action='admin'）

☑ 🔒 **红线自检**：grep -rn "SELECT * FROM users" app/api/v1/ 等无 user_id 过滤的查询**仅出现在 admin.py**

☑ 🔒 §5.7 的 18 项回写清单**全部完成**（逐项勾选，含 PRD / 数据库 / 技术文档 / UI/UX / 代码 / 脚本）

☑ 🔒 **浏览器真实渲染走查通过**（verify_ep8_browser.mjs **61/61**，consoleErrors=0）—— 超出原 DOD 要求的补充项

7.9.1 EP-8 执行记录（2026-10-10）

类别	内容
后端新增	app/services/admin_service.py（9 个接口的业务层）、app/api/v1/admin.py（9 条路由）
后端改动	app/services/auth_service.py（🟤 缺陷①）、app/dao/user_dao.py（+get_by_ids）、app/dao/metric_dao.py（+list_by_task_ids / count_by_task）、app/api/v1/router.py
schema	code/backend/schemas.py 新增 **23 个**管理端模型（🔒 明确不含 password_hash）
前端新增	web/src/pages/AdminDashboardPage.tsx（AD-P01）、web/src/pages/AdminManagePage.tsx（AD-P02）、web/src/components/RequireAdmin.tsx
前端改动	router/index.tsx、layouts/AppShell.tsx、components/NoticeBar.tsx（🟤 缺陷②）、components/result/LineCompare.tsx + DistributionChart.tsx（🟤 缺陷③）、api/client.ts、types/index.ts
验收结果	verify_ep8 **108/108** · verify_ep8_api **96/96** · verify_ep8_browser **61/61**（consoleErrors=0）· 前端 tsc 0 错 / eslint 0 告警 / 构建主包 **534 kB**
红线自检	无 user_id 过滤的跨用户查询**仅出现在 admin.py**（verify_ep8.py 第①节 AST 断言）

CH-01 的隔离例外边界（BE-ADM-01）：管理端是数据隔离红线 NFR-SEC-02 的**唯一显式例外**，且例外被压缩到最小 —— 例外代码只允许存在于 app/api/v1/admin.py 一个文件；9 条路由**全部**声明 CurrentAdmin（不是「大部分」）；未登录先认证 → 401，已登录但非管理员 → 40301，两种拒绝**不混用**（把「没登录」报成「没权限」会误导用户去要权限）。

审计格式（6 类子动作）：action='admin'，detail 固定为 sub=<子动作>;target=<目标>;by=<操作者>[;extra]，子动作取值封闭为 enable / disable / role_change / reset_pwd / hard_delete / switch_model。审计与业务**同事务**写入 —— 业务回滚则审计一起回滚，不会出现「操作失败但留下成功记录」。

> 📌 **EP-8 的三条设计取向（有意差异，不是遗漏）**
>

> ① **用户侧 API-20 是软删除，管理端 API-27 是硬删除。** 两者并存是有意的：用户删自己的任务要能反悔，所以只打 is_deleted；管理员删违规数据要彻底，所以级联清 predictions / metrics。但 datasets **保留**（外键 RESTRICT，数据集可能被别的任务引用）、operation_logs **保留**（删了任务不该删掉「谁删的」这条证据）。前端弹窗因此**必须**同时写「不可恢复」与「与您自己删除不同」，否则用户会以为管理端删了还能在历史里找回。
>
> ② **API-25 重置密码必须同时 token_version + 1。** 只改密码不 bump 版本号的话，用户在别的手机上仍能继续用旧 Token 操作 —— 管理员以为「已经踢出去了」，实际没有。API 验收里专门断言：重置后用旧 Token 调 API-04 返 401。
>
> ③ **主指标口径只算 L1。** 与 EP-5/EP-6 同源（算法层五铁律之一）：管理端看板展示的 MAPE 是 L1（走 LSTM）的结果，L2（规则兜底）实测 MAPE ≈ 60%，混进来会把 13.6% 抬到 25.9%，看板就会误导用户以为模型变差了。

7.9.2 EP-8 修复的 3 个真实缺陷

① ⚫ 停用用户仍能重新登录（安全缺陷，本阶段最严重的一个）

auth_service.login() 从 EP-2 起就**只校验密码，完全不看 user.status**。于是 API-24 把用户停用（status=0）后，他只要重新登录一次就能拿到新 Token —— **停用形同虚设**，且这条路径完全绕开了管理端的全部审计。

修法：在密码校验成功之后、签发 Token 之前补状态检查，失败时写一条 result='fail' 的登录日志并返 BAD_CREDENTIAL（🟤 **不返「账号已停用」** —— 那等于一个可枚举的账号状态探测接口）。
📌 **教训**：**"封禁"类需求必须同时堵三条路**：已有 Token（靠 token_version）、重新登录（靠 status 检查）、API 直调（靠 get_current_admin）。EP-8 三条都补齐了，但**只有第 2 条是这次才发现的** —— 前两条在 EP-2 就写了，第 3 条在 EP-8 才写。封禁功能天然是"三处协同"，漏一处不会报错、只会漏防。

② ⚫ NoticeBar 静默丢弃 data-testid

NoticeBar 的 Props 是封闭对象类型，**不透传任意 DOM 属性**。AdminManagePage 在删除成功回执上写了 data-testid="ok-msg"，TypeScript 因为索引签名缺失而没有报错（调用方多传属性在有默认导出的组件上容易漏检），运行时 React 直接把这个属性**丢掉** —— 页面显示完全正常，但验收脚本永远断言不到，且**不产生任何错误信息**。

修法：`Props` 加 `[key: data-${string}]: unknown` 并 `...rest` 透传。

📌 **教训**：**"页面正常但断言断言不到"是比"页面报错"更难查的一类失败**，因为没有任何错误线索。本次靠"删除后弹窗关闭 ✅ 但回执 ❌"这个不对称组合才定位到 —— 若当时把两项一起断言失败，就会误判成"删除功能整体没做"。

③ ⚫ **ECharts 主包体积回归（EP-3 成果被推翻）**

EP-3 曾把主包从 1381 kB 降到 343 kB（ECharts 拆成独立分包 + 懒加载）。EP-8 首次构建时主包又涨到约 **1.57 MB**。根因不是 EP-8 自己引入了 ECharts，而是 **EP-6 的 `LineCompare` / `DistributionChart` 在模块顶层静态 `import EChart`**，把整条静态引用链拉回主包。

修法：两处改 `lazy(() => import('../EChart'))` + `Suspense`（fallback 用既有 `Skeleton`）。修复后主包 **534 kB**，ECharts 独立为约 **1015 kB** 分包。

📌 **教训**：**懒加载的收益会被"后来者在别处静态 import 同一模块"悄悄抹掉**。这类回归不会报任何错，只是构建产物变大，必须靠"每阶段都记基线数字 + 每阶段都比"这套纪律才能发现。同理，剩余 534 kB 主要来自 antd / react-dom，是 EP-4 之后的合理增长，**不是 echarts 回归** —— 别看到"超过 EP-3 的 343 kB"就再去拆包。

7.9.3 EP-8 校验脚本自身的 8 处判据错误（**代码是对的，判据写错了**）

#	判据错在哪	后果
1	AST 检测 `@router.get` 时判 `d.func.id == "router"`，但 `d.func` 是 `ast.Attribute`	9 条路由一条都匹配不到，等于没检
2	路由动词从 `methods=` 参数取	FastAPI 装饰器不传 `methods`，取到的永远是空
3	用 `strip_comments` 后的 `router.py` 查「唯一显式例外」注释文字	注释被剥掉，判据必挂
4	令牌判据错数 `styles/index.css`	令牌实际定义在 `src/theme/tokens.css`，报"零令牌"
5	令牌口径把「变量名数量」当成「oklch 出现次数」	两个数不等，误判
6	误判 `HeroChart` 顶层导入为违规	它被 `HomePage` 以 lazy 引用，合规
7	`_audit` 的 commit 判据检查函数体内 `db.commit()`	**要求它不自行 commit**，判据写反了
8	🟤 浏览器脚本第 558 行`/管理端操作/.test(document ? true : true)`	**在 Node 侧引用 `document`** → 抛 `document is not defined` 直接 `FATAL` 中断，后面的断言全没跑

前 7 处是静态判据，第 8 处是运行时错误。**判 8 暴露了一个更值得记的问题**：浏览器脚本用了 `xxx || true` 这种"恒真表达式"来"兜底"，结果把一个语法/上下文错误变成了静默通过 —— 断言看起来跑了，实际什么都没验。已改为有实际判据的 `lg.subBadges >= 1 && lg.hasAdminAction && lg.logRows <= rowsBeforeFilter`（筛选后行数必须被收窄，只验"徽标还在"不够，因为筛选没生效时徽标照样在）。

7.9.4 EP-8 排查最耗时的一个根因：代理端口不匹配

浏览器走查初次运行只有 **27/60**，且失败极具迷惑性：普通用户"登录失败"、管理员登录成功但页面**数据全空**、所有表格 0 行、弹窗不出现。

真实原因有**两层**，都不在业务代码里：

1. **旧进程占端口**：5199 / 8021 上还挂着 EP-8 之前启动的进程，本次启动的新后端因 `Errno 10048` 直接退出。运行中的旧后端 `openapi.json` 里**一条 `/admin/*` 路由都没有**（实测只有 21 条），前端自然什么都拿不到。
2. **vite 代理目标不匹配**：`vite.config.ts` 的 `VITE_API_TARGET` 缺省是 **8000**（技术文档 §2.7 的约定端口），而测试后端跑在 **8021**。前端日志里是 `http proxy error: /api/v1/auth/login Error: connect ECONNREFUSED 127.0.0.1:8000` —— **后端日志里一条请求都没有**。

第 2 点是真正的元凶，且它**伪装成业务 bug**：页面能渲染、无空白、控制台也不报错（`consoleErrors=0`），只是数据永远为空。

📌 **教训（已固化进 `vite.config.ts` 文件头）**：启动 dev server **必须**带 `VITE_API_TARGET=http://127.0.0.1:<后端端口>`。判断代理是否通，看**后端**日志有没有请求最直接 —— 后端零请求 = 请求根本没到后端 = 代理配错，不要再去查前端代码。

另外还发现一处**脚本自身的脆弱点**：注册表单底部有《用户协议》《数据用途说明》两个必选复选框，脚本从未勾选，被前端校验拦下并停在 `/register`，表现为"登录失败"。已在脚本里显式勾选并新增一条断言（注册页用户协议已勾选）。同时把普通用户邮箱改为**时间戳**（`ep8b_${Date.now().toString(36)}@verify.com`）—— 邮箱全局唯一，固定邮箱导致脚本**第二次跑就必然失败**，验收脚本必须可重复执行。

7.9.5 EP-8 登记的口径偏离（**5 项**）

| 编号 | 原计划 | 实际做法 | 理由 |

|---|---|---|---|

| 1 | 管理端入口放侧栏分组 | 放**顶栏用户区**，紧邻「管理员」徽标（`QD-01`） | 两个入口都属管理，单独开侧栏分组会为 2 项改动全局导航结构，收益不匹配 |

| 2 | 新建管理端专用图表组件 | 复用既有 `EChart` + `MetricCard` + `NoticeBar` + `Skeleton` | `QD-01` 明确「不新增设计令牌」，同理不新增组件 |

| 3 | 模型切换后仅刷新页面 | 额外返回 `env_match` 与 `warnings`，页面显示漂移徽标 | `BE-ADM-03` 要求「不悄悄失败」，漂移必须**可见** |

| 4 | 硬删除用 `ON DELETE CASCADE` | 显式 `delete_by_task` 后再 `hard_delete` | 级联删* 会连带 `datasets`；显式删除才能精确控制「删什么、不删什么」，并回执行数 |

| 5 | 查询 schema 收窄 `status`/`role`/`action` 取值域 | **刻意不收窄**，非法值由服务层返 `40012` | pydantic 收窄会抛 `ValidationError` → 500；收窄放在服务层才能返正确的业务错误码 |

📌 **`EP-8` 最值得记的一条**：`NoticeBar` 丢弃 `data-testid`（缺陷②）与"注册协议未勾选"（脚本脆弱点）有一个共同点 —— **失败时不产生任何错误。**前者页面完全正常，后者页面只是"数据没出来"。这类缺陷只能靠**"每个可交互元素都要有独立断言锚点 + 脚本失败时必须带出上下文报错文案"**两条纪律根治，本阶段已分别落实为组件层透传 `data-*` 与脚本里注册页报错：`${regErr}`。

7.10 EP-9 集成联调与测试

编号	工作项	要点
`WS-9-01`	主干路径端到端	无人工介入走通全链路
`WS-9-02`	分支场景	`B1`~`B14` 逐条验证（开发方案 §2.2）
`WS-9-03`	边界与异常	空文件 / 单行数据 / 超长序列 / 特殊字符商品名 / 时区无关性（技术文档 §9.4）
`WS-9-04`	异常操作	二次上传 / 重复提交 / 中途刷新页面 → **不产生脏数据**
`WS-9-05`	性能测试	按技术文档 §9.7 全部 7 项
`WS-9-06`	兼容性	Chrome / Firefox / Safari / Edge 最新两版；1366×768

编号	工作项	要点
WS-9-07	无障碍审计	python scripts/a11y_audit.py 退出码 0（UX-AC-01）
WS-9-08	契约一致性	前后端字段名 / 类型 / 枚举值完全对齐，无隐式依赖
WS-9-09	校验链全绿	4 套脚本（见 §9.1）

DOD-EP9（✅ 2026-10-12 主体达成；第 4 条**部分达成**，Safari 未实测，已转 P1-01 遗留问题）

☑ 🔒 端到端 ≤ 60 s（示例数据）；极限 1000 序列 × horizon=30 通过或正确软失败

☑ 🔒 并发 3 个任务提交：全部成功、无锁冲突、各自 ≤ 60 s（NFR-PERF-09）

☑ 🔒 a11y_audit.py 退出码 0

☐ 四个浏览器 + 1366×768 无布局错乱 —— ⚠ **部分达成**：Edge / Chrome / Firefox 各 109/109 通过；Safari 因本机为 Windows 无法实测，登记为 P1-01

☑ 浏览器控制台**无错误**（警告可有，错误不可有）

☑ §9.1 的 4 套校验脚本全部通过

☑ 测试报告含：通过用例 / 失败用例 / 修复记录 / **遗留问题清单**

7.11 EP-10 文档与答辩交付

编号	工作项	要点
WS-10-01	测试报告	功能 + 边界 + 性能 + 遗留问题
WS-10-02	部署运维手册	环境准备 / 启动 / Nginx 配置 / 备份恢复 / 安全加固清单
WS-10-03	API 接口文档	导出 OpenAPI；🔒 Swagger 资源离线化（国内网络必需，技术文档 §10.4）
WS-10-04	演示 PPT	项目概况 / 技术方案 / 实现过程 / 目标达成度 / 难点与应对；🔒 **含性能实测数据**（附录 D）

编号	工作项	要点
WS-10-05	演示脚本	5 分钟内跑完「上传 → 预测 → 看结果 → 导出」
WS-10-06	演示数据冻结	固定数据集，现场可复现
WS-10-07	失败分支演示	现场展示 1 个失败分支（如上传缺列文件），证明异常处理真实可用
WS-10-08	文档收敛	6 份文档版本号统一、交叉引用无误、verify_doc_refs.py 通过；受管范围扩到**全部 9 份交付文档**；docs/*.md 与 outputs/ 导出件的齐备性 / 新鲜度 / TOC 域由 verify_doc_exports.py 守卫

DOD-EP10（✅ 2026-10-13 全部达成，验收证据见 §6.4 EP-10 执行记录）

☑ 任务书 §5.2 要求的文档**全部产出**：PRD / UI/UX / 数据库设计 / API 接口 / 测试报告 / 部署运维手册 —— **8 份文档齐备**（见 §12）

☑ AI 协作教程、RAG 语料库与提示词、项目说明按模板输出 —— 注：任务书提到的 skill.md 模板**未随任务书提供**，改以项目自身知识资产为源并已在 Q11 登记

☑ 🔒 演示 PPT 含性能实测数据（附录 D 基准）—— 18 页，P13 硬指标 / P14 性能实测（端到端 11.85 s vs SLA 60 s，余量约 5 倍；含 3 并发与极限软失败）

☑ 🔒 现场演示 **5 分钟内**跑完主链路；另有 1 个失败分支可演示 —— verify_demo_data.py 实测主链路自动化 **1.0 s**；另备「缺仓库列」与「负销量」两个失败分支

☑ verify_doc_refs.py 通过（0 悬空编号；受管文档 **9 份** = PRD + 数据库设计 + 7 份交付文档）

☑ verify_doc_exports.py 通过（**13/13** 组 docx·pdf 齐备、未过期、TOC 域已展开）

☑ 交付物清单（§12）**100% 齐备**

8. 里程碑与 Go/No-Go 判定

里程碑	判定时点	判定条件	不通过时的动作
MS-1 基建就绪	EP-0 结束	/health 可访问 + 前端空布局可渲染 + 版本锁定核对通过	修基建，不进 EP-1（基建问题会放大到每个后续阶段）
MS-2 认证闭环	EP-2 结束	注册登录改密登出闭环 + 限流可触发 + 无明文密码	补齐认证，不进 EP-3（认证是所有业务的前置）
MS-3 上传可用	EP-4 结束	5 类错误可复现 + 游客态可跑	补齐校验，不进 EP-5
MS-4 ★ **端到端跑通**	EP-6 结束	主干路径跑通 + 指标与算法产出**逐项一致** + 端到端 ≤ 60 s	**冻结新功能，只修缺陷**（M-G3 精神）；这是全项目最关键的 Go/No-Go
MS-5 业务闭环	EP-7 结束	历史 / 详情 / 对比 / 导出全部可用	✅ **已达成（2026-10-10）**：API-17 41 ms / API-18 133 ms / 软删除保数据幂等 / 对比最优值高亮 / 工作台与列表同源
MS-6 管理端可用	EP-8 结束	非管理员 403 + 全操作审计 + **18 项文档回写完成**	✅ **已达成（2026-10-10）**：9 路由全挂 CurrentAdmin（401 / 40301 分明）/ 6 类子动作审计与业务同事务 / 静态 **108** + 接口 **96** + 浏览器 **61** 全绿 / §5.7 的 18 项逐项回写完成
MS-7 质量达标	EP-9 结束	4 套校验脚本全绿 + 性能与兼容性通过	优先修 P0 缺陷；未通过项列入遗留问题清单
MS-8 交付就绪	EP-10 结束	交付物清单 100% 齐备 + 5 分钟演示可跑	优先补硬性交付物

MS-7 / MS-8 达成情况

· MS-7 ✅ **已达成（2026-10-12）**：verify_ep9_api 125/0、verify_ep9_perf 47/47、verify_ep9_browser 109/109、verify_ep9_contract 92/92、a11y_audit 退出码 0；37 套脚本全量回归全绿。唯一未达成项为 Safari（无该内核可测），列入 P1-01。

· MS-8 ✅ **已达成（2026-10-13）**：交付物清单（§12）100% 齐备；演示主链路自动化实测 **1.0 s**（硬线 5 分钟），另备 2 个失败分支可现场演示。

风险最高的两个里程碑：

里程碑	风险	已具备的缓解
MS-4	端到端超时 / 指标不一致	⏱ 实测端到端 7.9 s（余量 7.6 倍）、推理仅 21 ms、指标已达标（14.26%）；算法层 69/69 自检通过
MS-6	管理端破坏隔离红线	例外仅限 `admin.py`（BE-ADM-01）+ 强制 `get_current_admin` + 全操作审计 + 红灯自检 grep

---

9. 质量保障与验收

9.1 校验脚本矩阵（🔒 全部为「可复现断言」，非人工目视）

脚本	作用	触发时机	当前状态
`scripts/verify_schema.py`	DDL 可执行性、对象齐备、幂等、约束有效性、游客态、**CH-01 回归（role/action）**	数据库变更后	✅ 通过（EP-1 已补 `role` / `action` / 管理员种子断言）
`scripts/verify_orm.py`	**ORM 实体 ↔ `schema.sql` 一致性（表/列/类型/可空/索引）+ 红线自检（无 `DateTime`、无 `import algo`）**	**EP-1 起，models/dao 变更后**	✅ **新增（EP-1）**

脚本	作用	触发时机	当前状态
scripts/migrate_0001_ch01_role.py	CH-01 迁移（加 role 列 + action 扩 8 类 + 补管理员种子），幂等 + 自动备份	库结构升级时	✅ 新增（EP-1）
scripts/verify_schemas.py	Pydantic 模型行为 + 密码字节边界 + CH-01 角色模型（UserProfile.role）	接口契约变更后	✅ 57/57（EP-2 +3 条；admin 端模型待 EP-8）
scripts/verify_auth.py	API-01~05 端到端闭环 + 限流 + 边界（72/73 字节、大小写邮箱、坏 Token）+ 防枚举 + get_current_admin 权限 + 审计 + 无明文密码	认证链路变更后 / EP-2 起	✅ 新增（EP-2）39/39
scripts/sync_eval_metrics.py	硬指标「构建期注入」：outputs/eval_report.json → web/src/generated/eval_report.json（--check 只比对不写，供 CI）	EP-3 起；🔒 重训模型后必跑	✅ 新增（EP-3）--check 一致
scripts/verify_ep3.py	首页硬指标同源 + 原型痕迹（已剥注释）+ 页脚单点/不可折叠/4 模板挂载 + 四区块 + 全局组件 + 🔒 echarts 引入点守卫	EP-3 起；首页/页脚/指标链路变更后	✅ 新增（EP-3）29/29
scripts/verify_ep3_browser.mjs	Edge 无头 + CDP：4 模板走查（免责声明存在/可见/未折叠/文案完整）+ 硬指标逐项比对 + 主题切换图表变色 + 每页 consoleErrors=0	EP-3 起；布局/主题/全局组件变更后	✅ 新增（EP-3）47/47
scripts/verify_ep4.py	数据集上传的静态守卫：7 步顺序常量与 algo.config 同源 / 错误码齐备 / 路由注册顺序 / 游客依赖范围 / UUID 落盘名 / exports/tmp 分目录 / 前端 7 组件装配 / 🔒 预览表动态列 / 原型痕迹零命中	EP-4 起；上传接口、校验顺序、落盘路径变更后	✅ 新增（EP-4）77/77
scripts/verify_ep4_api.py	上传链路端到端：五类错误 + 三类上限 + 断档两分支 + 游客链路 + 明细下载与 TTL + API-07 幂等 + API-09 与 eval_report 同源 + BE-41 两分支 + 架构红线自检（AST 取 import）	EP-4 起；上传 / 校验 / 错误明细相关变更后	✅ 新增（EP-4）119/119
scripts/verify_ep4_browser.mjs	Edge 无头 + CDP：UX-P05 的 18 个真实状态逐个走查（真选文件真上传）+ 4 指标与源文件一致 + 预览列零泄漏 + 断档不自动继续 + 明细按钮显隐 + 模板 B 结构回归 + 每状态 consoleErrors=0	EP-4 起；上传页、错误卡、预览表变更后	✅ 新增（EP-4）173/173
scripts/cleanup_tmp.py	§3.8 数据清理运维入口（JT-10 / JT-11 / DB-12）；支持 --with-guest / --dry-run	运维定期执行；磁盘告警时	✅ 新增（EP-4）

脚本	作用	触发时机	当前状态
			三档 TTL 实测
scripts/verify_ep5.py	预测编排的静态守卫：🔒 AST 扫 services/ 零命中训练入口 / 游客链路两条路由无 RequireAuth / 五阶段常量前后端逐项相等 / 预算占比和为 1.0 且 inference 最大 / 软失败零 5xx / 序列超限预检早于建任务行 / 无「重新训练」入口（带否定标记的合规文案不算违规）/ 前端零实测数值字面量 / 右栏 useMemo 稳定引用 / 成本模型预估 vs 实测偏差 ≤ 25% / 🔒 _roll_multistep 函数体内零 build_inference_windows 调用	EP-5 起；predict_service / 阶段常量 / 参数页变更后	✅ 新增（EP-5）50/50
scripts/verify_ep5_api.py	端到端：🔒 h=14 端到端 ≤ 60 s（且余量 > 40%）/ 五阶段与 eval_report 同量级 + 批量前向毫秒级 / TASK_BUDGET_MS=1 → E_TIMEOUT 软失败（HTTP 200 + 3 建议 + BE-16 落库 + 半成品不落盘）/ MAX_SERIES=10 → 40004 且 tasks 行数不变 / 🔒 CPU↔GPU 两独立进程逐点比对 1,582 点 / 参数契约（HTTP 200 + code 口径）/ 游客 user_id=0 与越权 40401	EP-5 起；推理链路、预算、错误契约变更后	✅ 新增（EP-5）60/60
scripts/verify_ep5_browser.mjs	Edge 无头 + CDP：UX-P06 四区块（粒度只读 / 步长三选一 / 节假日开关 / <details> 高级设置含「不提供在线重训练」警示 / 4 张耗时预估卡随接口联动）+ 提交失败四件套 + UX-P07 五阶段进度与时间轴（5 段宽度互不相同且 inference 最宽）+ 完成态落点 /result/:taskId + 模板 B 结构回归 + 全程 consoleErrors=0	EP-5 起；参数页、执行页变更后	✅ 新增（EP-5）41/41
scripts/verify_ep5_equiv.py	⚫ 滚动多步改造前后逐位等价性：把改造前的 _roll_multistep 逐字保留为 reference_roll，对全部 L1 序列 × 3 档步长断言 np.array_equal	_roll_multistep / features 热路径每次改动后	✅ 新增（EP-5）3/3
scripts/sync_pipeline_meta.py	流水线元信息构建期注入：predict_service.STAGES / STAGE_LABELS / STAGE_BUDGET_RATIO + algo.config + features.HOLIDAYS → web/src/generated/pipeline.ts（--check 只比对不写）	EP-5 起；阶段常量 / 节假日表变更后；🔒 生成物勿手改	✅ 新增（EP-5）--check 一致
scripts/verify_ep6.py	结果页静态守卫：🔒 指标纪律组	EP-6 起；结果接	✅ 新增

脚本	作用	触发时机	当前状态
	（result_service 顶层不 import algo —— 函数内为列名常量 import 是允许的 / _metric_bundle 不做 round —— 需先 strip_docstrings 否则会命中解释性 docstring / 覆盖率取 tasks 冗余列而非 count_by_level）/ 结果组件内零二次聚合（禁 .reduce）/ 🔒 阈值集中为 THRESHOLDS 且与 models/metric.py 同源 / 明细走服务端分页 / 结果页不读本地算法产物 / /result/:taskId 无 RequireAuth	口 / 结果组件 / 阈值变更后	（EP-6）65/65
scripts/verify_ep6_api.py	结果链路端到端：🔒 四项指标与 outputs/eval_report.json 逐字段一致 / 🔒 覆盖率三项之和 = total / 🔒 SeriesCurve.baseline 与 MetricBundle 可空（稀疏数据 L3 场景）/ API-15 排序白名单（非法列名 40006）+ 分页上限 100 / API-16 CSV 的 BOM 与 7 列序 / 游客导出 401 / 越权 40401 / ⚫ 整个运行须在同一 lifespan 内（否则模型单例被清、要推理的用例全挂 40404）	EP-6 起；result_service / 导出 / 明细分页变更后	✅ 新增（EP-6）105/105
scripts/verify_ep6_browser.mjs	Edge 无头 + CDP：UX-P08 七区块逐项（结论条在首卡 / 4 指标卡 / 基线表含判定列 / 覆盖率面板 / 折线三线 / 直方图 12 箱 / 明细表）/ L3 过滤后全为 L2 / 游客导出按钮禁用 + 登录入口 / 空态两出口 / 🔒 超标态重放（必须拦 XHR 而非 fetch —— 前端走 axios）+ 警告不可折叠断言 / 全程 consoleErrors=0	EP-6 起；结果页变更后	✅ 新增（EP-6）42/42
scripts/verify_ep7.py	历史与对比静态守卫（12 节）：🔒 BE-39 路由顺序实测断言（history.router.routes 中 compare 序号 < {task_id}，防重构时顺序被改）/ 软删除幂等实现不走 _owned_task 断言 / _owned_task 必须判 is_deleted（列表与详情口径同源）/ 视图须含 dataset_id / TaskBrief 须声明失败三字段（🔒 pydantic 静默丢字段）/ 历史三路由全套 RequireAuth / CompareSeries 专用 schema 而非复用 SeriesCurve / UX-P11 逐行独立求最优（lowerIsBetter ? min : max，跳过 null）/ UX-P09 逐字输入 ID 且不匹配时 disabled / 工作台复用 list_history 不前端自聚合 / 前端零	EP-7 起；历史 / 对比 / 工作台变更后	✅ 新增（EP-7）89/89

脚本	作用	触发时机	当前状态
	硬编码实测数值（白名单 + rgba() 摘除）**		
scripts/verify_ep7_api.py	**历史链路端到端（10 节）：API-17 首屏 ≤ 1 s（实测 41 ms）/ 筛选与分页 / API-19 成功与失败双分支（error_code+error_msg+completed_stage）/ ⚫ 失败任务无 metrics 行时 timing_ms 返满零值（否则 500）/ API-18 2–4 个任务 + 越权 40301 且响应体不含任何指标 + 步长不一致进 warnings + overlap_dates 取交集 + 4 任务 ≤ 3 s（实测 133 ms）/ API-20 软删除后 predictions+metrics 行数完全不变 + 二次调用幂等 / 软删除后详情返 40401 / 工作台最近任务 == 历史列表首屏 / ⚫ 整个运行须在同一 lifespan 内**	**EP-7 起；history_service / task_dao 列表查询变更后**	**✅ 新增（EP-7）70/70**
scripts/verify_ep7_browser.mjs	**Edge 无头 + CDP（11 节，4 个页面探针）：UX-P09 汇总条 5 项 + 筛选空结果 + 3 行骨架 + 空态双出口 / 🔒 UX-P09-04 删除二次确认**四态（未输入 / 错 ID / 少一位均 disabled，正确才解禁）/ UX-P10 成功详情复用结果组件 + 参数快照冻结徽标 + 4 段阶段耗时 / UX-P10 失败详情块 + 归因 + 重试入口 / UX-P11 最多 4 任务（勾选 1/4 时禁用）+ 逐行最优值高亮 + 数据集不一致警告 / UX-P04 工作台真实渲染 + 最近任务表 / 全程 consoleErrors=0**	**EP-7 起；四个页面任一变更后**	**✅ 新增（EP-7）57/57**
scripts/verify_ep8.py	**管理端静态守卫（12 节 / 108 条）：🔒 BE-ADM-01 隔离边界（无 user_id 过滤的跨用户查询仅出现在 admin.py，用 AST 判定而非 grep）/ 9 条路由全部声明 CurrentAdmin /🔒 依赖注入必须是**注解形式 **admin: CurrentAdmin（写成 admin=CurrentAdmin 会把类本身当依赖传进去）/ 🔒 BE-ADM-02 模型切换事务（deactivate_all → activate_version 之间不得提前 commit）/ 🔒 BE-ADM-03 切换后 reload_active_model + env_match + warnings 不悄悄失败 / 审计 action='admin' 与 sub= 格式+ _audit** 不得自行 commit**（须与业务同事务）/ 🔒 9 个响应模型零 password_hash / 三条自我保护（不能停用自己 / 不能降级自己 / 不能动最后一个管**	**EP-8 起；管理端 / auth_service / NoticeBar / 图表组件变更后**	**✅ 新增（EP-8）108/108**

脚本	作用	触发时机	当前状态
	理员）/ ⚫ `login()` 必须检查 `user.status`（否则停用可被重新登录绕过）/ 前端零 hex + 零新设计令牌 / ECharts 懒加载不得回退为顶层静态导入 / §5.7 的 18 项文档回写逐项断言		
`scripts/verify_ep8_api.py`	管理端接口验收（13 节 / 96 条，临时库 + `TestClient` 全程同一 lifespan）：🔒 `API-22~30` 未登录 `401` / 非管理员 `40301`（两种拒绝不混用）/ 🔒 9 个响应体递归扫 `password` 字段 = 0 / `API-23` 分页 + 筛选 + 关键词 / 🔒 `API-24` 三条自我保护逐条/ ⚫ 停用后旧 Token 立即失效（`token_version+1`）/ `API-25` 重置密码同样联动 `token_version` / `API-26` 含已软删除任务并标 `is_deleted` / 🔒 `API-27` 硬删除级联 `predictions+metrics` 而 `datasets` 与 `operation_logs` 保留 + 回执行数 + 重复删返 `40012` / `API-28` `env_match` 标记 / 🔒 `API-29` 切换后唯一索引 `ux_model_active` 恰好 1 行 + `BE-ADM-03` 警告 / 🔒 4 类写操作各有 `action='admin'` 记录且 `detail` 可解析出 `sub/target/by` / `API-30` 非法 `action` 返 `40012`（不是 500）/ `API-22` 指标齐备 + `mape_trend` 时间正序 + `days` 越界返 `40012`	`EP-8` 起；`admin_service` / DAO 层查询变更后	✅ 新增（`EP-8`）96/96
`scripts/verify_ep8_browser.mjs`	Edge 无头 + CDP（10 节 / 61 条，4 个页面探针）：游客访问两页均落`/login`（不是 403）/ ☑ 注册协议勾选 + 三种身份切换 / 🔒 普通用户顶栏零管理端入口 + 侧栏零分组 + 无管理员徽标 + 🔒 手工改 URL 落到 403 明确页（含「联系系统管理员」）而非空白页 / `AD-P01` 5 张指标卡 + MAPE 走势 + 🔒 懒加载 ECharts 真的渲染出 SVG（查 `svg path` 元素数）+ 阈值 20% 可见 + 模型表 + 埋点概览 / `AD-P02` 三 Tab + 用户表零 `password_hash` 串 + 每行三操作 / 🔒 重置密码弹窗四态（初始禁用 / 弱密码禁用 / 提示缺什么 / 合规解禁）+ 文案明示「全部已登录会话会立刻失效」/ 🔒 强制删除二次确认四态（未输入 / 错 ID / 少一位均 `disabled`，逐字正确才解禁）+ 文案含「不可恢复」且明示与用户侧软删除不同 + 回执含连带删除行数 / 日志表子动作中文徽标 +	`EP-8` 起；管理端两页 / `NoticeBar` / 任一图表组件变更后	✅ 新增（`EP-8`）61/61

脚本	作用	触发时机	当前状态
	`detail` 原文可见 + 🔒 筛选 `admin` 后行数被收窄 / 全程 `consoleErrors=0`		
`scripts/verify_types.mjs`	前端类型语法与一致性	类型变更后	✅ 通过
`scripts/verify_tokens.py`	`tokens.css` 令牌守卫（`oklch` ≥56 / `#` =0）	主题变更后	✅ 通过（71 / 0）
`scripts/verify_doc_sql.py`	文档 §5 SQL 与 `schema.sql` 逐行一致 + 字典列数	表结构变更后	✅ 通过（**121 列**已同步）
`scripts/verify_doc_refs.py`	📌 **编号守卫**：文档引用的 `FR-*` / `NFR-*` 必须真实存在于 PRD；受管文档 **9 份**（含 `EXTRA_DOCS` 全量交付文档）；含**退役编号登记**（`RETIRED_CODES`，须带修正说明否则仍判失败）	文档变更后	✅ 0 悬空
`scripts/verify_doc_exports.py`	**导出件守卫（`WS-10-08`）：`docs/*.md` → `outputs/*.docx•*.pdf` 的齐备性 + 新鲜度（导出件 `mtime` 不得早于源 md）+ TOC 域已展开 + 页数留痕**	**文档导出后 / 提交前**	✅ **新增（`EP-10`）13/13**
`scripts/verify_docs_browser.mjs`	**接口文档离线化的真实浏览器断言（`WS-10-03`）：Edge 与 Chrome 各 16 项 —— `/docs` 的 `.opblock` 数量 > 0、`/redoc` 渲染出正文、DOM 内零外链、零 `loadingFailed`、零 4xx/5xx**	**`EP-10` 起；`main.py` 静态挂载 / vendor 资产变更后**	✅ **新增（`EP-10`）16/16**
`scripts/verify_demo_data.py`	**演示冻结端到端复现（`WS-10-05`/`10-06`/`10-07`）：冻结数据集幂等（逐字节可复现）/ 三级降级分级实测 / 2 个失败分支（缺仓库列、负销量）/ 主链路 50 项断言**	**`EP-10` 起；`demo/` 数据集或演示链路变更后**	✅ **新增（`EP-10`）50/50**
`scripts/export_openapi.py` / `build_api_doc_html.py`	导出 `openapi.json`（30 路径 / 32 操作 / 78 Schema）并打包为**单文件自包含**接口文档 HTML（零外部引用）	接口变更后	✅ **新增（`EP-10`）**
`scripts/make_demo_data.py` / `build_rag_corpus.py`	生成冻结演示数据集（含 2 个失败分支）与 RAG 语料库（**40 语料块 / 9 主题 / 5 组提示词**）	演示数据 / 知识资产变更后	✅ **新增（`EP-10`）**
`scripts/build_md_docx_formal.py` / `docx_export.sh` / `run_pdf.ps1`	**文档导出流水线**：md → 真 Heading + TOC 域 + 页码的 docx（python-docx）→ PDF（Word COM，`Fields.Update()` 展开目录域）。⚠ 中文路径搬运放 bash、Word 转换只发生在 ASCII 暂存目录（本机沙箱拦命令行	文档内容变更后	✅ **新增（`EP-10`）**

脚本	作用	触发时机	当前状态
	非 ASCII 字符）		
scripts/explain_queries.py	11 条典型查询的执行计划	索引变更后	✅ 10/11 命中索引
scripts/a11y_audit.py	无障碍审计	前端变更后	✅ **退出码 0**（53 项，PASS 51 / FAIL 0 / 豁免 2）
scripts/self_check.py	算法层自检	算法变更后	✅ **69/69**
scripts/eval_report.py	指标评估（--export-series）	**模型变更后必跑**	✅ 已产出

🔒 **GIT-7 纪律**：修改算法参数后**必须**重跑 train_and_register.py 与 eval_report.py --horizon 14 --export-series，否则界面数字会落后于模型（本项目历史上出现过「两套数字」问题）。

9.2 交付前验收检查清单（15 项，逐项确认）

已完成的检查项（EP-9 结束时的状态）：第 1~11 项、第 13 项、第 14 项、第 16~18 项均已由自动化脚本逐项证明。其中第 3 项在 EP-9 扩展为 **35 个脚本的全量回归**（含 EP-9 新增 5 套 + a11y_audit.py），全部**实跑**通过（见测试报告 §2.11）。

EP-9 收口结果：第 4 项（无障碍）由 a11y_audit.py 退出码 0 证明；第 14 项（🔒 免责声明全局可达）由 verify_ep3_browser.mjs 走查 **4 个模板**逐页断言（存在 / 可见 / 未折叠 / 未截断 / 文案完整）；第 12 项（四浏览器）**部分达成** —— Edge 在 1366×768 与 1600×900 双视口走查 **109/109**，Chrome / Firefox / Safari 因本机未安装而由脚本**如实登记为 BLOCKED**（而非伪装通过），已登记为测试报告 P1-01。第 15 项（/docs 渲染）留待 EP-10 复核。

#	检查项	判定方式	状态

#	检查项	判定方式	状态
1	Mock 开关已关闭	搜索 `VITE_USE_MOCK`，确认为 0 或不存在	☑ EP-7
2	原型痕迹已删除	`.state-bar` / `.spec-note` / `.diff-note` / `REAL_EVAL` / `README.html` 零命中	☑ EP-7（`verify_ep7.py` 断言）
3	校验脚本全绿	见 §9.1	☑ EP-9（**35 套全量实跑**，见测试报告 §2.11）；EP-10 收口时口径为 **38 套**（新增 `verify_docs_browser.mjs`、`verify_demo_data.py`、`verify_doc_exports.py`），计数规则 = `verify_*` 前缀 35 个 + `a11y_audit.py` + `self_check.py` + `eval_report.py`
4	无障碍审计通过	`a11y_audit.py` 退出码 0	☑ EP-9（**退出码 0**：53 项，PASS 51 / FAIL 0 / 豁免 2）
5	🔒 结果页指标与算法一致	对照 `outputs/eval_report.json` 逐项核对	☑ EP-6
6	🔒 覆盖率三项之和 = `total`	数据核对	☑ EP-6
7	🔒 主干路径 ≤ 5 步	走查计数	☑ EP-7（上传→参数→执行→结果→历史→对比 = 6 页内闭环）
8	所有错误提示含「错在哪 + 怎么改」	人工走查 ≥ 5 类	☑ EP-7（浏览器脚本覆盖上传 5 类 + 预测 4 类 + 历史删除四态）
9	🔒 MAPE 超标警告不可折叠	交互走查	☑ EP-6
10	🔒 L3 序列无数值预测	用降级数据走查	☑ EP-6（`case_l3_sparse`）
11	导出 CSV 中文不乱码	用 Excel 实际打开	☑ EP-6（`openpyxl` 读回）
12	四种浏览器可正常使用	Chrome / Firefox / Safari / Edge 最新版	☑ EP-9（**Edge / Chrome / Firefox 各 109/109**，1366×768 + 1600×900 双视口、控制台 0 错误；Safari 在 Windows 无发行版 → 平

#	检查项	判定方式	状态
			台限制，见 P1-01）
13	1366×768 布局正常	缩放到该分辨率走查	✅ EP-7（hide-md 断点 1080px 实测）
14	🔒 免责声明全局可达	任意页面可见页脚声明	✅ EP-9（verify_ep3_browser.mjs 走查 **4 个模板**逐页断言，47/47）
15	接口文档可访问	/docs 正常渲染（Swagger 资源已离线化）	☐ EP-10
16	🔒 **CH-01：非管理员访问管理端返回 403**	curl 实测	✅ EP-8（**61/61** 浏览器 + **96/96** 接口）
17	🔒 **CH-01：管理端写操作全部留痕**	查 operation_logs 中 action='admin'	✅ EP-8（6 类子动作，与业务同事务）
18	🔒 **CH-01：§5.7 的 18 项文档回写完成**	逐项勾选	✅ EP-8（verify_ep8.py ⑱ 断言）

第 1~15 项沿用技术文档 §9.6；第 16~18 项为 CH-01 新增。

截至 EP-8 的滚动状态：

- 第 **1** 项（Mock 开关）：✅ 已满足 —— web/ 全量搜索 VITE_USE_MOCK **0 命中**，且首页指标改为从 outputs/eval_report.json 构建期注入（QD-06），不存在「界面一套、算法一套」。

- 第 **2** 项（原型痕迹）：✅ 已满足 —— verify_ep3.py 的 ②-1/②-2 断言 web/src 下 .state-bar/.spec-note/.diff-note/原型演示指标常量 **零命中**，README.html 未迁移。

- 第 **14** 项（🔒 免责声明全局可达）：✅ 已满足 —— verify_ep3_browser.mjs 走查 **4 个模板**，均断言存在 / 可见 / 未折叠 / 未截断 / 文案完整（111 字含 3 个关键句）。

- 第 **16** 项：✅ 已满足 —— **两种拒绝不混用**：未登录 → 401（引导去登录），已登录但非管理员 → 40301（引导去要权限）。把「没登录」报成「没权限」会让用户白折腾，故专门断言两者不同。

- 第 **17** 项：✅ 已满足 —— 6 类子动作 enable / disable / role_change / reset_pwd / hard_delete / switch_model 均写 action='admin'，detail 格式 sub=..;target=..;by=..，且**与业务同事务**（业务回滚则审计一起回滚，不会出现「操作失败却留下成功记录」）。

- 第 **18** 项：✅ 已满足 —— §5.7 的 18 项已逐项回写（PRD / 技术文档 / UI/UX / 实施方案 / schema.sql / schemas.py），并由 verify_ep8.py 的第 ⑱ 节反向断言（文档里没有就挂）。

- 其余各项待对应阶段完成后勾选（第 5/6/9/10 项属 EP-6，第 11 项属 EP-6，第 15 项属 EP-10）。

EP-9 新增滚动状态：

- 第 **4** 项（无障碍审计）：✅ 已满足 —— `scripts/a11y_audit.py` 退出码 **0**，浅深双主题合计 FAIL **0**，证据 `outputs/ep9_a11y_log.txt`。
- 第 **12** 项（四浏览器）：◌ **部分达成** —— Edge 双视口 **109/109**（含控制台零错误）；Chrome / Firefox / Safari **本机无该内核**，脚本把缺失项登记为 `BLOCKED` 而非通过（这是刻意的：把「没测」写成「通过」是测试报告最严重的失真）。Firefox 不实现 CDP，须另接 WebDriver BiDi；Safari 不在 Windows 发行。
- 第 **14** 项（🔒 免责声明全局可达）：✅ 已满足 —— 4 个模板（营销 / 认证 / 主内容+右栏 / 应用页）逐页断言存在、可见、未折叠、未截断、文案完整。
- 第 **3** 项（校验脚本全绿）：✅ 已满足且**口径升级** —— EP-9 期间改动源码后，不再是「各阶段当时跑过」，而是**一次性全量实跑 35 套**（测试报告 §2.11）。这次回归直接发现了 `F-05`（`API-13` 越权码与全局不一致）。

9.3 需求覆盖自检

项	数量	本期	核对方式
PRD `FR-*` P0	55	✅ 全量	逐条对 `EP-*` 映射（§1.2.2）
PRD `FR-*` P1	14	✅ 全量	同上
PRD `NFR-*`	38	✅ 全量	分布在各阶段 `DOD` 中
`CH-01` 管理端	2 页面 / 9 接口 / 1 列	✅ 全量	§5 逐项
🔒 硬指标	4 项	✅ 已达标	⏱ 14.26% / 26.42% / 27.52% / 83.95%

---

10. 风险登记册

编号	风险	概率	影响	应对措施	归属阶段

编号	风险	概率	影响	应对措施	归属阶段
RG-01	算法指标不达标（MAPE > 20%）	低	高	⏱ 已达标（14.26%，余量 5.7 pp）；保留回退到已注册模型的能力	已完成
RG-02	端到端超 60 s	低	高	⏱ 实测 7.9 s（余量 7.6 倍）；序列上限 1000 + 超时软失败兜底	EP-5
RG-03	设计令牌移植走样（改写成 hex / 图表不重绘）	中	中	🔒 `tokens.css` 保留 `oklch()`；ECharts 主题从 CSS 变量读取 + 主题事件重绘；`DOD-EP0` 用脚本校验	EP-0
RG-04	**编号漂移复发**（文档引用 `FR-*` 语义错位）	中	中	📌 常驻守卫 `verify_doc_refs.py`；本项目曾在数据库文档查出约 30 处错位，**肉眼不可见，必须靠脚本**	全程
RG-05	🔒 **CH-01 破坏数据隔离红线**	中	**高**	例外仅限 `admin.py`（`BE-ADM-01`）+ 强制 `get_current_admin` + 全操作审计 + `DOD-EP8` 红灯自检 grep	✅ **EP-8 已闭环**（AST 断言：跨用户查询仅现于 `admin.py`；9 路由全挂 `CurrentAdmin`）
RG-06	bcrypt 72 字节导致 500	低	中	`schemas.py` 已内置字节校验 + 6 条回归用例（含 72/73 边界）	EP-2
RG-07	`NOCASE` 索引静默失效（`WHERE email=?` 全表扫描）	中	中	🔒 封装 `get_user_by_email()` 单点；`explain_queries.py` 守。**危险在于本机数据量小、两种写法都是微秒级，压测发现不了**	EP-1
RG-08	管理端种子账号无法登录（`EmailStr` 拒绝保留域）	中	中	⏱ 已实测确认陷阱；🔒 种子邮箱用 `admin@salesforecast.cn`（§5.3.3）	✅ **EP-8 已闭环**（浏览器脚本实测登录成功并进`/admin`）
RG-09	2 GB 显存被	低	中	`FR-MOD-10` CUDA→CPU 自	EP-5

编号	风险	概率	影响	应对措施	归属阶段
	占用导致 OOM			动回退；🔒 **不是简单重试**，而是 `empty_cache` → `.to('cpu')` → 重跑	
RG-10	SQLite 并发写锁	低	中	WAL 模式 + 批量插入 + `check_same_thread=False`；Uvicorn 单进程	EP-1 EP-9
RG-11	管理端两页无设计依据 → 视觉不一致	**高**	中	🔒 复用模板 A + 既有组件，**不新增设计令牌**；先出线框再实现（QD-01）	✅ **EP-8 已闭环**（零新令牌，`verify_ep8.py` 断言；复用 `MetricCard`/`EChart`/`NoticeBar`/`Skeleton`）
RG-12	前端工程从零搭建（依赖体积 / 国内网络）	中	中	npm 淘宝源；版本锁定；`node_modules` 不入库	EP-0
RG-13	单人开发跨度长导致返工	中	中	每阶段「可运行可演示」收口；严格串行；阶段顺序化	全程
RG-14	Word COM 导出 PDF 超时残留孤儿进程	低	低	显式长超时；核对 `StartTime` 确认归属后再清理（本项目已踩坑）	EP-10
RG-15	演示现场失败（数据 / 环境）	中	高	演示数据冻结；预跑一遍；准备失败分支的演示预案	EP-10
RG-16	🟤 **dev server 代理目标与后端端口不匹配 → 页面全空但不报错**	中	中	⏱ **EP-8 实测踩坑**：vite 代理缺省指向 **8000**，测试后端跑在 8021，表现为「页面能渲染、无 console error、但所有表格 0 行」，**后端日志一条请求都没有。**🔒 启动 dev server 必须带 `VITE_API_TARGET`；🔒 判断代理通不通**看后端日志有没有请求**（零请求 = 代理配错，别去查前端代码）；启动前先 `netstat` 确认端口无残留旧进程	全程
RG-17	**停用类需求**	中	**高**	⏱ **EP-8 实测踩坑**：`login()`	全程

编号	风险	概率	影响	应对措施	归属阶段
	漏堵"重新登录"这条路			从未检查 `user.status`，停用用户重新登录即可绕过 `API-24`，且完全绕开审计。🔒 封禁必须同时堵三条路：旧 Token（`token_version`）、重新登录（`status` 检查）、API 直调（`get_current_admin`）；`verify_ep8.py` / `verify_ep8_api.py` 已固化为常驻断言	

---

11. 协作方式与变更管理

11.1 单人开发的节奏（📌 决策 4：不排日历）

技术文档 §8.7 已给出按阶段的建议周期，**本期单人开发、不承诺日历日期**，因此将其转为**工作量权重**用于把握节奏（相对值，非承诺）：

阶段	相对权重	说明
`EP-0` 基建	★	一次性投入，做扎实后后续所有阶段受益
`EP-1` 数据库	★★	含 `CH-01` 变更与迁移脚本
`EP-2` 认证	★★	含限流与字节校验边界
`EP-3` 前台	★	静态内容为主，风险最低
`EP-4` 上传校验	★★★	7 步校验 + 5 类错误，细节密集
`EP-5` 预测编排 ★	★★★★	**最重**：状态机 + 批量推理 + 回退 + 超时

阶段	相对权重	说明
EP-6 结果可视化 ★	★★★★	**最重**：图表 + 指标口径 + 导出
EP-7 记录类	★★	3 页 + 对比逻辑
EP-8 管理端	★★★	9 接口 + 2 页面 + 18 项文档回写
EP-9 联调测试	★★★	分支 + 边界 + 性能 + 兼容
EP-10 文档答辩	★★	6 份文档收敛 + PPT

落地纪律：

#	纪律
1	**严格串行**：未通过上一阶段 DOD 不进下一阶段
2	**阶段内并行**：后端接口自测通过后再写对应前端页面（避免同时面对两端问题）
3	**每阶段一次演示**：自己走一遍「可运行、可演示」，留下截图或录屏
4	**提交前自检**：跑 §9.1 相关脚本 + `ruff` + `npm run typecheck`（技术文档 GIT-6）
5	**改算法必重跑**：`train_and_register.py` + `eval_report.py`（GIT-7 🔒）

11.2 变更管理（沿用技术文档 §8.6）

变更类型	处理流程	本次实例
新增接口	更新接口清单 + `schemas.py` + `types.ts` + `api-client.ts`，跑校验	CH-01 的 9 个接口
修改表结构	更新 `schema.sql` + 数据库文档 + 迁移脚本	CH-01 的 `role` 列与 `action` 值集
修改算法参数	更新 `algo/config.py`，**必须重跑训练与评估**	本期无
修改设计令牌	按 UI/UX §12.6 流程，重跑 `a11y_audit.py`	本期无

变更类型	处理流程	本次实例
调整需求范围	以 PRD 优先级表为唯一基准；**本次需回写 PRD**	CH-01（§5.7）

11.3 阶段交付方式

每阶段结束须提交	内容
1. 可运行代码	提交到版本库；能启动、能走通本阶段链路
2. DOD 勾选表	逐条勾选 + 证据（命令输出 / 截图 / 录屏）
3. 遗留问题	明确列出未解决项与原因（不得隐瞒）
4. 文档同步	若涉及接口 / 表结构 / 视觉，同步更新对应文档

---

12. 交付物清单

类别	交付物	位置	状态
代码	后端 app/（Web 层新建，algo/ 复用）	工作区	✅ **已实现**（EP-1~EP-10，含管理端 admin 模块与 /docs · /redoc 离线化）
	前端 web/（React 工程，13 页）	工作区	✅ **13 页已实现**（含管理端 AD-P01/AD-P02）
	配套资产 code/（schema / seed / schemas / types / api-client）	工作区	✅ 已同步至 EP-8（含 CH-01 管理端契约）
	离线静态资源 app/static/vendor/（Swagger UI 5.33.1 + ReDoc 2.1.5）	工作区	✅ **EP-10 新增**（随仓库冻结，国内受限网络下开箱可用）
数据库	data/app.db（8 表 / 121 列 / 1 视图 / 3 触发器 / 12 索引）	工作区	✅ 已初始化（EP-1，EP-7 视图加 dataset_id）

类别	交付物	位置	状态
	`code/db/schema.sql` + `seed.sql`（含 `CH-01`）	工作区	✅ 已有（`EP-7` 已同步视图变更）
模型	`outputs/models/lstm_v1.pt` + `report.json`	工作区	✅ 已有（`EP-10` 复核：权重 128 KB / 31,729 参数 / 峰值显存 111 MB）
	`outputs/models/demo_sales.csv`（示例数据集）	工作区	✅ 已有
文档	任务书 / 开发方案文档（内容 V1.1）/ PRD / UI/UX / 开发技术 / 数据库设计	`docs/`	✅ 已有（6 份）
	项目开发实施方案（本文档）	`docs/`	✅ 本版产出
	测试报告	`docs/`	✅ **已产出**（`EP-9`：《电商销量预测系统_测试报告_V1.0》）
	部署运维手册	`docs/`	✅ **已产出**（`EP-10`：《…部署运维手册_V1.0》，12 章 + 2 附录）
	AI 协作教程	`docs/`	✅ **已产出**（`EP-10`：《…AI 协作教程_V1.0》）
	RAG 语料库与提示词	`docs/`	✅ **已产出**（`EP-10`：《…RAG 语料库与提示词_V1.0》，**40 个语料块 / 9 主题 / 5 组提示词**）
	项目说明（参数文档）	`docs/`	✅ **已产出**（`EP-10`：《…项目说明_V1.0》，14 章 / 14 张参数表）
	演示脚本	`docs/`	✅ **已产出**（`EP-10`：《…演示脚本_V1.0》，含台词 / 备用问答 / 故障预案）
	项目文件总览与交付清单	`docs/`	✅ **已产出**（工作区整理新增：《…项目文件总览与交付清单_V1.0》，14 章；做 §12 的**磁盘落地映射**——逐目录列出文件 / 用途 / 是否交付 / 是否可再生 + 整理记录 + 维护约定。⚠ 交付范围与验收口径**仍以本 §12 为准**，该文档只做索引与记录）
	设计图 `docs/assets/db-design/`（7 张）+ 文档截图 `docs/assets/eval/`	`docs/`	✅ 已有

类别	交付物	位置	状态
	全部文档的 Word / PDF 导出件	outputs/	✅ **已产出**（EP-10 定型 + 工作区整理补入《项目文件总览与交付清单》：**14 份 .docx + 14 份 .pdf**，均含真 Heading 样式、TOC 域已展开、页眉页脚页码、Markdown 表格转真 Word 表格；合计 **769 页。**导出流水线固化为 scripts/docx_export.sh（stage / collect / list，中文路径搬运）+ scripts/run_pdf.ps1（纯 ASCII 暂存目录内跑 Word COM）+ scripts/verify_doc_exports.py（齐备性 / 新鲜度 / TOC 域守卫）
API 接口	outputs/电商销量预测系统_openapi.json	outputs/	✅ **已产出**（EP-10：**30 路径 / 32 操作 / 78 Schema / OpenAPI 3.1**）
	outputs/电商销量预测系统_API 接口文档_V1.0.html	outputs/	✅ **已产出**（EP-10：**单文件自包含，1.75 MB，零外部引用**）
	在线文档 /docs（Swagger UI）与 /redoc（ReDoc）	运行时	✅ **已离线化**（EP-10：真实浏览器 Edge · Chrome 各 **16/16**）
质量	校验脚本（**38 套**：算法 / 静态 / 接口 / 浏览器 / 无障碍 / 契约 / 文档）	scripts/	✅ **全量登记于 verify_ep9.py::CHAIN，反向登记检查 0 漏网**（测试报告 §2.11） 计数口径：verify_* 前缀 **35** 个 + a11y_audit.py + self_check.py + eval_report.py = **38**；其中 **37 套**属 verify_ep9.py::CHAIN 的反向登记检查范围（eval_report.py 是算法评测入口、非断言脚本，不入 CHAIN）
	outputs/ep9_* / ep10_* 证据文件（性能 / 契约 / 链 / 文档 / 演示）	outputs/	✅ 已落盘
答辩	演示 PPT（18 页）	outputs/	✅ **已产出**（EP-10：《电商销量预测系统_答辩演示_V1.0.pptx》，7 张内嵌真实截图

类别	交付物	位置	状态
			+ 1 个原生图表；`slidep` 校验 0 诊断 + PowerPoint 实渲染 18/18）
	演示脚本 + 冻结的演示数据	`docs/` + `demo/`	✅ **已产出**（`EP-10`：`demo/演示数据_正常.csv`（2,229 行 / 9 序列 / 逐字节可复现）+ 2 个失败分支数据集；`verify_demo_data` **50/50**，主链路自动化 **1.0 s**）

附录 A：立项决策记录（📌 7 条已确认 + 2 条管理端细化）

本附录为本次立项的**决策快照**。执行期以本表为准，不再回溯讨论。

#	决策项	结论	影响面
1	「前台企业官网」范围	**沿用现有 11 页设计**；前台 = `UX-P01`~`P03`	不新增前端页面设计
2	是否需要管理员端	**需要**，档位 **B 标准运营型**	新增 `CH-01`
3	管理员账号产生方式	`seed.sql` **内置种子账号**（幂等）	零新增注册入口
4	团队与工期	**单人开发，不排日历**	阶段只给顺序与门槛
5	本期实现范围	**P0（55）+ P1（14）全量**	P2 / P3 不排期
6	本轮交付	**只出实施方案，不写代码**	待「已确认，执行下一步」
7	技术栈	**不改动**（FastAPI + React，版本锁定）	编码期不得更换框架
8	管理端数据范围	**B 档**：模型版本切换 + 任务强制删除 + 用户重置密码 + 只读看板与审计	9 接口 / 2 页面 / 1 列
9	管理端安全边界	数据隔离红线的**唯一显式例外**，限定在 `admin.py`，全	`NFR-SEC-02` 修订

#	决策项	结论	影响面
		操作审计	

附录 B：待确认事项清单（QD-*）

本清单接续 PRD 附录 A（Q1~Q18）与开发技术文档附录 B（TQ-1~TQ-16），编号不重起。

所有 QD 项均已给出建议默认值，不阻塞 EP-0 启动；负责人可在回复「已确认，执行下一步」时一并拍板。

编号	问题	建议默认值	阻塞级别
QD-01	管理端入口与导航归属？（侧栏新增分组 vs 顶栏用户菜单）	⚠ **顶栏用户菜单增「管理后台」项，仅 `isAdmin` 可见。**理由：保持 IA 的「4 组 / 8 导航项」不变，避免普通用户看到空分组	P3
QD-02	`operation_logs.action` 扩展方式？（+1 值 vs +4 独立值）	⚠ **+1 值 `'admin'`（共 8 类）**，子动作写 `detail`。理由：CHECK 保持紧凑，管理端操作量小	P3
QD-03	管理端「强制删除任务」是硬删还是软删？	⚠ **硬删**（物理删除 + 级联 `predictions`/`metrics`）+ 审计留痕。理由：与用户侧软删除形成能力区分，真正释放存储	P2
QD-04	管理员能否查看用户上传的原始 CSV 内容？	⚠ **默认不可见**，只给元信息（文件名 / 大小 / 行数 / 商品数 / 仓库数）。理由：最小暴露原则，避免隐私越界	P2
QD-05	重置密码后是否强制用户首次登录改密？	⚠ **不新增字段。**做法：重置时 `token_version + 1` 使旧 Token 失效，新密码由	P2

编号	问题	建议默认值	阻塞级别
		管理员一次性告知。理由：避免再加 `must_change_pwd` 列	
QD-06	前台首页硬指标数字从哪来?	⚠ **从 `outputs/eval_report.json` 读取**（构建期注入或接口读取），**禁止手写。**理由：防「两套数字」复发	P1
QD-07	管理端接口是 8 个还是 9 个?	⚠ **9 个。** `API-28`（模型列表 GET）与 `API-29`（切换 PATCH）必须分离，避免单路径承担读写两种语义	P3
QD-08	`CH-01` 是否回写 PRD / UI/UX 等上游文档?	⚠ **必须回写**（§5.7 共 18 项）。变更流程强制要求；PRD `Q13` 需从「待确认」改为「已确认」	P0（属 `DOD-EP8`）
QD-09	PRD 自身的计数差异（§3.8 汇总 77 项 vs 编号去重 76 条）是否本轮修正?	⚠ **本轮一并修正。**理由：`CH-01` 已需回写 PRD，顺手收敛该差异	P3
QD-10	管理端埋点是否需要独立图表看板?	⚠ **B 档内做 1 折线（MAPE 走势）+ 1 饼图（事件占比）**，其余留待二期	P3

沿用上游仍开放项（不影响本期）：

编号	内容	来源
TQ-2	是否引入 Redis 缓存？（⚠ 若改为多进程部署，内存黑名单方案必须升级）	开发技术文档
TQ-12	`events` 埋点是否 90 天归档?	数据库设计文档
TQ-13	`model_registry` 保留几个历史版本？（建议 3 个）	数据库设计文档
TQ-15	游客数据（`user_id=0`）是否保留 30	数据库设计文档

编号	内容	来源
	天后清理?	
TQ-16	metrics.l2_mape 为 NaN 时是否在 eval_note 说明?	数据库设计文档
TD-DB-01	predictions 超过 100 万行时需重评性能	数据库设计文档
TD-DB-03	无自动化迁移工具，靠 PRAGMA user_version 手工管理	数据库设计文档
TD-DB-04	metrics.baseline_mape 为预留列，算法不产出，恒 NULL	数据库设计文档

附录 C：执行顺序速查（一页纸）

```
① EP-0 工程基建          → /health 通 + 前端空布局 + 56 令牌移植          ✅ 2026-10-09
② EP-1 数据库落地        → data/app.db + CH-01(role/action) + ORM/DAO     ✅ 2026-10-09
③ EP-2 认证              → 注册登录改密 + 限流 + get_current_admin 骨架   ✅ 2026-10-09
④ EP-3 前台首页          → UX-P01 + 免责声明 + 主题切换 + 硬指标同源      ✅ 2026-10-09
⑤ EP-4 上传与校验        → 7 步校验 + 5 类错误 + 游客态 + 断档二选一      ✅ 2026-10-09
⑥ EP-5 预测编排 ★        → 先脚本后 Web + 状态机 + 批量推理 + 回退 + 超时 ✅ 2026-10-10
⑦ EP-6 结果可视化 ★      → 指标口径(加权 MAPE/L1) + 覆盖率 + 图表 + 导出  ✅ 2026-10-10
⑧ EP-7 记录类            → 历史/详情/对比(路由顺序!) + 工作台             ✅ 2026-10-10
⑨ EP-8 管理端(CH-01)     → 9 接口 + 2 页面 + 全审计 + 18 项文档回写       ✅ 2026-10-10
⑩ EP-9 联调测试          → 主干 + B1~B14 + 边界 + 性能 + 兼容 + a11y      ✅ 2026-10-12
⑪ EP-10 文档答辩         → 测试报告 + 部署手册 + API 文档 + PPT + 演示    ✅ 2026-10-13
```

每阶段必做：勾 DOD → 跑校验脚本 → 留证据 → 列遗留问题 → 收口演示。

当前进度：EP-0 ~ EP-10 全部完成（11/11），交付物清单（§12）已 100% 齐备。

里程碑 MS-1~MS-8 全部达成或已登记为可接受遗留；前台 11 页 + 管理端 2 页 + 6 个后端模块已可用，**业务闭环、质量达标、交付就绪三条线全部收口。**EP-10 唯一未达成项为 DOD-EP9 第 4 条的 Safari 实测（本机为 Windows，无该内核），已转遗留问题 P1-01。

十八条最易踩的坑（本项目已验证）、

#	坑	后果	防法
1	用户侧「错误明细」与正式导出未分目录	临时文件被 7 天清理误保留	分目录 exports/tmp/，TTL 1 小时
2	POST /history/compare 注册在 /history/{task_id} 之后	compare 被当 task_id，报 40012	路由注册顺序固定
3	管理端种子邮箱用 .invalid 域	管理员永远无法登录（EmailStr 先拒）	用 admin@salesforecast.cn
4	**services/ 里做完状态变更后 raise BizError，指望 get_db 提交**	get_db 契约是「异常→rollback」，**变更被回滚。**EP-2 的实测后果是**登录限流永不生效**（fail_count 每次归零）；EP-4 的实测后果是**五类校验失败一条记录都不落库**，API-11 的错误明细永远拿不到；EP-5 的实测后果是**超时软失败任务查不到**	需要保留的副作用**在抛错前显式 db.commit()**（BE-17 的正当例外）。判据：这条缺陷**在单次请求的返回值上看不出来** —— 接口该报的错都报对了，只有再查一次库才露馅（EP-4 的原话是「复现得了但查不到」）
5	用 datetime.min / 0 之类的「哨兵时间」表示「未锁定」	「判断是否已解锁」的 remain <= 0 会同时命中「根本没锁」→ 误删正在累积的计数 → **限流静默失效**（EP-2 实测）	「计数中」与「已锁定」必须用**不同的类型**表达（如 None vs datetime）
6	**图表依赖 ECharts「默认色板按系列序号取色」**	只要有「不可见/占位」系列（如置信带），线条拿到的颜色就会后移，**自绘图例与图表颜色对不上**（EP-3 实测：线条是橙/红，图例画的是青/绿）	需要与图例严格一致时，系列**显式绑定令牌色**（readTokenVar('--c1')），并让 option 以主题为依赖重建
7	**主题相关逻辑写在 store 里、且 store 静态 import 了重型图表库**	① 图表库被打进主包，懒加载形同虚设（EP-3 实测主包被 ECharts 撑到 1.38 MB）；② 注册发生在 CSS 注入之前 → getComputedStyle 取到**空色板**	让**唯一使用者**在 init() 前同步注册（图表组件内），store 只维护 isDark 状态；主包随即从 1381 kB 降到 343 kB
8	**拿正则扫源码做「痕迹/字面量」守卫，却不先剥注释**	注释里写「已移除 X」反而被判违规 —— 守卫变成噪音，最后被人关掉（EP-1 已踩过一次，EP-3 又踩，EP-5 第三次）	守卫一律先剥 // 与 /* */ 再扫；能用 AST 就别用正则（EP-4 已改用 ast.walk 取真实 import）
9	**无头浏览器复用固定**	上次残留的子进程仍持有 profile 锁 → 新实例	① 每次用全新 profile 目录；② 退出时

#	坑	后果	防法
	`--user-data-dir`，且只 `edge.kill()` 父进程	**静默挂起**：不报错、不产出一张截图、脚本空转 15 分钟（EP-4 实测）。任何「一直不结束也不报错」的自动化都先怀疑资源锁	`taskkill /PID <pid> /T /F` 杀**进程树**；③ 调试端口按 PID 分散并在起跑前自检未被占用；④ 给单次截图套 15 秒上限（`captureScreenshot` 的 CDP 超时是 120 秒，先挂两次就是 4 分钟）
10	**用同一族选择器取值（`[data-testid=x] span`）或按下标取兄弟节点（`aside[1]`）**	把不该收的节点一起收进来：徽标的 `span` 把内部 ✓/✗ 图标也收了（长度翻倍）；模板 B 的 `aside[0]` 是导航侧栏、右栏其实在 `[1]` —— 断言会稳定地取到错误对象（EP-4 实测两类各一次，EP-5 又在 `data-testid` 落点与选择器层级上踩了 4 次）	选择器限定到**承载语义的那一层**；兄弟节点**按内容查找**而不是按下标。⚠ 更根本的防法是**先给组件加稳定的 `data-testid`**，而不是让走查脚本去猜 DOM 结构
11	🟣 **滚动多步推理里，每步都调「整条序列特征表」构建器**	同一条序列被重复建表 `horizon` 次，`horizon=30` 实测 **72.2 s —— 击穿 60 s 硬约束**（EP-5 实测）。且耗时随步长**超线性**增长（h=7 17.5 s → h=30 63.9 s 软失败）	把「取尾窗」从 pandas 路径里拆出来：用纯 numpy 一次算好日历矩阵 + 向量化取窗（`features.calendar_matrix` / `last_window_fast`），降到 **23.8 s**。⚠ 两条配套：① `_trailing_mean` **必须用 `sliding_window_view`**，用 `cumsum` 差分会引入累加误差、破坏与 pandas 的逐位相等；② 先试「只取尾部切片」只省 11% —— 瓶颈是 pandas 每次调用的**固定开销**，不是行数
12	🟣 **同一族概念存在两个「耗时」口径却共用一个字段**	「单遍评测 7,873 ms」与「任务端到端 16,618 ms」差一倍以上，共用一个字段会让参数页显示「预估 7.9 s、实际 16.6 s」；更糟的是**校验脚本拿 7,873 去比端到端预估，得出「偏差 99%」的假告警**	并列两个字段并各自命名语义：单遍 `estimated_ms`（评测基准）+ `estimate_by_horizon[]`（端到端预估）。**每个耗时字段都要在文档里写明它是哪一遍**（EP-5 登记为口径偏离 ③）
13	🟣 **同进程改 `Settings` 属性来构造「换个配置」的用例**	`@dataclass(frozen=True)` ⇒ 直接 `FrozenInstanceError`；若改成非 frozen 又会污染后续用例	「换个配置再跑一次」只能**独立子进程 + 环境变量**（EP-5 的 `scripts/_ep5_cfg_probe.py`）。同理，**模块级单例**（如模型 holder）也不能同进程换设备 —— CPU/GPU 一致性验证必须两个进程各跑一份，否则比对到的是缓存句柄的「假一致」
14	🟣 **按 RESTful 直觉断言业务错误的状态码**	本项目契约是 **HTTP 200 + `body.code`**（除 401 类，见 `errors.py::register_exception_handlers`），判据写 `status == 400` 会**全部假失败**，而真正要拦的红线（软失败不得退化成 5xx）反而没被测到	断言 `body.code`；并单独加一条「业务错误**不是** 4xx/5xx」的守卫，防止有人把契约改成 RESTful（EP-5 踩过，8 处判据错误里最易复发的一条）

#	坑	后果	防法
15	🟣 **依赖注入写成 `user=CurrentUser`（默认值形式）而非 `user: CurrentUser`（注解形式）**	传进去的是**类本身**不是实例，`user.id` 拿到的是 SQLAlchemy 的 `InstrumentedAttribute` —— 不报错，一路带到 DAO 层才炸，且报错信息完全指不到真正的原因	`Annotated[X, Depends(...)]` 只能走**注解形式**。判据：FastAPI 依赖用默认值写法，本质上就是「给形参声明了一个默认值」，语法合法、类型检查也过得去（`EP-7` 踩过，4 处）
16	🟣 **服务层加了返回字段，但 schema 没声明**	**pydantic 静默丢弃**未声明字段：不报错、不告警，响应里就是没有。前端拿到 `undefined`，看起来像「后端没存失败原因」，排查方向会整个跑偏	服务层返回结构与 schema **逐字段对齐**；schema 变更后跑 `verify_schemas.py`。判据：任何「后端明明返回了、前端就是拿不到」的现象，第一反应查 schema 声明（`EP-7` 实测：`TaskBrief` 丢 `error_code`/`error_msg`/`completed_stage`）
17	🟣 **「列表里过滤 + 详情里不过滤」用了不同口径**	软删除的任务从列表消失了，直接输 `/history/{id}` 却还能打开（`predictions` 还在库里）。用户视角是「删了但没删掉」，且同一任务两处口径矛盾	凡「列表过滤、详情不过滤」的字段，两处**必须同步**判定。修法要点：**判不存在**与**判幂等**要拆开 —— `_owned_task` 一律把软删除视为不存在（返 `40401`），而软删除的幂等分支改走 `get_by_id` 自判，不复用它（`EP-7` 实测，见 §7.8.2 ③）
18	🟣 **DAO 层的删除类函数从写完起就没人调用过**	`soft_delete` 用 `{Task.is_deleted: 1}` 作字典键（须为字符串列名），SQLAlchemy 2.0 抛 `InstrumentedAttribute is not supported`。潜伏到 `API-20` 第一次真正调用才暴露	DAO 写完≠被调用过。**删除类函数天然「最后才跑一次」**，最容易藏笔误 —— 且它往往同时是另一处口径漏洞（#17）的检测点。判据：新增写操作 DAO 时，手工构造一次调用断言，不要指望集成阶段才发现

文档结束

本方案覆盖 11 个实施阶段、2 个最重阶段（`EP-5`/`EP-6`）、1 个变更项（`CH-01`，含 9 接口 / 2 页面 / 1 列 / 18 项文档回写）、15 条风险、10 条待确认项、18 项交付前验收。

所有关于数据库变更与安全边界的断言均经 **`sqlite3` 实测**（§5.3）与 **Pydantic 实跑**（§5.3.3），非纸面推导。

本方案与上游文档冲突时，**以上游为准**（§0.3 冲突仲裁规则）。